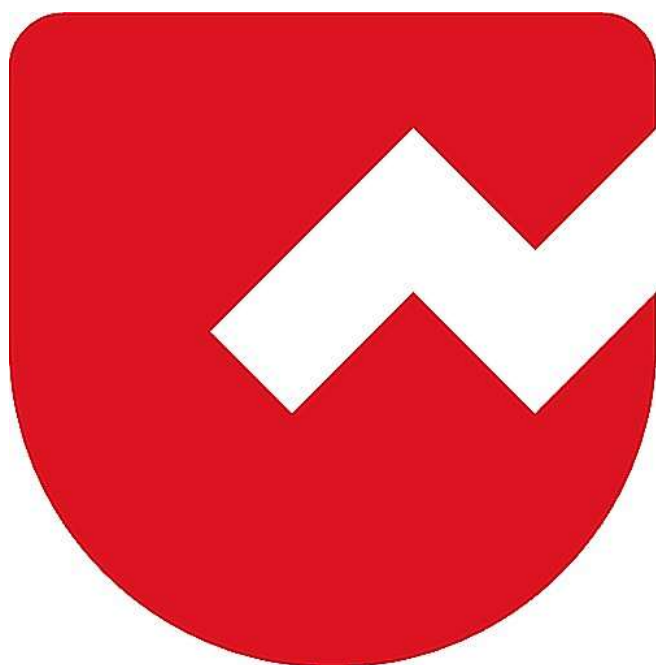




Zoho Schools for Graduate Studies



Notes

Day-1

PROGRAMMING BASIC PART -1

1) Why the parameter of the main method is a String [], not float []/ int []?

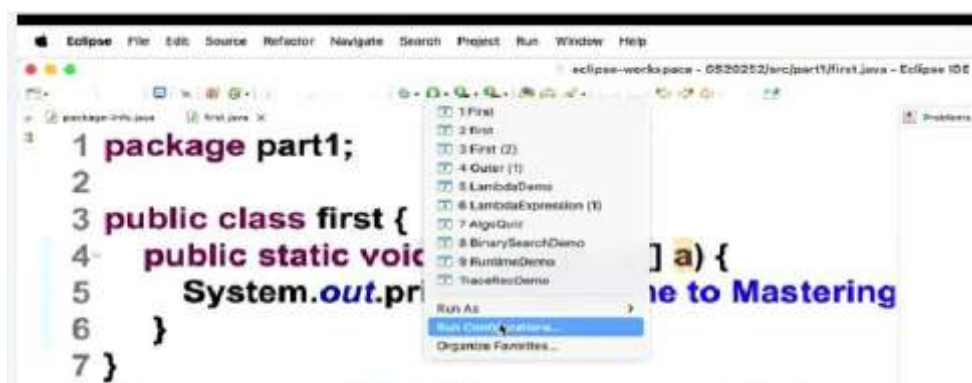
- ⇒ String [] can accommodate any datatype after conversion.
- ⇒ It accepts all 6 numeric datatypes and any object type variables.

2) Why do we use command-line arguments instead of getting inputs from the user?

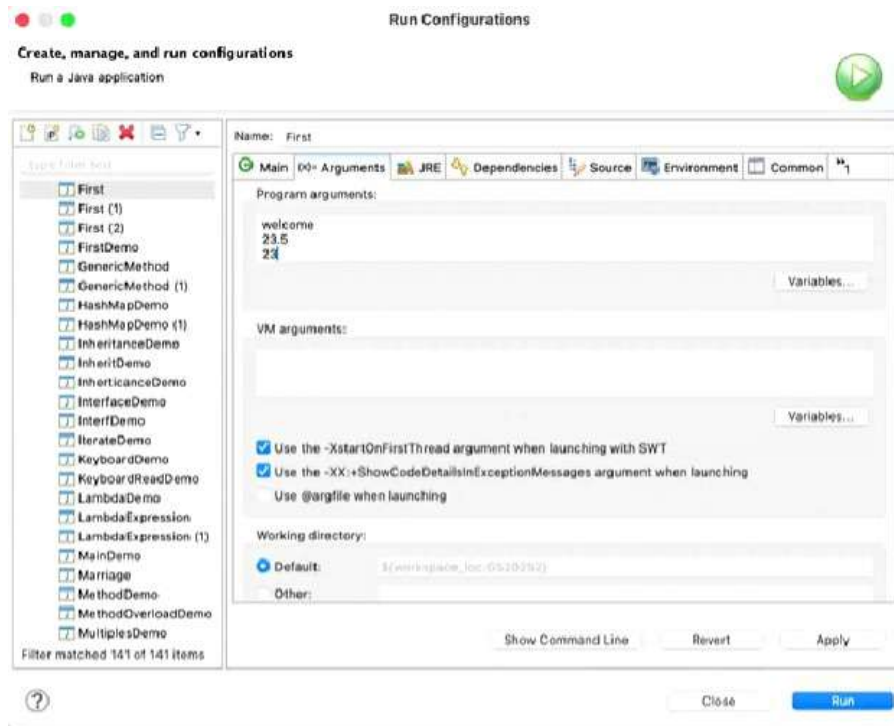
- ⇒ Command-line arguments allow us to supply inputs to the main method without user interaction.
- ⇒ They can also be displayed and used directly in the program.

3) How can we supply the String[] arguments to the main method in Eclipse?

Step1: Open **Run Configurations**.



Step 2: Enter the required arguments under the **Arguments** tab.



⇒ Here space between Strings considered as a separator (new argument).

Example 1:

Welcome

23.5

23

Thamarai Selvam San

⇒ If you want multiple words as a single argument, enclose them in quotes " ".

Example 2:

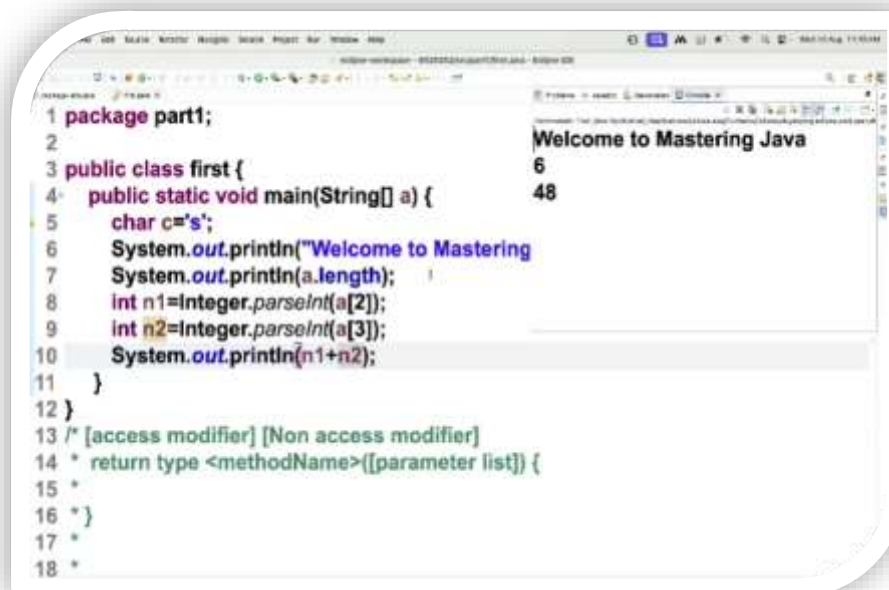
Welcome

23.5

23

"Thamarai Selvam San"

4) How can we use it in our program?



The screenshot shows an IDE with a Java program on the left and its output on the right. The code defines a class 'first' with a 'main' method that prints the length of the first argument, parses the second and third arguments as integers, and prints their sum. The output shows 'Welcome to Mastering Java', '6', and '48'.

```
1 package part1;
2
3 public class first {
4     public static void main(String[] a) {
5         char c='s';
6         System.out.println("Welcome to Mastering
7         System.out.println(a.length);
8         int n1=Integer.parseInt(a[2]);
9         int n2=Integer.parseInt(a[3]);
10        System.out.println(n1+n2);
11    }
12 }
13 /* [access modifier] [Non access modifier]
14 * return type <methodName>([parameter list]) {
15 *
16 * }
17 *
18 *
```

Output:

```
Welcome to Mastering Java
6
48
```

- I. Length of the arguments (Example 1):
a.length => output: 6
- II. Length of the arguments (Example 2):
a.length => output: 4
- III. System.out.println(a[2]+a[2]) => output: 2323 // here "+" act as concatenation
- IV. System.out.println(a[2]+'s') => output: 138 // here 's' changed as ASCII value 115
- V. int n1 = Integer.parseInt(a[2]); // Converts 3rd command-line argument to integer. (Example:23)
int n2 = Integer.parseInt(a[3]); // Converts 4th command-line argument to integer. (Example:11)
System.out.println(n1 + n2); // Prints the sum of the two integers. => output 34

5) java.lang => Integer => parseInt

public static int parseInt([String](#) s) throws
[NumberFormatException](#)

Parses the string argument as a signed decimal integer.

Parameters:

s - a String containing the int representation to be parsed

Returns:

the integer value represented by the argument in decimal.

Throws:

[NumberFormatException](#) - if the string does not contain a parsable integer.

6) Plus Operator (+):

“+” Acts as **concatenation** when at least one operand is a **Non-numeric**.

“+” Acts as **addition** when both operands are **numeric**.

“+” binary operator / implicitly overloaded operator

7) Wrapper Classes:

- ⇒ Wrapper classes are the object representation of primitive data types.
- ⇒ Classes: Byte, Short, Integer, Long, Float, Double, Character, Boolean.

8) Naming Convention:

1. Class Names

- Use UpperCamelCase (PascalCase).
- Each word starts with an uppercase letter.
- Example: StudentDetails, BankAccount.
- Using lowercase is legal, but not good practice.

2. Method Names

- Use lowerCamelCase.
- The first word starts with a lowercase letter.
- Each subsequent word starts with an uppercase letter.
- Example: calculateSum(), getStudentName().

3. Variable Names

- Same as method names → lowerCamelCase.
- Example: totalMarks, studentAge.

4. Constants / final variables

- Use uppercase letters with underscores.
- Example: PI, MAX_SPEED.

5. Packages

- Always lowercase.
- Example: java.util, part1.

6. Keywords

- Always lowercase (defined by Java).
- Example: class, public, static.

Naming Conventions:

1. If several words are linked together to form a name for an identifier, then the first letter of the inner words should be capitalized (upper case)
2. Class names and interface names must begin with an uppercase
 - a) class names should be typically be nouns
 - b) interface names should be typically be adjectives
3. Methods and variables must begin with a lower case
 - a) method names should be typically be verb-noun pairs
 - b) variable names should be short and meaningful
4. Constants must be fully capitalized with underscore connecting multiple words.

```
// onetwothree -> OneTwoThree  
// class OneTwoThree{ }  
// interface OneTwoThree{ }  
// void oneTwoThree{ }  
// int oneTwoThree=5;  
// package oneTwoThree;
```

Question:

How many implicitly overloaded operators are there in Java?



Zoho Schools for Graduate Studies



Notes

Day-2

JAVA LANGUAGE BASICS PART -1

IDENTIFIERS

Identifiers are names given to identify various programming elements(components) such as variables, constants, array name, string name, method name, class name, etc

RULES FOR IDENTIFIERS IN JAVA

- **Can contain** → letters (A–Z, a–z), digits (0–9), underscore (`_`), and dollar sign (`$`).
Example: `myVar`, `_value`, `$amount`, `student1`
- **Cannot start with a digit.**
`1student` (invalid)
`student1` (valid)
- **Cannot use Java keywords (reserved words).**
`class`, `int`, `public`
- **Case-sensitive** → `Age` and `age` are different.
- **No spaces or special characters** (like `@`, `-`, `#`).
`total marks`
`student-name`
- **Should be meaningful** (for readability).
`studentName` instead of `sn`
- **No length limit**, but keep it reasonable.

Valid: `name`, `age1`, `_rollNo`, `$salary`

Invalid: `123name`, `class`, `student-name`, `total marks`

- **Total Special Symbols – 32**
- **In Java Identifiers we use only 2 special character (_ , \$)**

RULES FOR NAMING IDENTIFIERS IN JAVA

- If several words are linked together to form a name for an identifier, then the first letter of the inner words should be capitalized (upper case)- Camel Case Notation
- Class names and interface names must begin with uppercase (Pascal Case Notation)
 - a) class names should be typically be nouns
 - b) interface names should be typically be adjectives
- Methods and variables must begin with a lower case
 - a) method names should be typically verb-noun pairs
 - b) variable names should be short and meaningful
- Constants must be fully capitalized with underscore. connecting multiple words

ADVANTAGES OF NAMING IDENTIFIERS IN JAVA

- ✓ Improved Code Readability
- ✓ Enhanced Maintainability

SOURCE FILE

- A source file is a file that contains Java code, usually defining one or more classes.

- It must have the file extension `.java`.
- The source file name should exactly match the public class name inside the file.

SOURCE FILE DECLARATION RULES

- Only one public class is allowed per Java source file
- The source file name must exactly match the public class name
- Multiple non-public classes, interfaces, or enums can be included in a single file.
- The package statement (if present) must be the first line in the file.
- Statements should follow the package declaration and appear before the class declaration.
- If there is no package statement, import statements (if any) must come first before class declarations.

- In source file we can create “n” no.of classes
- **Every class should contain main method?**
Ans: Every class can have main method.

Class contains,

1.PUBLIC

How many Public class can a source file have ?

Ans: Atmost one (0 or 1)

2. NON PUBLIC

How many Non Public class can a source file have?

Ans: “n” no. of Non-public classes

If our source file don't have public class,

We can give any name to source file or we can give any one of the non-public class name.

Can we create “n” number of public class in Notepad?

Ans: No. In Notepad, you can have only one public class per file.

DATATYPES

Java as a Strongly Typed Language

Strong Typing: Java enforces strict type rules. Every variable and expression has a clearly defined type, and all assignments are checked for compatibility. Type mismatches result in compile-time errors.

Primitive Types: Java includes eight primitive (or simple) types:

Integers: byte, short, int, long

Floating-point: float, double

Character: char

Boolean: boolean

JAVA INTEGER TYPES

- **Types Defined: Java provides four signed integer types:**

- **byte**
- **short**
- **int**
- **long**

- Signed Only: Java does not support unsigned integers (positive-only values).

Type	Size	Range	Common Uses & Notes
byte	8-bit	−128 to 127	Efficient for raw binary data, file/network streams.
short	16-bit	−32,768 to 32,767	Rarely used; sometimes for memory-constrained systems.
int	32-bit	−2,147,483,648 to 2,147,483,647	Most commonly used; ideal for loops, array indexing. Promotes performance due to type promotion.
long	64-bit	−9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	Used for large whole numbers (e.g., astronomical calculations).

- Type promotion: byte and short are automatically promoted to int in expressions, which often makes int more efficient.
- Use long when int isn't sufficient for storing large values.

Type	Precision	Size	Use Case & Notes
<code>float</code>	Single	32-bit	Good for saving memory when precision isn't critical (e.g., dollars and cents). May lose accuracy with very large/small values.
<code>double</code>	Double	64-bit	Preferred for high-precision calculations, especially in scientific or iterative computations. All math functions like <code>sin()</code> , <code>cos()</code> , <code>sqrt()</code> return <code>double</code> .

JAVA CHAR TYPE

- Type Definition: `char` is a 16-bit data type used to store characters.
- Unicode-Based: Java uses Unicode, a universal character set that supports characters from all human.
- Range: `char` values range from 0 to 65,535. There are no negative values.
- ASCII Compatibility:
 ASCII characters (0-127) are part of Unicode.
 ISO-Latin-1 (0-255) is also supported.

JAVA BOOLEAN TYPE

- Boolean is a primitive type used for logical values.
- Possible Values: Only two – `true` and `false`.
- Usage: Returned by relational operators like `<`, `>`, `==`, etc.
- Required in control statements such as `if`, `while`, and `for`.

How range for data types is calculated?

$$-2^{(n-1)} \text{ to } 2^{(n-1)} - 1$$

- **n**
Total number of bits used to store the value.
- **Sign bit**
The highest-order bit (bit at position $n-1$) determines the sign:
 - 0 → non-negative
 - 1 → negative
- $-2^{(n-1)}$
This is the **minimum** value.
 - All magnitude bits zero except the sign bit → most negative number.
 - Example for $n = 8$: $-2^7 = -128$.
- $2^{(n-1)} - 1$
This is the **maximum** value.
 - Sign bit zero, all other bits one → largest positive number.
 - Example for $n = 8$: $2^7 - 1 = 127$.

Calculating range for short, int, long, char

- byte → 8 bits → range = -128 to 127 (-2^7 to 2^7-1)
- short → 16 bits → range = -32,768 to 32,767 (-2^{15} to $2^{15}-1$)
- int → 32 bits → range = -2,147,483,648 to 2,147,483,647 (-2^{31} to $2^{31}-1$)
- char → 16 bits (unsigned) → range = 0 to 65,535 (0 to $2^{16}-1$)

DERIVED DATA TYPES

In Java, data types are classified into:

1. **Primitive Data Types** → byte, short, int, long, float, double, char, boolean
2. **Derived Data Types** → These are built from primitive types.

Derived Data Types in Java are:

- Arrays
- Classes
- Interfaces
- Strings
- Objects
- Enumerations (enum)

These are called derived because they are created using primitive data types or by combining existing data types.

ESCAPE SEQUENCE

Escape sequences are special character combinations used to represent characters that are difficult or impossible to type directly. They begin with a backslash (\) and are commonly used in character and string literals.

```
System.out.println("She said \"Java is fun!\" ");
```

KEYWORDS IN JAVA

Keywords are special words in the Java language that have predefined meanings and cannot be used as identifiers (like variable names, class names, or method names). They form the core syntax of Java.

As per the official Oracle Java documentation,
There are 51 keywords (in Java SE 17+).

Complete list as per Oracle's official documentation:

abstract, assert, boolean, break, byte, case, catch, char, class, const*, continue, default, do, double, else, enum*, extends, false*, final, finally, float, for, goto*, if, implements, import, instanceof, int, interface, long, native, new, null*, package, private, protected, public, return, short, static, strictfp**, super, switch, synchronized, this, throw, throws, transient, true*, try, void, volatile, while

RESERVED WORDS IN JAVA

Reserved words are words that are set aside by Java for special meaning.

They cannot be used as identifiers (variable names, class names, etc.).

Contextual (Reserved) Keywords

exports, module, non-sealed, open, opens, permits, provides, record, requires, sealed, to, transitive, uses, var, when, with, yield

Reserved but Unused Keywords

const, goto, strictfp

Reserved Literals

true, false, null

Are they the same?

- All keywords are reserved words
- But not all reserved words are keywords.

How many keywords and reserved words?



Zoho Schools for Graduate Studies



Notes

Day-3

JAVA LANGUAGE BASICS PART -4

Variable

Variables are named memory location that can take different values during a program execution.

Syntax :

data type variable_name = value;

Three types of variable : Local variable, Instance Variable, Static Variable

Local Variable

- Variables that are declared **inside a block**
- Local Variables **must be initialized** before it's usage **except Array**

Block : A block is set of statements enclosed within a pair of curly braces {} and can have one or more statements between the braces

- It can have data declaration (inside a class, outside the method)
- Every block can have new data types
- Can declare a variable inside a block
- All methods are blocks , but not all blocks are methods

Instance Variable

Variable declared inside the class but outside all the methods or blocks

- Values can be assigned during the declaration or within the constructor
- Instance variable if uninitialized , it'll get the default values corresponding to it's data types

Example :

```
1 package part1;
2
3 public class VariableDemo {
4     int b; // non-static instance variable
5     public static void main(String[] args) {
6         int a;
7         System.out.println("hi");
8         a=8;
9         System.out.println(a);
10        System.out.println(b);
11    }
12    // A non-static member cannot be accessed directly inside a static context
13 }
14
```

Compiler Error : Can't make a static reference to the non static field b

Then , How to access it?

A non static member can be accessed inside a static context through it's object reference or object variable

- To create object : by using “new” operator

Example for using “new” operator:

```
1 package part1;
2
3 public class VariableDemo {
4     int b; // non-static instance variable
5     public static void main(String[] args) {
6
7         int a;
8         System.out.println("hi");
9         a=8;
10        System.out.println(a);
11        System.out.println(new VariableDemo().b);
12    }
13    // A non-static member cannot be accessed directly inside a static context
14    //A non-static member can be accessed inside a static context through its
15    //object reference or object variable
16 }
```

NOTE : When we should give the obj name -> when we want to use the obj more than once

DEFAULT VALUES :

DATA TYPES	DEFAULT VALUES
Byte	0
Short	0
Int	0
Long	0
Float	0.0f
Double	0.0d
Char	\u0000(empty character)
boolean	false
String	null

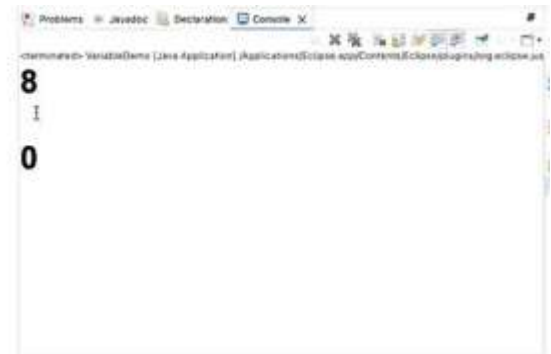
Static Variable

Variables that are declared with the static keyword inside a class but outside of all the methods, constructors or blocks.

- There will be only one copy of a static variable per class and it will be shared by all the objects created from the class
- Static Member can be accessed directly inside a static context or you can access it using a class name

By accessing ,using static context :

```
1 package part1;
2
3 public class VariableDemo {
4     char b; // non-static instance variable
5     static int c; // static instance variable
6     public static void main(String[] args) {
7         int a;
8         a=8;
9         System.out.println(a);
10        System.out.println(new VariableDemo().b);
11        System.out.println(c);
12    }
13    // A non-static member cannot be accessed directly inside a static context
14    //A non-static member can be accessed inside a static context through its
15    //object reference or object variable
16    // A static member can be accessed directly inside a static context
```



By accessing, using class name:

```
1 package part1;
2
3 public class VariableDemo {
4     char b; // non-static instance variable
5     static int c; // static instance variable
6     public static void main(String[] args) {
7         int a;
8         a=8;
9         System.out.println(a);
10        System.out.println(new VariableDemo().b);
11        System.out.println(VariableDemo.c);
12    }
13    // A non-static member cannot be accessed directly inside a static context
14    //A non-static member can be accessed inside a static context through its
15    //object reference or object variable
16    // A static member can be accessed directly inside a static context
```

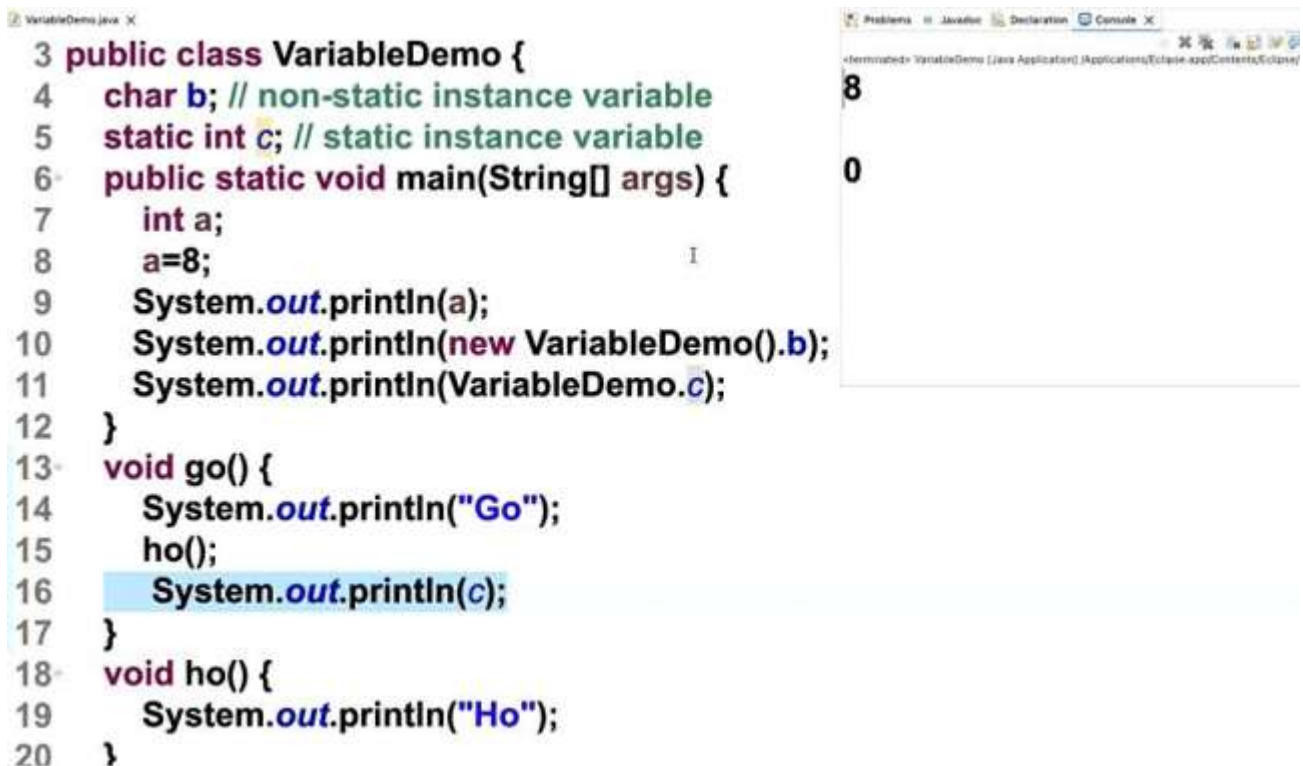
- A non-static member can be accessed directly inside a non-static context

Example :

```
7     int a;
8     a=8;
9     System.out.println(a);
10    System.out.println(new VariableDemo().b);
11    System.out.println(VariableDemo.c);
12    }
13    void go() {
14        System.out.println("Go");
15        ho();
16    }
17    void ho() {
18        System.out.println("Ho");
19    }
```

- A static member can be accessed directly inside a non-static context

Example :



The screenshot shows an IDE with two panels. The left panel displays the source code for `VariableDemo.java`. The code defines a `public class VariableDemo` with a `main` method and two other methods, `go` and `ho`. The `main` method prints the value of a static variable `c` and a non-static variable `b` from a new instance. The `go` method prints "Go" and calls `ho`, and `ho` prints "Ho". The right panel shows the console output, which contains the numbers 8 and 0, corresponding to the first two `println` statements in the `main` method.

```
3 public class VariableDemo {
4     char b; // non-static instance variable
5     static int c; // static instance variable
6     public static void main(String[] args) {
7         int a;
8         a=8;
9         System.out.println(a);
10        System.out.println(new VariableDemo().b);
11        System.out.println(VariableDemo.c);
12    }
13    void go() {
14        System.out.println("Go");
15        ho();
16        System.out.println(c);
17    }
18    void ho() {
19        System.out.println("Ho");
20    }
```

8
0

In the above program here why , Go and Ho are not getting executed?

Cause, they're not called.

We can call them **directly or indirectly** by using the method through **main method**

Example :

```
7    int a;  
8    a=8;  
9    VariableDemo v;  
10   v=new VariableDemo();  
11   System.out.println(a);  
12   System.out.println(v.b);  
13   System.out.println(VariableDemo.c);  
14   v.go();  
15  
16   }  
17   void go() {  
18       System.out.println("Go");  
19       ho();  
20       System.out.println(c);  
21   }  
22   void ho() {  
23       System.out.println("Ho");  
24   }
```

NOTE: A user defined method will get chance of execution only if it's invoked(called) either directly or indirectly through main method

Why a Main Method called statement required?

Main is the entry point and exit point of the execution, so **user defined method** will get chance of execution only if it is called **either directly or indirectly** through **main**



Zoho Schools for Graduate Studies



Notes

Day-4

SCOPE AND LIFE TIME OF VARIABLE

1. What is the output of the following code?

```
class First{  
    int a;  
    public static void main(String[] args){  
        System.out.println(a);  
    }  
}
```

Ans: Compiler Error

Reason: Non-Static Instance Variable cannot be accessed directly inside the static context.

2. What is the output of the following code?

```
class First{  
    int a;  
    public static void main(String[] args){  
        First f;  
        System.out.println(f.a);  
    }  
}
```

Ans: Compiler error

Reason: Object is not even created just Object name is given.

3. What is the output of the following code?

```
class First{  
    int a;  
    public static void main(String[] args){  
        First f = new First();  
        System.out.println(f.a);  
    }  
}
```

Ans: 0

Reason: Instance Variable if uninitialized gets default initialization.

4. What is the output of the following code?

```
class First{  
    int b;  
    public static void main(String[] args){  
        int a;  
        First f = new First();  
        System.out.println(a);  
    }  
}
```

Ans: Compiler Error

Reason: Local Variable (a) is not initialized

5. What is the output of the following code?

```
class First{  
    int a;  
    public static void main(String[] args){  
        int a = 2;  
        System.out.println(a);  
    }  
}
```

Ans: 2

Reason: Local Variable can have same name as that of Instance Variable but bad programming practice. In Main method we initialized local variable so it gets printed.

6. What is the output of the following code?

```
class First{  
    int a;  
    public static void main(String[] args){  
        int a = 2;  
        First f = new First();  
        System.out.println(f.a);  
    }  
}
```

Ans: 0

Reason: Here the print statement uses instance variable. Instance variable if uninitialized will have the default value.

7. What is the output of the following code?

```
class First{  
    public static void main(String[] args){  
        int a;  
        First f = new First();  
        a = 3;  
        f.go();  
        System.out.println(a);  
    }  
    void go(){  
        int x = a;  
        System.out.println(x);  
    }  
}
```

Ans: Compiler Error

Reason: In go() method x is initialized with (a) but a is not initialized in that scope.

SCOPE: The region in a program where upto which the variable is accessible or visible.

LIFE TIME:

- How long the variable exists before it is destroyed.
- Destroying refers to deallocating the memory that was allotted to the variables when declaring it.

Variable Type	Scope	Lifetime
Instance variable	Throughout the class except in static methods	Until the object is available in the memory
Class variable	Throughout the class	Until the end of the program
Local variable	Within the block in which it is declared	Until the control leaves the block in which it is declared

Local Variable:

- Accessible within the block.
- Destroyed outside the block.
- Allocated in Stack memory.
- Access modifiers cannot be used for local variables.
- Final is the only non-access modifier used.

Instance Variable:

- Accessible within the class(Methods,blocks and constructors).
- Visibility of instance variable can be controlled using access modifiers.
 - Public – Can access outside the class also
 - Protected – Subclasses of same and different packages

- Private – only in that class.
- Generally it is recommended to use private
- Allocated in Heap memory
- Out of 8 Non-access modifiers only static, final and transient are applicable to instance variable.
- Life time: Until the object is available in the memory.

Static Variable:

- Accessible throughout the class
- Only one copy of static variable per class exist
- Shared by all objects created from the class
- Used to refer common property for all objects
- Allocated in class area or static memory area
- Get memory at the time of class loading
- Destroyed when the program terminates
- Scope is similar to instance variable
- Generally it is recommended to declare as public

8. What is the output of the following code?

```
class First{
    int a;
    public static void main(String[] args){
        int a;
        First f = new First();
        a = 3;
        f.go();
        System.out.print(a); }
}
```

```
void go(){
    int x = a;
    System.out.print(x);
}
}
```

Ans: 03

Reason: In the method go() instance variable is used whereas in the main method local variable is used.

9. What is the output of the following code?

```
class First{
    int a;
    public static void main(String[] args){
        int a;
        First f = new First();
        a = 3;
        f.go();
        System.out.print(a+f.a);
    }
    void go(){
        int x = a;
        a = 3;
        System.out.print(x);
    }
}
```

Ans: 06

Reason: In the method go() instance variable is assigned a value and in the main method both variables are used.

10. What is the output of the following code?

```
do{  
  
    int x = 0;  
  
    System.out.println(x++);  
  
}while(x<=10);
```

Ans: Compiler Error

Reason: In the condition we used x but x is local variable. So in dowhile loop declare and initialize the variable out of the dowhile loop.

11. What is the output of the following code?

```
int x;  
  
do{  
  
    x = 0;  
  
    System.out.println(x++);  
  
}while(x<=10);
```

Ans: 0 Prints infinitely.

Reason: x is initialized everytime while the loop runs.

12. What is the output of the following code?

```
int x=0;  
  
do{  
  
    System.out.print(x++);  
  
}while(x<=10);
```

Ans: 012345678910

Reason: Initialization done outside the do while and just the incrementation takes place inside the loop.



Zoho Schools for Graduate Studies



Notes

Day-5

Literals

A literal is a constant value that is directly written in the code (fixed value assigned to a variable).

They are case-insensitive

1) Integer Literals

- Whole numbers (without decimal).
- Can be written in:
 - Decimal (base 10): `int a = 25;`
 - Octal (base 8, prefix `0`): `int b = 025;`
 - Hexadecimal (base 16, prefix `0x/0X`): `int c = 0x1A;`
 - Binary (base 2, prefix `0b/0B`): `int d = 0b1010;`

2) Floating-Point Literals

- Numbers with decimal point or exponent (scientific notation).
- Example: `3.14`, `2.5e3` ($\rightarrow 2500.0$)
- By default $\rightarrow$ double, use `f` or `F` for float:
 - `float f = 3.14f;`

3) Character Literals

- Single character inside single quotes: `'A'`, `'9'`
- Can use escape sequences:
 - `'\n'` (newline), `'\t'` (tab), `'\\'` (backslash), `'\"'` (single quote)

4) String Literals

- Sequence of characters inside double quotes.
- Example: `"Hello"`, `"Java\nWorld"`

5) Boolean Literals

- Only two values: `true` or `false`

6) Null Literal

- Represents `"no object"`.
- Example: `String s = null;`

Underscore Usage

- Underscore improves readability of numeric literals.
- Cannot be placed at start/end, before decimal/suffix/prefix.
- From Java 9, `_` is reserved, not usable as variable name.

Valid Examples

```
int oneMillion = 1_000_000;  
long creditCard = 1234_5678_9012_3456L;  
double pi = 3.14_159_265;  
int binary = 0b1010_1011;
```

Invalid Examples

```
int x = _100;    // cannot start with _  
int y = 100_;    // cannot end with _  
double d = 100_.0; // not next to decimal  
float f = 3.14_f; // not next to suffix  
int hex = 0x_FF; // not just after 0x  
int _ = 10; // Allowed in Java 7/8  
int _ = 20; // Compilation error
```

Operators in Java

Operand

An operand is the data (value, variable, or object) on which an operator acts.

Operator

An operator is a symbol that represents an action to be performed.

Operation

An operation is the complete action of applying an operator on operands.

Types of Operators (From high to low precedence)

1. Unary Operator
2. Arithmetic Operator
3. Shift Operator
4. Relational Operator
5. Bitwise Operator
6. Logical Operator
7. Ternary Operator
8. Assignment Operator

Questions

1)Negation

Code

```
int a=5;  
System.out.println(~a);
```

Output

6

Explanation

General Rule

For any integer a:

$$\sim a = -(a + 1)$$

Therefore:

$$\sim 5 = -(5 + 1) = -6$$

2)Modulus operator

Code

```
System.out.println(-12%5);
```

Output

-2

Explanation

Perform modulus operation and give the sign of the numerator for the result

3) Modulus operator

Code

```
System.out.println(12%-5);
```

Output

2

Explanation

Perform modulus operation and give the sign of the numerator for the result

4) Arithmetic operator

Code

```
short a=5,b=6;  
Short c=a+b;  
System.out.println(c);
```

Output

Runtime type error

Explanation

Addition is performed only for integer values. So, after performing the addition operation the result is of type int and must be cast back to short

Corrected code

```
short a=5,b=6;  
Short c=(short)a+b;  
System.out.println(c);
```

Output

11

5) Arithmetic operator

Code

```
int a = 4, b = 3, x = 5;  
x /= a * b;  
System.out.println(x);
```

Output

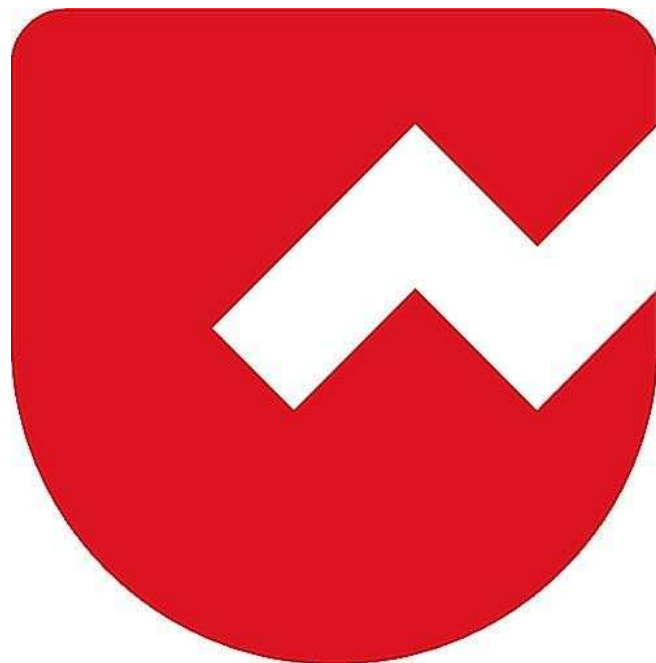
0

Explanation

- Evaluate $a * b$ first (multiplication has higher precedence than assignment):
 $a * b = 4 * 3 = 12$.
- Compute $x / (a*b)$ using integer division:
 $x / 12 = 5 / 12$. Since x , a , b are int, this is integer division — fractional part discarded. $5 / 12 \rightarrow 0$ (0.416... truncated to 0).



Zoho Schools for Graduate Studies



Notes

Day-6

Control Statement

1. What are Control Statements?

In programming, statements are executed sequentially, one after another, by default. However, sometimes we need to change the flow of execution based on certain conditions or requirements. This is where Control Statements are used.

Definition:Control statements are programming constructs that alter the normal, sequential execution of code.

Purpose:They enable decision-making, looping, and unconditional jumps in a program.

2. Categories of Control Statements:

Control statements can be broadly classified into two types:

Conditional Control Statements:

These control the flow of execution based on whether a condition is true or false.

Examples: if, if-else, if-else-if ladder, switch-case, ternary operator (?:).

Unconditional Control Statements:

These transfer control from one part of the program to another without evaluating any condition.

Examples: break, continue, return, goto (in some languages).

Conditional Control Statements:

1. if Statement :

Syntax:

```
if (condition) {  
    // Code executed when condition is true  
}
```

Explanation:

Executes a block of code only if the condition evaluates to true..

Example:

```
int age = 18;  
if (age >= 18) {  
    System.out.print("You are eligible to vote.");  
}
```

2. if-else Statement:

Syntax:

```
if (condition) {  
    // Code executed if condition is true  
} else {  
    // Code executed if condition is false  
}
```

Explanation:

Provides an alternative block of code when the condition is false.

Example:

```
int marks = 40;  
if (marks >= 50) {
```

```
    System.out.print("Pass");
```

```
} else {
```

```
    System.out.print("Fail");
```

```
}
```

3. if-else if Ladder:

Syntax:

```
if (condition1) {  
    // Code block 1  
}  
  
else if (condition2) {  
    // Code block 2  
}  
  
} else { // Default block  
}
```

Explanation: Used to check multiple conditions in sequence.

Example:

```
int marks = 85;  
if (marks >= 90) {  
    System.out.print("Grade A");  
}  
else if (marks >= 75) {  
  
    System.out.print("Grade B");  
}  
else {  
  
    System.out.print("Grade C");  
}
```


4. switch-case Statement:

Syntax:

```
switch (expression) {  
    case value1: // Code block 1  
        break;  
    case value2: // Code block 2  
        break;  
    default:    // Default block  
  
}
```

Explanation:

Used when multiple values of a single expression are compared.

Example:

```
int day = 3; switch (day) {  
  
    case 1: System.out.print("Monday"); break;  
    case 2: System.out.print("Tuesday"); break;  
  
    case 3: System.out.print("Wednesday"); break;  
    default: System.out.print("Invalid Day");  
  
}
```

5. Ternary Operator (?):

Syntax:

```
variable = (condition) ? value_if_true : value_if_false;
```

Explanation:

condition – An expression that results in either true or false.

value_if_true – Value assigned to variable if condition is true.

value_if_false – Value assigned to variable if condition is false.

Example:

```
int age = 20;  
  
String result = (age >= 18) ? "Adult" : "Minor";  
  
System.out.println(result);
```

Types of Unconditional Control Statements in Java:

1. break Statement:

Purpose:

Used to terminate a loop or a switch statement immediately and transfer control to the next statement outside the loop/switch.

Syntax:

```
break;
```

Example:

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) {  
        break; // Exits loop when i = 3  
    }  
    System.out.println(i); //output=1 2  
  
}
```

2. continue Statement:

Purpose:

Skips the current iteration of a loop and continues with the next iteration.

Syntax:

`continue;`

Example:

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) {  
        continue; // Skips printing when i = 3  
    }  
  
    System.out.println(i); //output 1 2 4 5  
}
```

3. return Statement:

Purpose:

Ends the execution of a method and optionally returns a value.

Syntax:

`return;` // For methods with void return type

`return value;` // For methods returning a value

Example:

```
public static int add(int a, int b) {  
    return a + b; // Returns sum to caller  
}  
  
public static void main(String[] args) {  
    System.out.println(add(5, 3)); //output=8  
}
```

4. goto Statement (Not in Java):

Note: Java does not support goto because it makes code difficult to read and maintain. Instead, Java uses labels with break or continue to control flow in nested loops.

Example (Using Labeled Break):

```
outer: for (int i = 1; i <= 3; i++) {  
    for (int j = 1; j <= 3; j++) {  
        if (i == 2 && j == 2) {  
            break outer; // Exits both loops  
        }  
        System.out.println(i + " " + j); //output=1 1 1 2 1 3 2 1  
    }  
}
```

1) what is the output of the following code upon execution?

```
public class First{  
    public static void main(String[] args){  
        int x=1;  
        if(x)  
            System.out.println("Success");  
        else  
            System.out.println("Failure");  
    }  
}
```

a) compiler error

b) runtime error

c) java

d) none of above

Reason:

The image shows a code snippet with an if(x) condition. In Java, the condition inside an if statement must be a boolean value (true or false).

An int value cannot be directly used as a boolean condition. This would result in a compiler error.

2) what is the output of the following code upon execution?

```
public class First{  
    public static void main(String[] args){  
        int x=1;String s=null;  
        if(x==s)  
            System.out.ptintln("success");  
        else  
            System.out.println("Failure");  
    }  
}
```

a) compiler error

b) runtime error

c) java

d) none of above

Reason:

Another image shows `if(x==s)`, where `x` is an `int` and `s` is a `String`. You cannot directly compare a primitive

`type (int)` with a reference type (`String`) using the `==` operator. This will also cause a compiler error.

3) what is the output of the following code upon execution?

```
public class First{  
  
    public static void main(String[] args){  
        int x=10,y=23;  
        boolean b=x>=y;  
        if(b==true)  
            System.out.println("Success");  
        else  
            System.out.println("Failure");  
    }  
}
```

- a) compiler error
- b) run time error
- c) success
- d) Failure

Reason:

The statement `b=true` is an assignment operator (`=`), not a comparison operator (`==`).

It assigns the value `true` to `b`. The result of an assignment operation is the value that was assigned

Therefore, `b=true` evaluates to `true`, and the `if` condition is met. As a result, the code will print `Success`.

Tautology:

Definition: A proposition is called a tautology if it is always true, regardless of the truth values of its individual components.

Contradiction:

Definition: A proposition is called a contradiction if it is always false, regardless of the truth values of its individual components.

4) what is the output of the following code upon execution?

```
public class First{  
    public static void main(String[] args){  
        int=4;  
        long y=x*4-x++;  
        if(y>12)  
            System.out.println("Success");  
        else  
            System.out.println("Failure");  
        else  
            System.out.println("Draw");  
    }  
}
```

Reason:

This final example shows an if-else structure with an extra else clause. * An if statement can only have one corresponding else block. Having two else blocks is a syntax error. This will result in a compiler error.

5) For what value is 'x' the following the code yields "Failure" of output?

```
public class First{
    public static void main(String[] args){
        if(x<4){

            System.out.println("success");

        }
        else if(x<3)

            System.out.println("Failure");
    }
}
```

- a) values greater than x
- b) values lesser than x
- c) For value of x=3
- d) none of the above

Reason:

The image shows if (x<4) ... else if (x<3). An if-else if ladder evaluates conditions sequentially. The first condition x<4 is checked. If x=3, this condition is true, and "Success" is printed.

The else if block is never checked because the first if condition was met.

This demonstrates how the ladder structure works: once a condition is true, the rest of the ladder is skipped.

6) what is the output of the following code upon execution?

```
public class First{

    public static void main(String[] args){
        int x=1,y=1
        int z=x>5?y++:x++;
        System.out.println(x+ " " +y+ " " +z);
    }
}
```

Reason:

The code `int z=x>5 ? y++ : x++;` is a ternary operator (also known as a conditional operator).

This is a shorthand for an if-else statement.

Here, x is 1.

The condition `x>5` is false.

The code after the colon (:) is executed: `x++`.

The value of x is incremented to 2.

The value of the expression `x++` (which is the original value of x, 1) is assigned to z.

The output would be: 2 1 1.

7)what is the output of the following code upon execution?

```
public class First{ public static void main(String[] args){  
  
    int x=20/3;  
    switch(x){  
    case 3:System.out.println("1");  
    case 5:System.out.println("5");  
    case 2:System.out.println("2");  
    default:System.out.println("3");  
    }  
    }  
}
```

Reason:

The switch statement is used to execute one of many code blocks based on a value. *

Here, x is 20/3, which in integer division is 6.

The switch statement checks the value of x against the case labels.

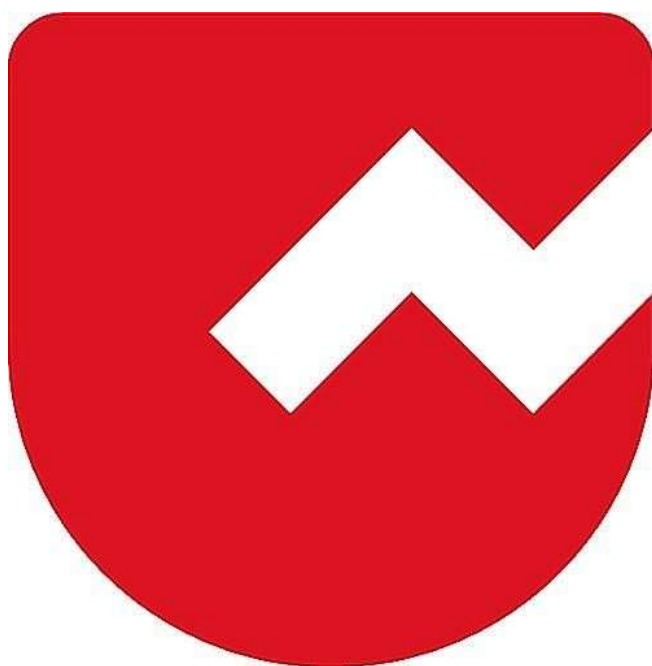
None of the case labels (3, 5, or 2) match the value 6.

The code falls through to the default block.

The output will be: 3.



Zoho Schools for Graduate Studies



Notes

Day-7

Day 8 - Switch Case & Iterative Statements

1. Switch Case Rules in Java

The switch statement allows multi-way branching based on the value of an expression.

Key Rules:

1. The switch expression must evaluate to:
 - byte, short, char, int (primitive types)
 - enum types
 - String (Java 7 onwards)
 - Wrapper classes of primitives (Byte, Short, Character, Integer)
2. Case values must be constants or literals – variables are not allowed.
3. Case values must be unique.
4. The default case is optional.
5. break is used to exit the switch, otherwise fall-through occurs.
6. Switch can have 0 or n number of cases.

Example:

```
public class SwitchDemo {  
    public static void main(String[] args) {  
        int x = 2;  
        switch(x) {  
            case 1: System.out.println("One"); break;  
            case 2: System.out.println("Two"); break;  
            case 3: System.out.println("Three"); break;  
            default: System.out.println("Invalid");  
        }  
    }  
}
```

Output:

Two

2. Fall Through in Switch

If break is not used, control passes from the matched case to the next case(s) until it finds a break or the switch ends. This behavior is called fall-through.

```
int x = 2;
switch(x) {
    case 1: System.out.println("One");
    case 2: System.out.println("Two");
    case 3: System.out.println("Three");
    default: System.out.println("Default");
}
```

Output:

Two

Three

Default

3. Iterative Statements

Used to execute a set of statements repeatedly until a condition is satisfied.

Types of Loops in Java:

- for loop
- while loop
- do-while loop
- Enhanced for loop (for-each)

3.1 For Loop

Structure:

```
for (initialization; condition; increment/decrement) {

}
```

Explanation:

1. Initialization → executed once before loop starts.
2. Condition → checked before every iteration. If false, loop exits.
3. Increment/Decrement → updates the loop variable after every iteration.

Example:

```
for(int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}
```

Output:

```
1  
2  
3  
4  
5
```

3.2 Enhanced For Loop

Used for iterating over arrays or collections.

Cannot modify the collection while iterating.

Heterogeneous objects can be stored in a collection, but iteration will treat them as Object type if not generic.

Syntax:

```
for (dataType variable : collection) {  
  
}
```

Example:

```
int[] arr = {10, 20, 30};  
for(int x : arr) {  
    System.out.println(x);  
}
```

Output:

```
10  
20  
30
```

4. Default in Switch

The default block executes when no case matches.

- default is optional.

- It can appear anywhere in the switch, not only at the end.

Example:

```
int x = 5;
switch(x) {
    default: System.out.println("Default");
    case 1: System.out.println("One");
}
```

Output:

Default

One

5. Case Count in Switch

A switch can have:

- 0 cases → only default (or even empty).
- n number of cases → depends on requirements.

Iterative Examples

1. Do-While Loop Example

```
public class IterativeDemo {
    public static void main(String[] args) {
        int n = 16;
        do {
            n *= 2; // n = n * 2
        } while (n < 100);
        System.out.println(n);
    }
}
```

Output: 128

Explanation: The loop multiplies **n** by 2 until it reaches 128. The condition fails when **n = 128** since **128 < 100** is false.

2. Nested For Loop Example

```
public class IterativeDemo {
    public static void main(String[] args) {
        int count = 0;
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 2; j++) {
                for (int k = 0; k < 2; k++) {
                    count++;
                }
            }
        }
    }
}
```

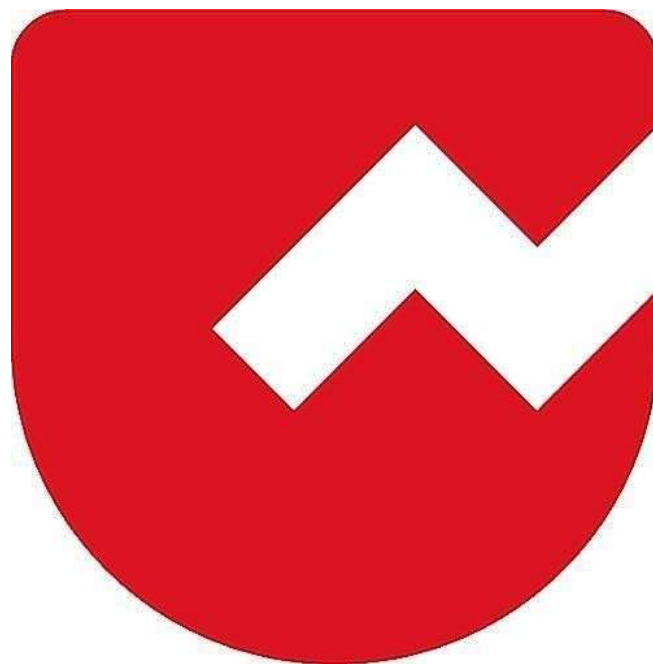
```
        }  
    }  
}  
System.out.println(count);  
}  
}
```

Output: 12

Explanation: The innermost loop executes $3 \times 2 \times 2 = 12$ times, so count becomes 12.



Zoho Schools for Graduate Studies



Notes

Day-8

Day 9 -Unconditional Control Statements

Unconditional Control Statements:

Those statements that change or alter the flow of control unconditionally are referred as unconditional control statements.

Examples: 1.break, 2.continue and 3.return.

1.what a 'continue' statement does?

It abruptly terminates the current iteration of the loop and proceeds with the next iteration.

- It cannot be used inside a “if” statement without loop.
- After “continue” no statements are allowed.

Example:

```
for(int i=1;i<=10;i++) {  
    if(i==4 | i==8) continue;  
    System.out.println(i);  
}
```

output: 1

2

3

5

6

7

9

10

what a labeled 'continue' statement does?

It abruptly terminates the current iteration of the labeled loop and proceeds with the next iteration of the labeled loop.

Example:

```
outer: for(int i=1; i<=3; i++){  
    Inner: for(int j=1; j<=3; j++){
```

```
        if(i==j) continue outer;
```

```
        System.out.println(i+ “ ” +j);
```

```
    }}
```

output:

2 1

3 1

3 2

2.what a 'break' statement does?

It abruptly terminates the current loop and takes the flow of control outside the current loop.

Example:

```
for(int i=1;i<=10;i++) {  
    if(i==4) break;  
    System.out.println(i);  
}
```

output: 1

2

3

what a labeled 'break' statement does?

It abruptly terminates the labeled loop and takes the flow of control outside the labeled loop.

Example:

```
outer: for(int i=1; i<=3; i++){  
    Inner: for(int j=1; j<=3; j++){  
        if(i==j) continue Inner;  
        System.out.println(i+ “” +j);  
    }  
}
```

output:

2 1

3 1

3 2

3. what a 'return' statement does?

It takes the flow of control from the current method to the called method.

Example:

```
int n=3;

System.out.println(go(5) + n);

}

static int go(int x) {

int y=4;

return x+y;

}
```

output: 12

what is the output of the following code :

```
int nTerms=5;

int sum=0;

for(int i=0;i<nTerms;i++){

if(i%2==1){

nTerms++;

continue;

} else {

sum+=i;

}

System.out.print(sum);

} // output : 20
```

Reason:

This final example shows a for loop in which the loop control variable (nTerms) is modified inside the loop body. *Each time the condition $i \% 2 == 1$ is true, the value of nTerms is increased.* This extends the range of the loop and causes more iterations to be executed than originally expected. During the even iterations, the value of i is added to sum and printed. As a result of these extended iterations, the loop finally ends with the accumulated value 20 being printed last. (here continues is meaningless). what is

the output of the following code :

```
int i=10;

outer: while(true){
    if(i==0){
        i--;
        break outer;
    }
    System.out.print(i+" ");
} }
```

Output:infinite Loops

Reason:

This final example shows a while(true) loop with a labeled break (break outer). The condition if(i == 0) is never satisfied because the value of i is not being changed inside the loop body when it is greater than 0. As a result, the statement System.out.print(i + " "); keeps executing with the same value 10. Since i remains unchanged, the loop never terminates and the value 10 is printed infinitely.

what is the output of the following code :

```
int i=10;

outer: while(true){
    if(i==0){
        continue outer;
        i--;
    }
    System.out.print(i+" ");
} } //output: compiler error
```

Reason:

Any code written after the continue inside the same block becomes unreachable.

what is the output of the following code :

```
int sum=0;

for(int i=0;i<5;i++){
    for(int j=0;j<5;j++){
        if(i==j)
            break;
        else sum+=i+j;
    } } System.out.println(sum);
}
```

output: 40

Reason:

This final example shows a nested `for` loop with a `break` statement. The inner loop terminates whenever `i == j`. For each value of `i`, the loop only adds pairs `(i + j)` until `j` becomes equal to `i`. After that, the `break` immediately ends the inner loop, and control returns to the outer loop.

Because of this early termination, not all combinations of `i` and `j` are added—only those where `j < i`. The accumulated result of these additions produces a final `sum` value of 40, which is printed.



Zoho Schools for Graduate Studies



Notes

Day-9

METHODS

1. What is Method ?

Enclosing set of statement that performs unique task.

2. Syntax of Methods

```
[access modifier] [non-access modifier] returnType <methodName>([parameterList]) [throws]  
[Exception list] {  
    // set of statements  
}
```

3. Why we need a method or what is the primary purpose of a method?

- **Code Re-usability:** A method needs to be written only once, but it can be invoked multiple times from different places in the program whenever required.
- **Modularity:** Modularity breaking program into parts. Methods divide a large program into smaller, manageable, and organized pieces.

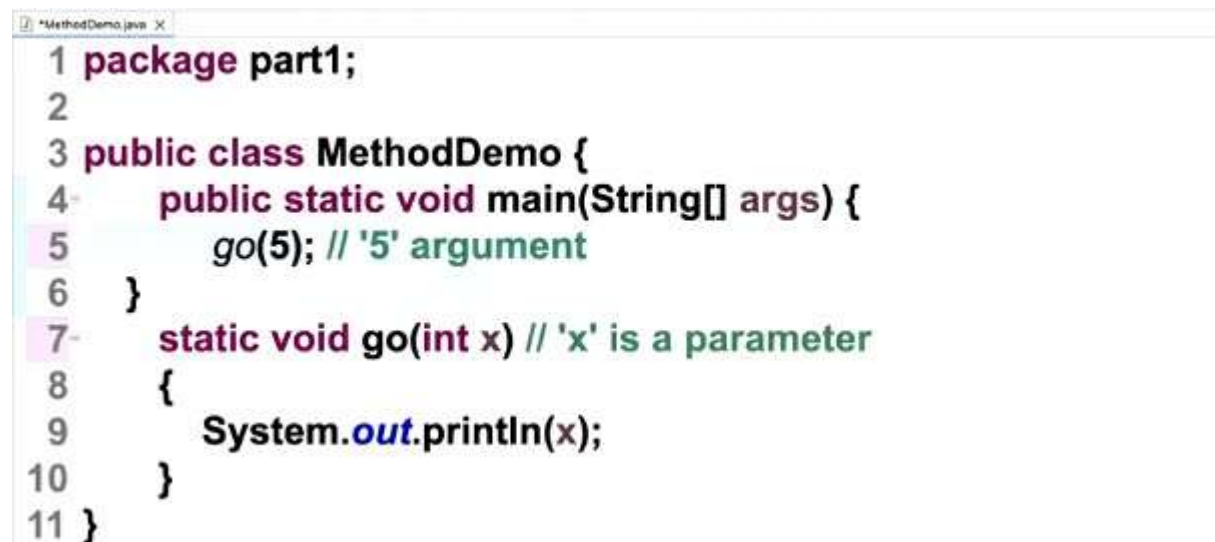
4. What is the advantages of Modularity?

- Easy debugging
- Improves code readability

5. Where generally in the program logic we write?

- All logic and executable statements are written inside methods.
- Business logic resides inside a method.

6. Parameter and Arguments



```
*MethodDemo.java X  
1 package part1;  
2  
3 public class MethodDemo {  
4     public static void main(String[] args) {  
5         go(5); // '5' argument  
6     }  
7     static void go(int x) // 'x' is a parameter  
8     {  
9         System.out.println(x);  
10    }  
11 }
```

7. What is the difference between parameter and arguments

- Parameters are variable(that stores the copy of an arguments)
- Arguments are data or value.

8. How many parameters method can have ?

- N numbers of parameters

9. Variable Length Arguments

```

1 package part1;
2
3 public class MethodDemo {
4     public static void main(String...args) { //variable length arguments
5         go(5); // '5' argument
6     }
7     static void go(int x) // 'x' is a parameter
8     {
9         System.out.println(x);
10    }
11 }

```

- varargs allow a method to accept zero or more arguments of the same type.
- Instead of fixing the number of parameters, you use ... (three dots) after the type.

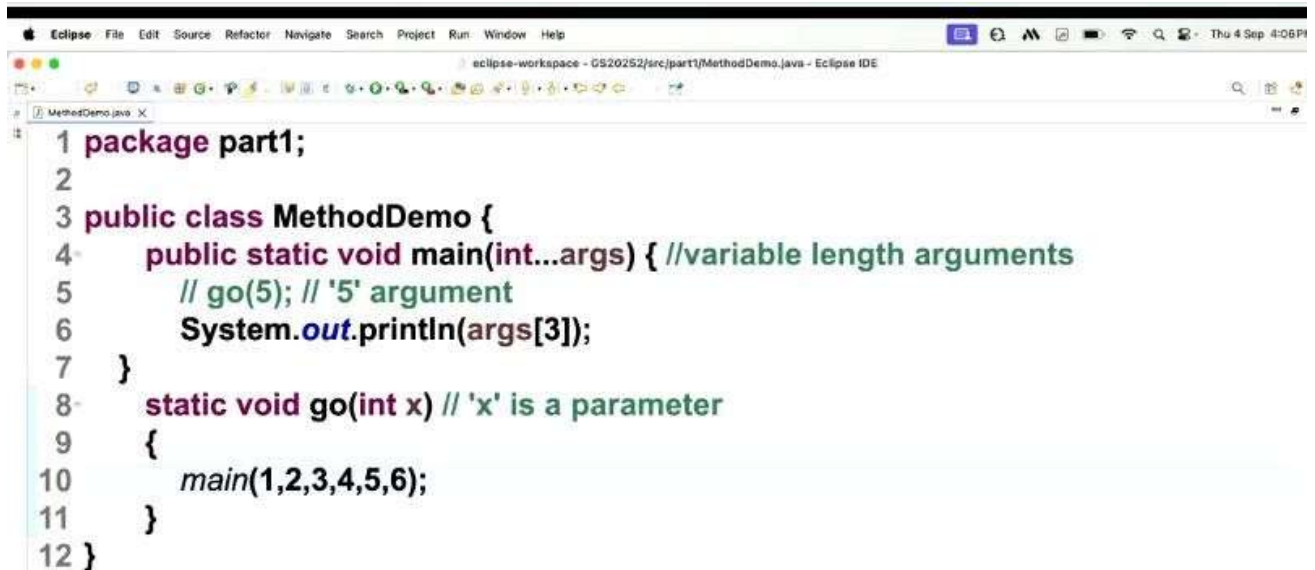
10. Can I call the main method from another method?

- Yes, It is perfectly legal to call the main method from another method. but the method which invoke the main method should be static.

```

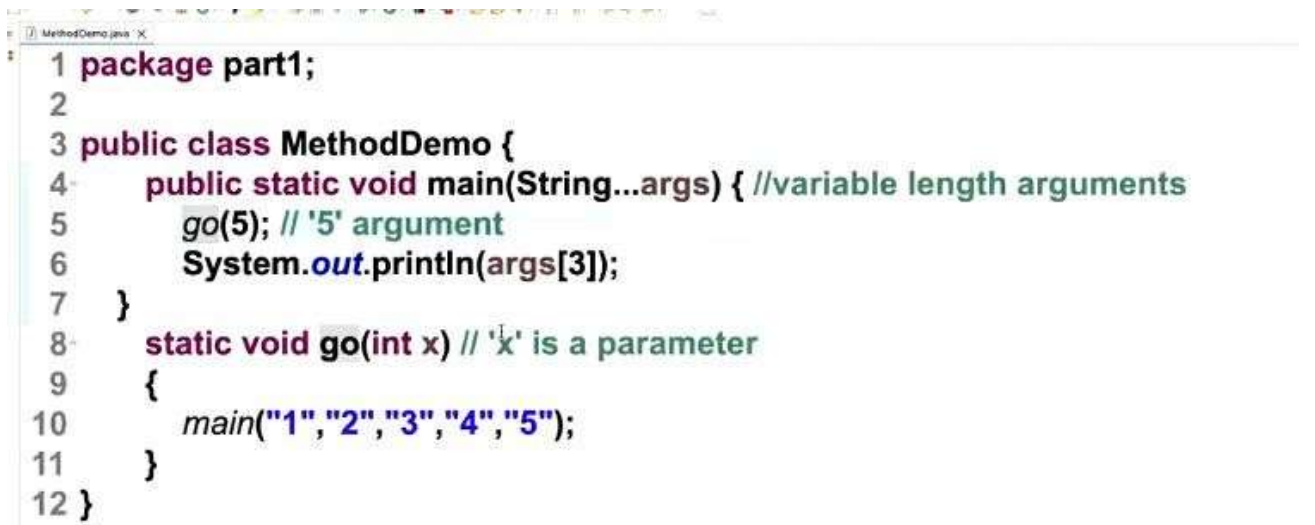
1 package part1;
2
3 public class MethodDemo {
4     public static void main(String...args) { //variable length arguments
5         // go(5); // '5' argument
6         System.out.println(args[3]);
7     }
8     static void go(int x) // 'x' is a parameter
9     {
10         main("1","2","3","4","5");
11     }
12 }

```



```
1 package part1;
2
3 public class MethodDemo {
4     public static void main(int...args) { //variable length arguments
5         // go(5); // '5' argument
6         System.out.println(args[3]);
7     }
8     static void go(int x) // 'x' is a parameter
9     {
10         main(1,2,3,4,5,6);
11     }
12 }
```

- **Output :** Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 3.
- Because, The statement `main("1", "2", "3", "4", "5");` is inside `go(int x)` but `go()` is never called.



```
1 package part1;
2
3 public class MethodDemo {
4     public static void main(String...args) { //variable length arguments
5         go(5); // '5' argument
6         System.out.println(args[3]);
7     }
8     static void go(int x) // 'x' is a parameter
9     {
10         main("1","2","3","4","5");
11     }
12 }
```

- **Output :** Exception in thread "main" java.lang.StackOverflowError
- Because there's no stopping condition, the program will keep calling `main` and `go` until the stack overflows.

```

1 package part1;
2
3 public class MethodDemo {
4     public static void main(String...args) { //variable length arguments
5         ho(1,2,3,4,5);
6     }
7     static void ho(int...a)
8     {
9         System.out.println(a[3]);
10    }
11 }

```

Output : 4

```

1 package part1;
2
3 public class MethodDemo {
4     public static void main(String...args) { //variable length arguments
5         ho(1,2,3,4,5);
6     }
7     static void ho(int...a)
8     {
9         System.out.println(a.length);
10    }
11 }

```

Output : 5

```

1 package part1;
2
3 public class MethodDemo {
4     public static void main(String...args) { //variable length arguments
5         ho(1,2,3,4);
6     }
7     static void ho(int b, int...a)
8     {
9         System.out.println(a.length + b);
10    }
11 }

```

Output : 4

The image shows two screenshots of a Java IDE. The top screenshot shows a code snippet where the varargs parameter 'b' is not the last parameter in the method signature 'ho(int...a, int...b)'. The bottom screenshot shows a similar snippet where the varargs parameter 'a' is not the last parameter in the method signature 'ho(int...a, int b)'. Both snippets are for a class 'MethodDemo' in package 'part1'.

```

1 package part1;
2
3 public class MethodDemo {
4     public static void main(String...args) { //variable length arguments
5         ho(1,2,3,4,5);
6     }
7     static void ho(int...a,int...b)
8     {
9         System.out.println(a.length + b);
10    }
11 }

```

```

1 package part1;
2
3 public class MethodDemo {
4     public static void main(String...args) { //variable length arguments
5         ho(1,2,3,4,5);
6     }
7     static void ho(int...a,int b)
8     {
9         System.out.println(a.length + b);
10    }
11 }

```

These code will not compile.
Because,

RULE 1 : when you use variable-length arguments it **must be the last parameter** in the method signature.

RULE 2 : A method can have at most one varargs parameter in its parentheses.

11. Can I declare a method with default keyword?

Yes can, but the method only appear in the interface, In other words Interface alone can have default methods.


```

1 package part1;
2
3 public class MethodDemo {
4     public static void main(String...args) { //variable length arguments
5         ho(1,2,3,4,5);
6     }
7     static String ho(int...a)
8     {
9         System.out.println(a.length);
10        return ;
11    }
12 }

```

Output : the code will not compile.
Because, Null is not a String.

If you want to return a empty string

```

1 package part1;
2
3 public class MethodDemo {
4     public static void main(String...args) { //variable length arguments
5         ho(1,2,3,4,5);
6     }
7     static String ho(int...a)
8     {
9         System.out.println(a.length);
10        return "";
11    }
12 }

```

```

1 package part1;
2
3 public class MethodDemo {
4     public static void main(String...args) { //variable length arguments
5         ho(1,2,3,4,5);
6     }
7     static String ho(int...a)
8     {
9         System.out.println(a.length);
10        return null;
11    }
12 }

```

- It is perfectly legal for a method to return `null`.
- `NULL` -> Means Undefined or Unknown

12. Can I interchange access modifier and non-access modifier ?

- Non-access modifier can appear before Access modifier.



Zoho Schools for Graduate Studies



Notes

Day-10

1. What are Overloaded Methods?

Methods that share the same name but differ in their parameter list are called **overloaded methods**.

Example:

```
package part1;
public class OverLoadDemo {
public static void main(String[] args)
{
OverLoadDemo o=new OverLoadDemo();
o.printMessage("Happy Teacher's Day!");
o.printMessage("Happy Teacher's Day to Mr. ", " Gnanavel");
}
void printMessage(String msg)
{
System.out.println(msg);
}
void printMessage(String msg1, String msg2)
{
System.out.println(msg1 + msg2);
}
}
```

2. When to prefer overloading (or) Why overloading?

Overloading is preferred whenever a similar type of operation needs to be performed on different kinds of data.

Example: `add(int a, int b)` and `add(double x, double y)`

3. What is the advantage of overloading?

Method overloading offers convenience to the programmer.

Example:

The developer does not need to create and remember unique method names whenever the same operation is performed on different data types.

4. What is the qualifying criteria to determine method overloading?

The following are the two main criteria:

1. Overloaded methods must share the same name.
2. Overloaded methods must differ in their parameter list (number of parameters, type of parameters, or order of parameters).

Rules :

1. Which overloaded method is invoked is decided at compile time based on the type of the arguments.

```
public class OverLoadDemo {
    public static void main(String[] args) {
        OverLoadDemo o = new OverLoadDemo();
        o.add((double)34, (float)56);
        o.add((float)34, (double)56);
    }

    void add(double a, float b) {
        System.out.println("DoubleFloat sum = " + (a + b));
    }

    void add(float a, double b) {
        System.out.println("FloatDouble sum = " + (a + b));
    }
}
```

OUTPUT:

```
DoubleFloat sum = 90.0
FloatDouble sum = 90.0
```

2. Overloaded methods must share same name

```
package part1;

public class OverLoadDemo {
    public static void main(String[] args) {
        OverLoadDemo o = new OverLoadDemo();
        o.add(34.0, 56.4f); // matches add(double, float)
        o.add(34.5f, 57.0); // matches add(float, double)
    }

    void add(double a, float b) {
        System.out.println("DoubleFloat sum = " + (a + b));
    }

    void add(float a, double b) {
        System.out.println("FloatDouble sum = " + (a + b));
    }
}
```

OUTPUT:

DoubleFloat sum = 90.4

FloatDouble sum = 91.5

3. Overloaded methods must vary in parameter list

```
class CorrectOverload {
    void add(int a, int b) {
        System.out.println("Sum (int,int) = " + (a + b));
    }
    void add(double a, double b) {
        System.out.println("Sum (double,double) = " + (a + b));
    }
    public static void main(String[] args) {
        CorrectOverload o = new CorrectOverload();
        o.add(10, 20);
        o.add(5.5, 6.5);
    }
}
```

OUTPUT:

Sum (int,int) = 30

Sum (double,double) = 12.0

4. Overloaded methods can vary in return type

```
package part1;
public class OverLoadDemo {
    public static void main(String[] args) {
        OverLoadDemo o = new OverLoadDemo();
        int r1 = o.add(34, 56);    // calls add(int,int)
        double r2 = o.add(34.5, 57); // calls add(double,int)
        System.out.println("Result from int add = " + r1);
        System.out.println("Result from double add = " + r2);
    }
    int add(int a, int b) {
        return a + b;
    }
    double add(double a, int b) {
        return a + b;
    }
}
```

OUTPUT:

Result from int add = 90

Result from double add = 91.5

5. Overloaded methods can vary in access modifier

```
class OverLoadDemo {  
    public void add(int a, int b) {  
        System.out.println("Public add(int,int) = " + (a + b));  
    }  
    private void add(double a, double b) {  
        System.out.println("Private add(double,double) = " + (a + b));  
    }  
    protected void add(int a, double b) {  
        System.out.println("Protected add(int,double) = " + (a + b));  
    }  
    public static void main(String[] args) {  
        OverLoadDemo o = new OverLoadDemo();  
        o.add(10, 20);  
        o.add(5.5, 6.5);  
        o.add(10, 6.5);  
    }  
}
```

OUTPUT:

Public add(int,int) = 30

Private add(double,double) = 12.0

Protected add(int,double) = 16.5

6. Overloaded methods can throw any new unchecked or checked Exceptions

```
package part1;  
import java.io.FileNotFoundException;  
public class OverLoadDemo {  
    public static void main(String[] args) throws Exception {  
        OverLoadDemo o = new OverLoadDemo();  
        o.add(34, 56.4f);    // may throw unchecked exception  
        o.add(34.5f, 57);    // no exception  
        o.add(10, 20);        // may throw checked + unchecked exception  
    }  
    static public void add(double a, float b) throws ArithmeticException {  
        System.out.println("DoubleFloat sum = " + (a + b));  
    }  
}
```

```

    }
    void add(float a, double b) {
        System.out.println("FloatDouble sum = " + (a + b));
    }
    static private int add(int a, int b) throws ArithmeticException,
FileNotFoundException {
        System.out.println("IntInt sum = " + (a + b));
        return (a + b);
    }
}

```

OUTPUT:

```

DoubleFloat sum = 90.4
FloatDouble sum = 91.0
IntInt sum = 30

```

7. Overloaded methods can throw zero or fewer or more exceptions

```

package part1;
import java.io.FileNotFoundException;
public class OverLoadDemo {
    public static void main(String[] args) throws Exception {
        OverLoadDemo o = new OverLoadDemo();

        o.add(10, 20);
        o.add(10.5, 20.5);
        o.add("Hello", "Java");
    }
    void add(int a, int b) {
        System.out.println("Sum (int,int) = " + (a + b));
    }
    void add(double a, double b) throws FileNotFoundException {
        System.out.println("Sum (double,double) = " + (a + b));
    }
    void add(String a, String b) throws FileNotFoundException,
ArithmeticException {
        System.out.println("Concatenation = " + (a + b));
    }
}

```

OUTPUT:

```

Sum (int,int) = 30
Sum (double,double) = 31.0
Concatenation = HelloJava

```

8. Overloaded methods can appear within the same class or can occur in the sub-class too.

9. Which overloaded method is invoked is only decided at compile time based on the reference type.

10. Method overloading is possible even without inheritance.

11. Static methods can be overloaded

```
package part1;
```

```
public class OverLoadDemo {  
    static void add(int a, int b) {  
        System.out.println("Static add(int,int) = " + (a + b));  
    }  
    static void add(double a, double b) {  
        System.out.println("Static add(double,double) = " + (a + b));  
    }  
    public static void main(String[] args) {  
        OverLoadDemo.add(10, 20);  
        OverLoadDemo.add(10.5, 20.5);  
    }  
}
```

OUTPUT:

```
Static add(int,int) = 30  
Static add(double,double) = 31.0
```

12. Method overloading is possible even without inheritance.

13. There must exist only one valid overloaded method for any call to a overloaded method. Otherwise, it leads to ambiguity when more than one valid overloaded method exist.

```
package part1;  
public class OverLoadDemo {  
    public static void main(String[] args) {  
        OverLoadDemo o=new OverLoadDemo();  
        o.add(34,56.4f);  
        o.add(34.5f,57);  
    }  
    void add(double a, float b)
```



```

{
System.out.println("DoubleFloat sum=" + (a + b));
}
void add(float a, double b)
{
System.out.println("FloatDouble sum=" + (a + b));
}

```

Explanation:

1. The error is **Ambiguity Error during method overloading resolution** (not runtime, but at compile-time).
2. You will get a compiler error: "reference to add is ambiguous". Because the compiler cannot decide which overloaded method to call when both are equally valid after type promotion.

14. 'main' method can be overloaded but cannot be overridden.

```

package part1;
public class OverLoadDemo {
    public static void main(String[] args) {
        System.out.println("Inside main(String[] args)");
        main(10);
        main("Hello Java");
    }
    public static void main(int a) {
        System.out.println("Inside main(int): " + a);
    }
    public static void main(String msg) {
        System.out.println("Inside main(String): " + msg);
    }
}

```

Output:

```

Inside main(String[] args)
Inside main(int): 10
Inside main(String): Hello Java

```

15. constructors can be overloaded but cannot be overridden.

```

package part1;
class Student {
    int id;

```

```

String name;
Student() {
    System.out.println("Default constructor called");
}
Student(int id) {
    this.id = id;
    System.out.println("Constructor with id: " + id);
}
}
public class OverLoadDemo {
    public static void main(String[] args) {
        Student s1 = new Student();
        Student s2 = new Student(101);

    }
}

```

OUTPUT:

Default constructor called
 Constructor with id: 101

```

package part1;
public class OverLoadDemo {
    public static void main(String[] args) {
        OverLoadDemo o = new OverLoadDemo();

        o.add(34, 56);    //int,int = exact match
        o.add(34.5, 56);  // double,int = matches add(double,int)

        //double cannot be passed to int
        o.add(34.5, 56.7);
    }
    void add(int a, int b) {
        System.out.println("Int version: " + (a + b));
    }
    void add(double a, int b) {
        System.out.println("Double version: " + (a + b));
    }
}

```

OUTPUT:

1. First two calls work fine and print results.
Int version: 90
Double version: 90.5
 2. Third call give compile-time error (double cannot be passed to int).
-

1. Vary in number of parameters

```
void s(int a) { }  
void s(int a, int b) { }
```

Explanation:

- Overloaded methods must differ in parameter list. Here, one method has 1 parameter, another has 2 parameters.
- Here, one method has **1 parameter**, another has **2 parameters**.

2. Vary in order of data types

```
void s(int a, float b) { }  
void s(float a, int b) { }
```

Explanation:

- Overloaded methods can have same number of parameters but different types or order.
- Here, both have 2 parameters, but order is different (`int, float` vs `float, int`).

3. Same parameter types (Not allowed)

```
void s(int a) { }  
void s(int b) { }
```

Explanation:

- Overloaded methods cannot have the same parameter types and same order.
- Only return type or variable name is different here, which does not count.
- This will give compile-time error: *“method s(int) is already defined”*.



Zoho Schools for Graduate Studies



Notes

Day-11

String Part - I

1. What is a String ?

In java , a String is an Object that represents a sequence of characters.

2. How is a String literal represented in Java?

In Java, a **String literal** is a sequence of characters enclosed within double quotes (" ").

```
package part1;

public class StringDemo{

    public static void main(String[] args)

    {

        String str = "Hello";

    }

}
```

String Literal in Python

In Python, a string literal is also represented by enclosing characters in quotes.

Single quotes ' '

Double quotes " "

Triple quotes ''' ''' or """ """ (for multi-line strings).

'Hello', "Hello", '''Hello'''

3. Which package does String class belongs to ?

In java, the String class belongs to the java.lang.package.

```
import java.lang.String;
```

4. What are the ways of creating a String ?

1. Using String Literal

One way of creating a String is by directly assigning a literal to a variable

```
String s1 = "Hello";
```

2. Using new Keyword

The other way is by using the new keyword.

```
String s2 = new String("Hello");
```

```
public class StringDemo{  
    public static void main(String[] argos)  
    {  
        String str1 = "Hello";  
        String str2 = "Hello";  
        String str3 = new String("Hello");  
    }  
}
```

5. What is the difference between creating a String using a literal and using the new keyword in java?

```
public class StringDemo{  
    public static void main(String[] args)
```

```

{
String str1 = "Hello";

String str2 = "Hello"; // String constant pool Heap memory

String str3 = new String("Hello");

System.out.println(str1 + " " +str2 + " "+str3);

System.out.println(str1==str2); // checks reference equality(memory address)

System.out.println(str2==str3); // Different memory location so false

}
}

```

Output:

Hello Hello Hello

true

False

('==') Equality operator checks of similarity of memory references .

```

String str1 = "Hello";

String str2 = "Hello"; // String constant pool

String str3 = new String("Hello");

System.out.println(str1 + " " +str2 + " "+str3);

System.out.println(str1==str2);

System.out.println(str2==str3);

System.out.println(str1.equals(str2)); // Check the content similarity

System.out.println(str1.equals(str3));

```

Output:

Hello HelloHello

true

false

true

True

Equality operator VS .Equals () Method:

== (Equality Operator)

Compares **references**
(**memory addresses**)

Checks reference equality only

Checks if both references point
to the same object in memory
(SCP vs Heap matters)

Only if both references point to
the exact same object

.equals () Method

Compares **contents/values**
(if overridden in class)

Same as == (Object's
default `equals ()` also
checks reference)

Overridden to compare
actual characters (content)

If the **values inside** objects
are same

6. Creating a Multi-Line String

MultiLine String can be enclosed within Triple double quotes closed in java

```
public class StringDemo {  
    public static void main(String[] args) {  
        String str1 = ""  
        Hello  
        Mndhbdhbdhb ///Perfectly legal in java  
        "";  
  
        String str2 = "Hello"; // String constant pool  
  
        String str3 = new String("Hello");  
  
        System.out.println(str1 + " " +str2 + " "+str3);  
  
        System.out.println(str1==str2);  
  
        System.out.println(str2==str3);  
  
        System.out.println(str1.equals(str2)); // Check the content similarity  
  
        System.out.println(str1.equals(str3));  
  
    }  
}
```

Output:

mndhbdhbdhb

Hello Hello

Whenever you used new operator then objects are created becomes a objects created only in Heap Memory.

Which is a preferred approach ?

Use String literals or Use new keyword

The choice depends on the **problem definition**

String are immutable can not modify the string

7. String Methods :

1. **length()** - Return length of string

```
System.out.pritnln(str2.length); - compiler error
```

```
String str2 = "Hello";
```

```
String str4 = "God";
```

```
System.out.println(str2.length()); // no of characters
```

Output:

5

2. **concat()** - joins two strings together

```
String str2 = "Hello"; //String constant pool
```

```
String str4 = "God";
```

```
str2.concat(str4); // can not modify the original string - output-Hello
```

```
System.out.println(str2);
```

```
//concatenation
```

```
str2 =str2.concat(str4); //output - HelloGod
```

Concat () - create new memory location will be created.

3. **SubString()** -

```
public class StringDemo {
```

```
    public static void main(String[] args) {
```

```
        String str2 ="Hello";
```

```

String str4="God";

str2=str2.concat(str4);

System.out.println(str2);

System.out.println(str2.substring(4,6)); //it will stop till 5th index
}

}

```

Output:

HelloGod

oG

```
System.out.println(str2.substring(4,4)); //used same index
```

Output:

Empty String

```
System.out.println(str2.substring(4,8)); // Substring end of last index
```

Output:

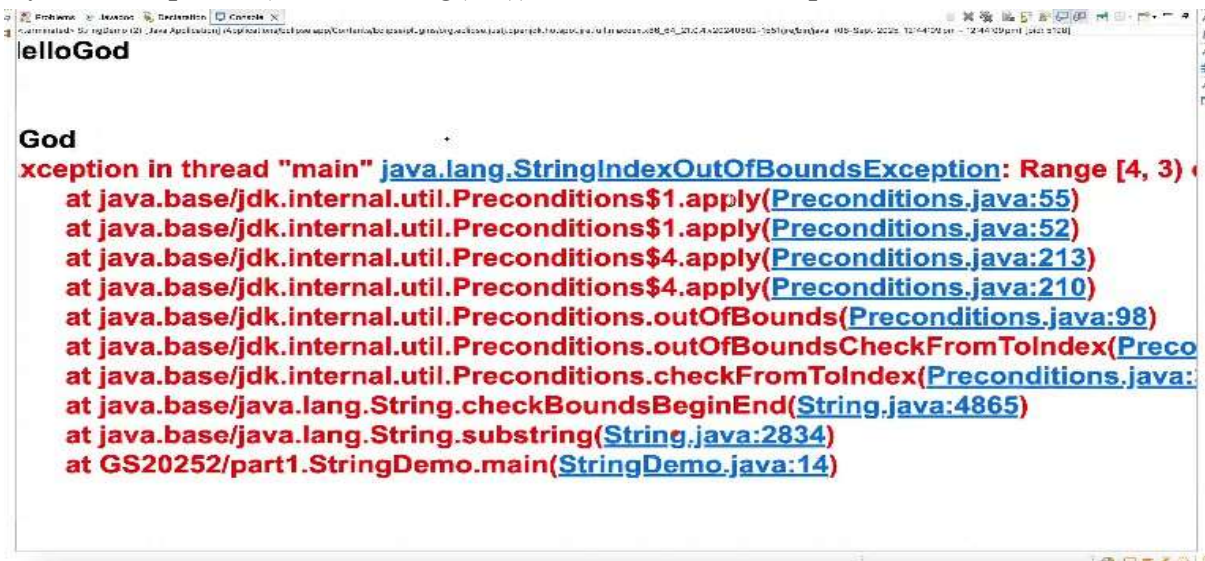
8

```
System.out.println(str2.substring(4,9)); // Runtime error
```



String index outOf bound Exception

System.out.println(str2.substring(4,3)); //Runtime Exceptions



System.out.println(str2.substring(4)); // From 4th index till end of the string

Output:

oGod

4. **startsWith()**- Checks whether a string begins with a given prefix.Returns true/false.

```
System.out.println(str2.startsWith("lo")); // false
```

5. **endsWith()**- Checks whether a string ends with a given suffix.Returns true/false.

```
System.out.println(str4.endsWith("od")); // true
```

6. **charAt()** - Returns the character at a specific position(index)in a string

```
System.out.println("str2.charAt(4): " + str2.charAt(4)); // 'o'
```

7. **indexOf()**- It accepts the string . four Overloaded method in indexOf().

```
System.out.println(str2.indexOf('o'));
```

Output:

4

```
System.out.println(str2.indexOf('f')); // -1 the character not available
```

8. **stripLeading()**-Removes all leading(starting) whitespace characters from a string.Similar to trim(), but trim() removes both leading and trailing spaces , while stripLeading() removes only from the start.

```
System.out.println(str2.stripLeading());
```

Output:

HelloGod

9. **stripTrailing()** - removes **trailing spaces** only.

Output:

HelloGod

10. **trim()** - Removes leading (starting) and trailing (ending) spaces from a string.

11. **getBytes()** - It converts a string into a equivalent byte array based on the character encoding.

Character take two bytes.

Output:

[B@17550481

12. **Arrays.toString.getBytes()**-Print the byte array values in readable way.

```
System.out.println(Arrays.toString(str2.getBytes()));
```

Space ascii value - 32

Output:

32,32,32,72,101,108,108,111,100,32,32,32] //Equivalent Integer.

13. **toCharArray()** -Converts a string into character array(char[]).Each character in the string becomes one element in the array.

```
System.out.println(Arrays.toString(str2.toCharArray()));
```

Output:

[, , , H,e,l,l,o,G,o,d, , ,]

```
public class StringDemo {  
    public static void main(String[] args) {  
        String str2 ="Hello";  
        String str4="God";  
        str2=str2.concat(str4);  
        System.out.println(str2);//HelloGod  
        System.out.println(str2.length());  
        System.out.println(str2.substring(4,4));  
        System.out.println(str2.substring(4,8));  
        System.out.println(str2.substring(4));  
        System.out.println(str2.indexOf("f"));  
        System.out.println(str2.stripLeading());  
        System.out.println(str2.stripTrailing());  
        System.out.println(str2.trim());  
        System.out.println(Arrays.toString(str2.getBytes()));  
        System.out.println(Arrays.toString(str2.toCharArray()));  
        System.out.println(str2.startsWith("he"));  
        System.out.println(str2.endsWith(" "));  
    }  
}
```

Output:

HelloGod

8

oGod

oGod

-1

HelloGod

HelloGod

HelloGod

[72, 101, 108, 108, 111, 71, 111, 100]

[H, e, l, l, o, G, o, d]

false

false



Zoho Schools for Graduate Studies



Notes
Day-12

ARRAYS

1)What are arrays?

Arrays are object that can store multiple elements or items or datum of similar data type in contiguous (or continuous) memory location.

2)How many ways you can declare an array?

There are 2 ways to declare an array.

3)How to declare an array?

1. `int[] a;` //java style of declaring an array
2. `int b[];` // c/c++ style of declaring an array

4) Legal ways to declare a 2-D array:

```
int[][] arr; // recommended way to declare a 2-D array
int [] c [];
int [] [] d;
int e [] [];
```

But these ways are not recommended.

5)Can we specify the size of array at the time of declaration?

No, we can never specify the size of array at the place of declaration.

```
//int[5] a;
//Gives compiler error
```

No memory will be allocated at the time of declaration.

Size of an array should be specified only at the time of creating or constructing or initializing the array.

6) Creating or constructing or initializing the array:

1.By using “new” keyword

```
int[] a = new int[5];
```

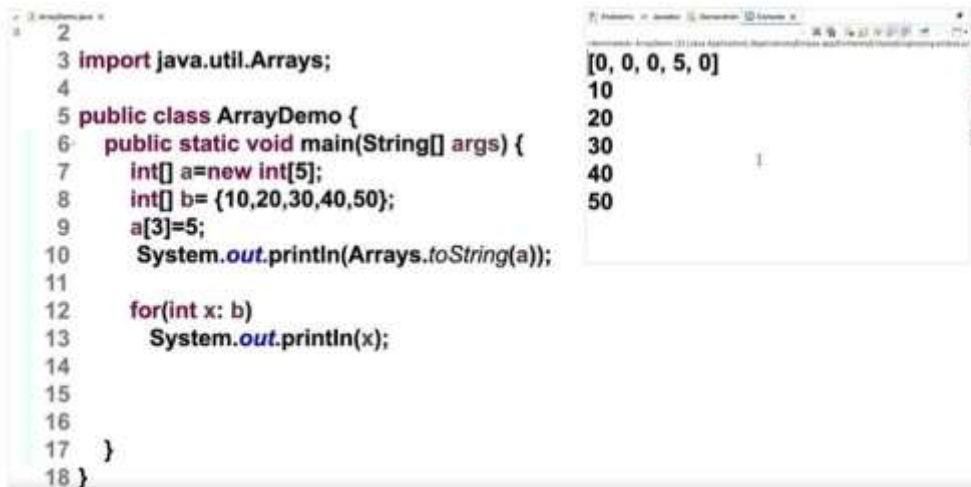
If we are not giving the size of the array means it gives compiler error.



2.Directly constructing the array or initializing the array

```
int[] b = {10,20,30,40,50};
```

Default initialization will apply even if the array is partially initialized.

The image shows a screenshot of an IDE with two windows. The left window displays Java code for an array demo. The right window shows the output of the program.

```
2
3 import java.util.Arrays;
4
5 public class ArrayDemo {
6     public static void main(String[] args) {
7         int[] a=new int[5];
8         int[] b= {10,20,30,40,50};
9         a[3]=5;
10        System.out.println(Arrays.toString(a));
11
12        for(int x: b)
13            System.out.println(x);
14
15
16
17    }
18 }
```

Output:

```
[0, 0, 0, 5, 0]
10
20
30
40
50
```

3.Initialising the array at the place of construction

```
int[] c = new int[]{1,2,3,4,5};
```

Note:

In second case,

```
int[] b;
b = {10,20,30,40,50};
```

We cannot do initialization and declaration separately. It gives compiler error.

In first case,

```
int[] a = new int[5];
a={1,2,3,4,5};
//Compiler error
```

*Arrays can be initialized directly only at the place of declaration.

In third case, can we specify the size of array?

Eg: `int[] c = new int[7]{1,2,3,4,5,6,7};`

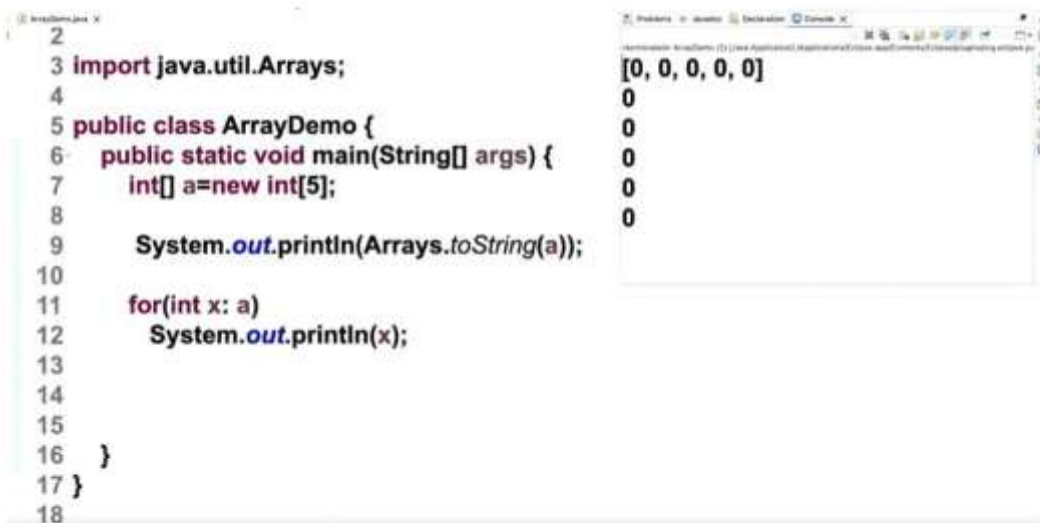
No, we cannot specify the size of the array here.

The size of the array must not be specified when the array is

initialized at the place of construction or instantiation or construction of an array.

7)How can we access the array elements?

- 1.Enhanced for loop or for each loop
- 2.Arrays.toString(); (present in utility package)



The screenshot shows an IDE with two panels. The left panel displays the source code for a class named `ArrayDemo`. The code imports `java.util.Arrays`, defines a `main` method that creates an integer array `a` of size 5, and prints the array using `Arrays.toString(a)` and each element using an enhanced for loop. The right panel shows the output of the program, which is the string `[0, 0, 0, 0, 0]` followed by five lines of `0`.

```
2  
3 import java.util.Arrays;  
4  
5 public class ArrayDemo {  
6     public static void main(String[] args) {  
7         int[] a=new int[5];  
8  
9         System.out.println(Arrays.toString(a));  
10  
11         for(int x: a)  
12             System.out.println(x);  
13  
14  
15  
16     }  
17 }  
18
```

```
[0, 0, 0, 0, 0]  
0  
0  
0  
0  
0
```

Arrays have default initialization even if it is local variable.

8)Anonymous array:

Directly defining the array at the place of creation.



The screenshot shows an IDE with a single panel displaying Java code. The code defines a `print` method that takes an integer array `a` and prints it using `Arrays.toString(a)`. In the `main` method, it creates an anonymous array `new int[] {10,20,30,40}` and prints it. It also demonstrates modifying an element of an array.

```
17     a[3]=5;  
18     System.out.println(Arrays.toString(a));  
19  
20     for(int x: b)  
21         System.out.println(x);  
22  
23     ad.print(new int[] {10,20,30,40}); // anonymous array  
24 }  
25 void print(int[] a) {  
26     System.out.println(Arrays.toString(a));  
27 }  
28 }
```

9)Is array a collection?

No, array is not a collection.

Reasons:

- 1.Collection are growable in nature. (dynamic)
Arrays are fixed in size. (static)
- 2.Collection can store either homogeneous or heterogenous data
Arrays can store similar data types.
- 3.Collections have rich inbuilt method support.
Arrays doesn't have rich readymade method support.
- 4.Collection can store only objects.
Arrays can store either primitive or objects.
- 5.Every collection is built on top of standard data structure.
Array does not have a standard data structure.

10? Why array index starts at 0?

The index of an array element often corresponds directly to memory address.

Starting from 0 simplifies the calculation of memory address.

For instance, the memory address of the first element is the base address of the array plus 0 times the size of each element, which simplifies to just the base address.

$$\text{arr}[\text{index}] = \text{base reference} + (\text{index} * \text{bytes})$$


```
31 a[0] = base reference (2000)
32 a[0] = 2000 to 2003
33 a[1] = 2004 to 2007
34 a[2] = 2008 (2000 + 2*4)
35
36 a[5] = 2000 + (5*4) = 2020
```

11) What is the output of the code snippet?

```
23 int j[][] = new int[4][];
24 j[0] = new int[]{0,1,3};
25 j[1] = new int[]{6,4};
26 j[2] = new int[]{1,7,6,8,9};
27 j[3] = new int[]{5};
28 System.out.println(j[2][2] == j[1][0]);
29
```

The given array is an example for jagged array.

Jagged array - Each row can have number of columns. (Dissimilar length of elements)

Output: true

12) Cloning of an array:

Creating an identical copy of an object called object cloning.

13) What is the output of code snippet?

```

1 package part1;
2
3 public class Array2Demo {
4     public static void main(String[] args) {
5         int arr[] = {10,20,30};
6         int cloneArr[] = arr.clone();
7         System.out.print(arr==cloneArr);
8         arr[1]=200;
9         System.out.print(arr[1]==cloneArr[1]);
10    }
11 }

```

Output:

```

1 package part1;
2
3 public class Array2Demo {
4     public static void main(String[] args) {
5         int arr[] = {10,20,30};
6         int cloneArr[] = arr.clone();
7         System.out.print(arr==cloneArr);
8         arr[1]=200;
9         System.out.print(arr[1]==cloneArr[1]);
10    }
11 }
12
13

```

```

falsefalse

```

14)Cloning of one-dimensional array:

```

1 package part1;
2
3 public class Array2Demo {
4     public static void main(String[] args) {
5         int arr[] = {10,20,30};
6         int cloneArr[] = arr.clone();
7         System.out.println(arr==cloneArr);
8         System.out.println(arr);
9         System.out.println(cloneArr);
10        arr[1]=200;
11        System.out.println(arr[1]==cloneArr[1]);
12    }
13 }

```

```

false
[I@17550481
[I@735f7ae5
false

```

When we clone the one-dimensional array, it will not point to the original object.

In one dimensional array it will do deep copy.

When we create a cloning of one-dimensional array it will create a new object. If we did modification in original array means it does not affect the cloned array.

15)Cloning of two-dimensional array:

Shallow copy happens in two-dimensional cloning.

```
1 package part1;
2
3 public class Array2Demo {
4     public static void main(String[] args) {
5         int arr[][]= {{1,2,3,4},{5,6}};
6         int cloneArr[][]=arr.clone();
7         System.out.println(arr);
8         System.out.println(cloneArr);
9         System.out.println(arr[1]==cloneArr[1]);
10        arr[1][1]=200;
11        System.out.println(arr[1][1]==cloneArr[1][1]);
12    }
13 }
```

Output:

```
1 package part1;
2
3 public class Array2Demo {
4     public static void main(String[] args) {
5         int arr[][]= {{1,2,3,4},{5,6}};
6         int cloneArr[][]=arr.clone();
7         System.out.println(arr);
8         System.out.println(cloneArr);
9         System.out.println(arr[0]);
10        System.out.println(cloneArr[0]);
11        System.out.println(arr[1]==cloneArr[1]);
12        arr[1][1]=200;
13        System.out.println(arr[1][1]==cloneArr[1][1]);
14    }
15 }
```

```
[[I@64616ca2
[[I@735f7ae5
[1@180bc464
[1@180bc464
true
true
```

Shallow Copy:

Only the outer object is copied. Inner reference-type fields (objects) are shared between the original and the clone.

Deep Copy:

Both the outer object and all inner referenced objects are cloned. So changes in one object do not affect the other.

16)What is the output of the following code snippet upon execution?

```
int[] a=new int[] {2,6,23,44,55};  
System.out.println(Arrays.binarySearch(a, 50));
```

- 1
- 2
- 3
- 5

Arrays.binarySearch():

The Arrays.binarySearch() method is part of the java.util.Arrays class and implements the binary search algorithm.

Binary search is an efficient search algorithm that works on sorted arrays by repeatedly dividing the search space in half.

If the key is found, the method returns its index. If the key is not found, it returns a negative value representing the insertion point.

Output:

```
1 package part1;  
2 import java.util.Arrays;  
3  
4 public class Array2Demo {  
5     public static void main(String[] args) {  
6         int[] a=new int[] {2,6,23,44,55};  
7         System.out.println(Arrays.binarySearch(a,  
8     }  
9 }
```

-5

17)What is the output of the following code snippet upon execution?

```

1 package part1;
2 import java.util.Arrays;
3
4 public class Array2Demo {
5     public static void main(String[] args) {
6         int[] a= {6, -4, 12,0,-10 };
7         int x=12;
8         System.out.println(Arrays.binarySearch(a, x));
9     }
10 }

```

Output:

```

1 package part1;
2 import java.util.Arrays;
3
4 public class Array2Demo {
5     public static void main(String[] args) {
6         int[] a= {6, -4, 12,0,-10 };// -10 -4 0 6 12
7         int x=12;
8         System.out.println(Arrays.binarySearch(a,
9     }
10 }

```

2



Zoho Schools for Graduate Studies



Notes

Day-13

WHAT IS JAVA ?

Java is a

- Simple,
- Powerful,
- Platform Independent,
- Purely Object Oriented,
- Multi – Threaded,
- High level Programming language.

Explanation for these Features :

Simple – It retains the Syntax of c and C++ syntax.

Powerful - Java is powerful because it is **platform-independent, object-oriented, secure, robust, and supported by rich libraries with strong community support.**

Why java becomes platform independent ?

Java follows WORA (Write Once, Run Anywhere) architecture.

Byte codes made java platform independent because it can run on any system with a JVM.

In java , the source code is compiled by the Java compiler into intermediate bytecode, the JVM translates the byte code to platform specific code.

Note : any language can achieve platform independence if it uses an intermediate representation + a virtual machine (just like Java).

What Qualifies the language to be Object – Oriented ?

The language qualifies to be Object Oriented if and only if it satisfies the three object oriented principles :

- Encapsulation
- Inheritance
- Polymorphism

Multi-Threaded :

Java have the important feature of multi threading where we can do multiple tasks concurrently,with in the same program.

High Level Programming Language :

Instructions Given to the machines in user – friendly / user – understandable language

Note : Low level Programming languages , where the instructions where given to the machines in machine understandable form (like 0's and 1's)

Additional Info on other languages :

1) why Java Script is Object based language ?

Java Script supports Encapsulation have only classes and objects , but does not support inheritance. So java script is object bases language.

2) Why C++ is partially object Oriented ?

C++ is partially object-oriented because we can write code with classes or without classes. Java is fully object-oriented because everything must be inside a class.

History Of Java :

- **1991** → James Gosling, Patrick Naughton, and Mike Sheridan at Sun Microsystems started a project called the Green Project.
- **Objective of Green Project** → To create a language for small electronic devices (like remote controls) that could be platform-independent.
- The first language they made was called Oak (named after an oak tree outside Gosling's office).
- Later, **Oak** had to be renamed because another company already had a product with that name.
- They chose the name **Java** because the team drank a lot of Java coffee, and they wanted a short, unique, and catchy name

Java Editions :

1. Java Platform, Standard Edition (Java SE)
2. Java Platform, Enterprise Edition (Java EE)
3. Java Platform, Micro Edition (Java ME)

Features of java :

Java is Secure :

- Java uses its own **JRE**, part of **JVM (Sandbox)** for execution.
- Predecessor languages used the **runtime environment of the operating system**.
- **Class loader** loads classes into JVM, and **Bytecode verifier** checks for malicious code.

Java is Robust

- Robust means “**Strong**”.
- Memory management mistakes and runtime errors cause program failures.
- Automatic memory management prevents explicit harm to memory.
- Exception handling addresses mishandled runtime error

Interpreted and High Performance

- **Interpreter** converts bytecode into machine-readable binary code.
- Java uses **JIT (Just-In-Time compiler)** for very high performance.
- Interpreters are relatively slower, but **JIT compiler improves processing speed**.

Portable

- Java programs can run across different platforms without modification.
- Platform independence + standard libraries make Java portable.

Architecture Neutral

- Bytecode is designed to be independent of processor architecture.

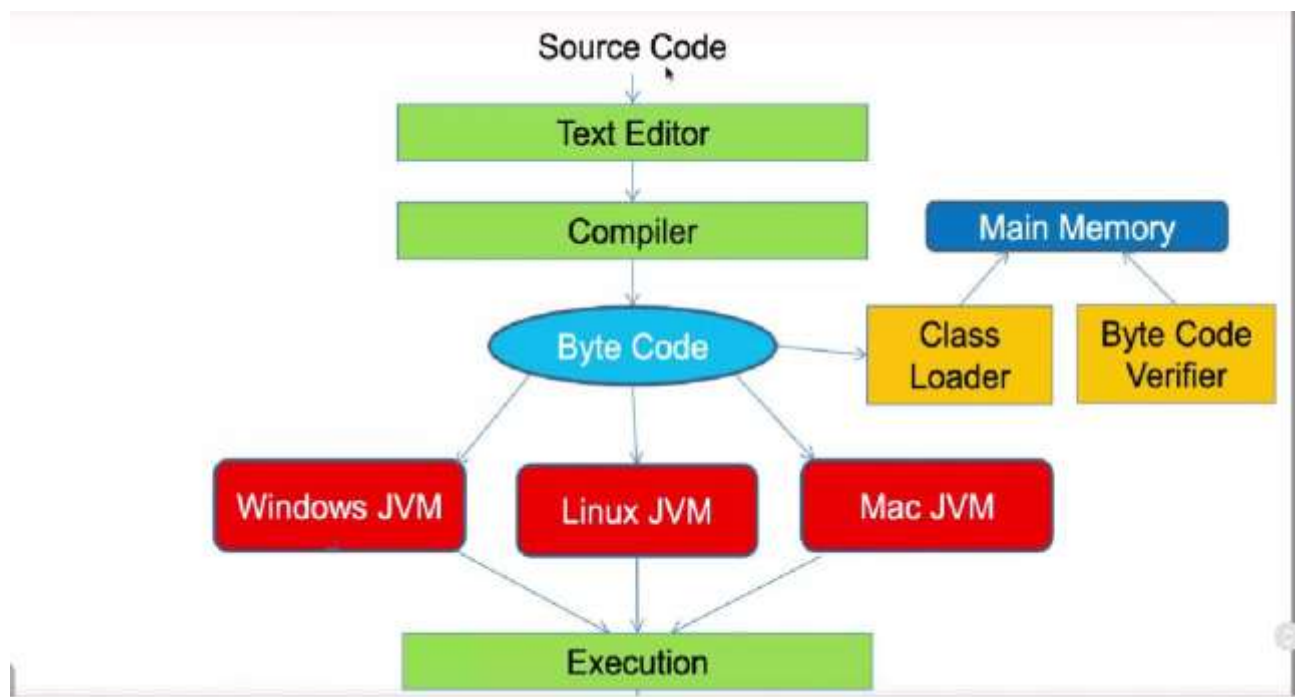
Dynamic & Extensible

- Classes can be loaded dynamically at runtime.
- Java supports integration with other languages (e.g., C/C++ using JNI).

Uses of Java

- **Desktop Applications** → e.g., calculators, media players, IDEs
- **Web Applications** → e.g., e-commerce sites, online portals
- **Mobile Applications** → especially Android apps (since Android uses Java).
- **Networking Applications** → e.g., chat apps, email servers, online games.
- **Enterprise Applications** → e.g., banking software, ERP, large-scale business systems.
- **Scientific Applications** → e.g., simulations, research tools.
- **Game Development** → both desktop and mobile games.

Working of Java Program Execution



- Java program is written as **source code** using a text editor.
- The source code (.java file) is compiled by the **Java Compiler** into **Byte Code** (.class file).

- **Byte Code** is platform-independent and can run on any operating system with JVM.
- The **Class Loader** loads the byte code into **main memory**.
- The **Byte Code Verifier** checks the byte code for validity and security.
- The **Java Virtual Machine (JVM)**, specific to each OS (Windows JVM, Linux JVM, Mac JVM), converts the byte code into machine code.
- The machine code is executed by the CPU, producing the program's output.



Zoho Schools for Graduate Studies



Notes

Day-14

What is a real-world object?

It is a Tangible entity that has a well-defined boundary. Hence, they have some shape or structure.

Classify real-world objects?

a. Living (Animate)

Herbivore, Carnivores, Omnivores (Similarity in attributes or behavior)

b. Non-living (Inanimate)

What is behavior?

Actions exhibited by an Object are termed its behavior.

Do all objects exhibit a behavior?

No.

What is the cause of the behavior of an Object?

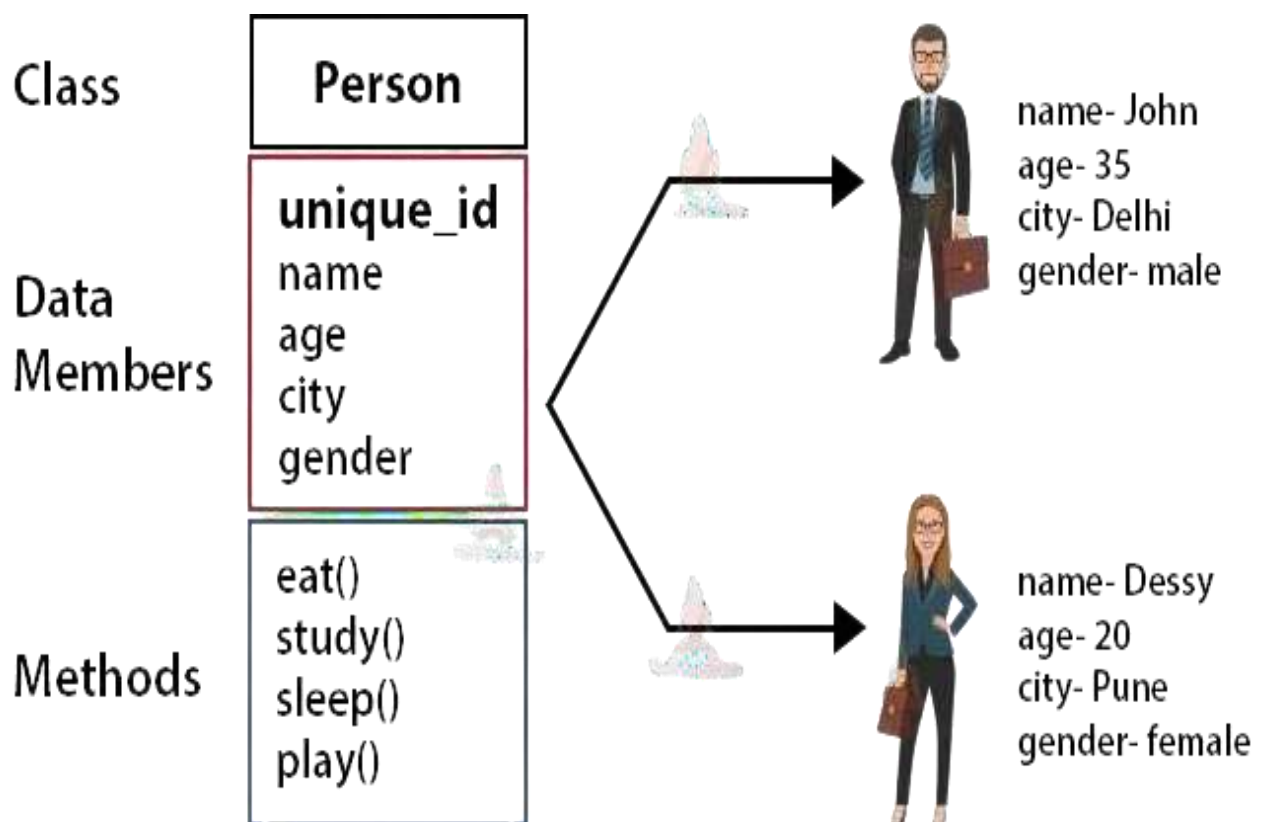
Mind

Which Object exhibits behavior?

Non-living objects don't exhibit any behavior due to the absence of a mind.

What are attributes or fields, or properties of an Object?

The physical characteristics of an object constitute its attributes or fields, or properties.



What is a class?

It is a model (blueprint, prototype, or template) of an Object that captures its attributes and behavior.

How to model an object?

- 1) Create an instance variable for every required attribute.
- 2) Create an instance method for every required action.

How many objects can be constructed from a class?

N - number of objects can be constructed that are identical.

What is an Object?

It is an intangible entity that is obtained from its class using the 'new' operator.

Object creation = Instantiation.

How are the members of an object accessed?

The members of an Object are accessed using the period or dot operator.

```
package part2;
```

```
public class Pen{
```

```
    String color;
```

```
    int price;
```

```
    void write() {
```

```
        System.out.println("Pen is used for writing ..");
```

```
    }
```

```
}
```

```
package part2;
```

```
public class PenDemo{
```

```
    public static void main(String[] args){
```

```
        Pen p;
```

```
        p=new Pen();
```

```
        System.out.println(p.color);
```

```
        System.out.println(p.price);
```

```
        p.write();
```

```
}
```

```
==IBP 11'1! _.:l's  
ull  
0  
Pen is used for writing...
```



Zoho Schools for Graduate Studies



Notes

Day-15

Modeling the 'Car' Object

Attributes (Instance Variables):

- `String color`: To store the color of the car.
- `float mileage`: To store the car's mileage (kilometers per liter).

Behavior (Method):

- `void start()`: A method to represent the car starting, which prints "Car is started...".

```
1 package part2;
2
3 public class Car {
4     String color;
5     float mileage;
6
7     void start() {
8         System.out.println("Car started...");
9     }
10 }
11
```


The Problem with Direct Access

- A `CarDemo` class was created to instantiate and use the `Car` object.

Initial Code Problem: The attributes (`color`, `mileage`) were directly accessed from the `CarDemo` class (e.g., `c.color`).

```
// In CarDemo.java
public class CarDemo {
    public static void main(String[] args) {
        Car c = new Car();
        System.out.println(c.color);
        System.out.println(c.mileage);
        c.start();
    }
}
```

Output:

null

0.0

Car started...

- **Key Issue Identified:**

This is a "bad programming practice." One class's members should not be directly accessible by another class. This can lead to uncontrolled modifications (e.g., "anyone can modify a member directly").

Introducing Encapsulation: The **private** Keyword

To prevent direct access, the **color** and **mileage** attributes in the **Car** class were declared as **private**.

```
// In Car.java
public class Car {
    private String color;
    private float mileage;
    // ... methods
}
```

- A **private** member is only accessible **within its own class**.
- This change caused a **compilation error** in **CarDemo** because **c.color** was no longer visible.

Indirect Access: Getters and Setters

- Direct access is prevented, but we still need a controlled way to read and modify the private attributes. This is achieved through public methods.

Getter (Accessor Method)

- A method designed to **read** the value of a private attribute.

Example: **public String getColor()** returns the value of the **color** variable.

```
// In Car.java
// getter or accessor method

String getColor() {
    return color;
}
```

Setter (Mutator/Modifier Method)

- A method designed to **modify** the value of a private attribute.

Example: `public void setColor(String color)` takes a new color as an argument and assigns it to the private `color` variable.

```
// In Car.java  
// setter or modifier or mutator method  
  
public void setColor(String c) {  
    color = c;  
}
```

Car.java (Encapsulated)

```
public class Car {  
    private String color;  
    private float mileage;  
  
    // Getter for color  
    public String getColor() {  
        return color;  
    }  
  
    // Setter for color  
    public void setColor(String newColor) {  
        color = newColor;  
    }  
  
    // ... other methods  
}
```

The `this` Keyword

- A problem arises if the parameter name in a setter is the same as the instance variable name (e.g., `setColor(String color)` where the instance variable is also `color`).

The line `color = color;` becomes ambiguous. The compiler prioritizes the local variable.

- **Solution:**

Use the `this` keyword. `this` is a reference to the **currently invoked object**.

Correct Implementation: `this.color = color;` explicitly states that the instance variable (`this.color`) should be assigned the value of the parameter (`color`).

Ambiguous Code (Incorrect)

```
public void setColor(String color) {  
    color = color;  
    // Assigns parameter to itself!  
}
```

Clear Code with 'this' (Correct)

```
public void setColor(String color) {  
    this.color = color;  
    // Instance var = parameter  
}
```

Encapsulation

- **Definition:** The process of binding (or bundling) attributes and the methods that operate on them into a single logical unit, the class.
- Implementing a class itself is an act of encapsulation.
- **Fully Encapsulated (or Strictly Encapsulated) Class:** A class where all instance variables are **private** and all methods are **public**. This is the ideal to aim for.

Coupling

- **Definition:** The degree of dependency one object has on another.
- **Goal:** Aim for **Loose Coupling** (or Weak Coupling). Objects should be as independent as possible, with minimal reliance on the internal workings of other objects.

Cohesion

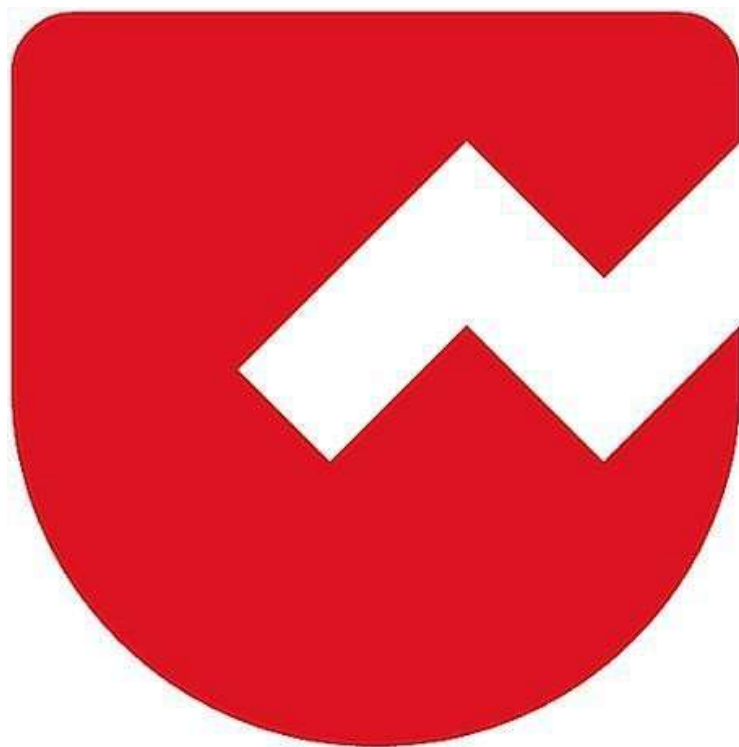
- **Definition:** The degree to which the elements inside a single module (or class) belong together. It's a measure of self-dependency.
- **Goal:** Aim for **High Cohesion**. A class should be focused on a single, well-defined purpose.

Summary of Best Practices

1. Mark instance variables/attributes as **private**.
2. Provide **public getter and setter methods** for controlled access to private attributes.
3. Design objects to be **loosely coupled** and **highly cohesive**.
4. An "object reference" (like `c` in `Car c = new Car();`) is not a direct memory address but a unique, system-assigned alphanumeric string used to identify the object.



Zoho Schools for Graduate Studies



Notes

Day-16

Constructor

Definition :

- A constructor is a special method inside a class.
- It is invoked implicitly at the time of object creation.
- It is used to initialize the state of an object.
- We can't invoke constructor like a normal method whenever required.

State of an Object :

- The value of the instance variables at a given instant of time is referred to as the state of an object.

Characteristics of Constructor :

- Cannot be invoked explicitly like a normal method.
- Does not have a return type.
- Shares the class name.

Purpose of Constructor :

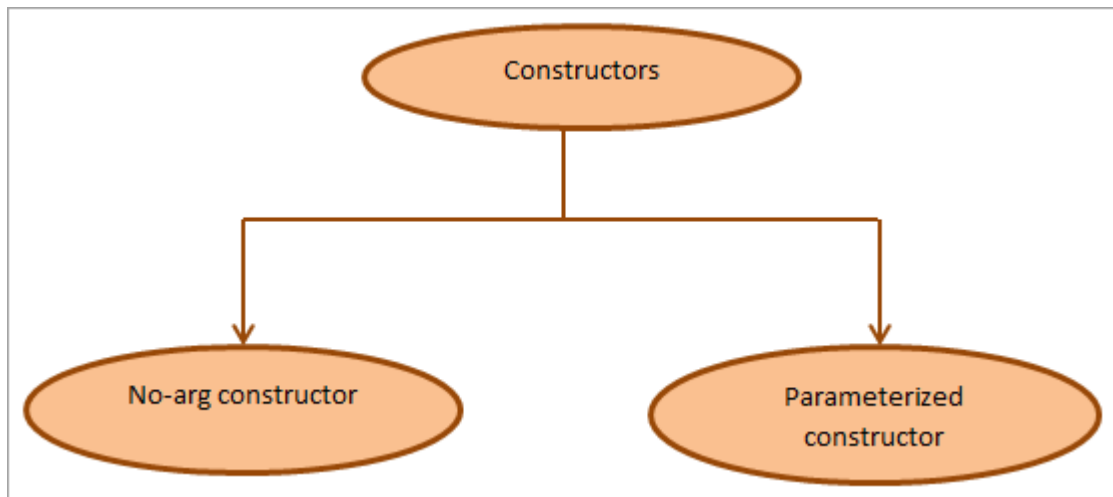
- Constructors are generally used to initialize state of an object

What is a Constructor?

- A special type of method invoked implicitly at the time of instantiation of an object, for the purpose of initializing the object.

Types of Constructors :

1. Parameterized Constructor → accepts arguments.
2. Non-Parameterized Constructor → no arguments.



Parameterized Constructor :

- A constructor that accepts arguments to initialize the object.

Non - Parameterized (or) Parameterless Constructor:

- A constructor that does not accept arguments.
- Supplied by default only if there is no constructor in the class.

Copy Constructor :

- A constructor used to initialize the state of an object similar to another object through a constructor its termed as copy constructor.
- Used in object cloning.

Object Cloning :

- Creating an identical copies of an object is referred as object cloning.


```

2
3 public class Book {
4     String title;
5
6     Book(){ // parameterless constructor or zero argument
7         this("Head First Java!");
8     }
9
10    Book(String title){ // parameterized constructor
11        this.title=title;
12    }
13
14    Book(Book b){ // copy constructor - object cloning
15        Book(b.title);
16    }
17
18 }
19

```

Constructor Overloading :

- Constructors with the same name but different parameter lists are called overloaded constructors.
- Allows creating objects in different ways.

```

class Student {
    String name;
    int age;
    String course;

    // Constructor 1 – No arguments
    Student() {
        name = "Unknown";
        age = 0;
        course = "Not Assigned";
    }

    // Constructor 2 – One argument
    Student(String n) {
        name = n;
        age = 0;
        course = "Not Assigned";
    }
}

```

```
}  
  
// Constructor 3 – Two arguments  
Student(String n, int a) {  
    name = n;  
    age = a;  
    course = "Not Assigned";  
}  
}
```

Constructor Chaining :

- Calling one constructor from another in the same class using `this()`.

Function Chaining :

- A chain of successive function calls is termed as Function Chaining.

Significance of 'this' reference :

- Resolves naming conflicts between instance variables and local variables.
- Can be used to call another constructor (`this()`).

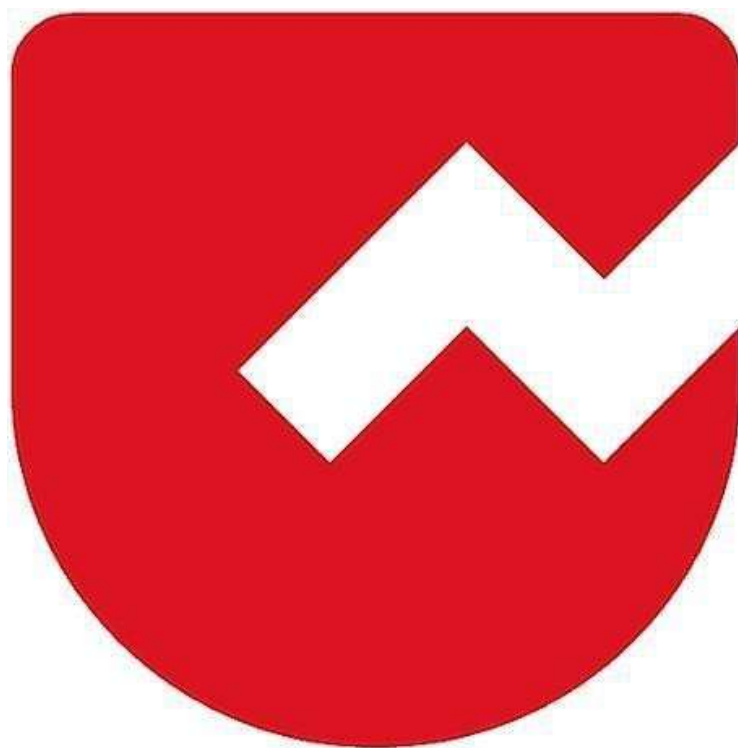
Notes :

- A zero argument (or) parameterless constructor is applied implicitly only in the absence of any constructor in the class.
- A parameterless constructor is applied by default only in the absence of any constructor in a class.
- It's perfectly invoke a normal method from a constructor.
(bad programming practice)
- A constructor can't be invoked like a normal method when required.

- `this()` call if present must appear as a first state in a constructor.
- It's perfectly legal for a normal method to share the class name.
(bad programming practice)



Zoho Schools for Graduate Studies



Notes

Day-17

INHERITANCE

WHAT IS INHERITANCE:

- It is a property by which one class(sub-class) acquires the **inheritable members** of another class(super-class).

```
public class Animal { // base class or parent class
    int lifespan;

    void eat() {
        System.out.println("Animal eats");
    }
}
```

```
public class Dog extends Animal {
    // Dog has no member inside the class
    // but it inherit members from Animal
    // Hence the members lifespan & eat() used in both Animal & Dog class
    // to avoid code replication
}
```

```
public class InheritanceDemo {
    public static void main(String[] args) {
        Dog d = new Dog();
        System.out.println(d.lifespan);
        d.eat();
    }
}
```

NOTE:

- Subclass will not inherit all members from superclass.
- Subclass use “**extends**” keyword to inherit from its superclass.

WHAT ARE THE ADVANTAGES OF INHERITANCE :

- **Code Reusability** or to avoid code redundancy or duplication or replication.
- To obtain **package access control**/package private access control/default access control.

WHEN TO PREFER INHERITANCE? OR WHY INHERITANCE?

- Inheritance is preferred whenever a class wants to reuse or share **certain members**(can be attributes or behaviours) of another class.

SUBCLASS:

- The class that is **inheriting** is referred as subclass or child class or derived class.

SUPERCLASS:

- The class that is **inherited** is referred as superclass or base class or parent class.

```
public class Animal { // base class or parent class
    int lifespan;

    void eat() {
        System.out.println("Animal eats");
    }
}
```

```
public class Dog extends Animal { //Derived class or child class or sub class
    void bark() {

        System.out.println("Dog barks lol.. lol.. lol..");
    }

    void eat() { // method overriding
        // want to capture specific behavior of Dog
    }
}
```

```

        super.eat();

        System.out.println("Dog eats milk, meat, biscuit...");
    }

    void wagTail() {

        System.out.println("Dog wags its Tail...");

    }
}

```

```

public class InheritanceDemo {

    public static void main(String[] args) {

        Dog d = new Dog();

        System.out.println(d.lifespan);

        d.eat();

    }

}

```

NOTE:

- The **super keyword** is used to inherit the parent class members.
- It is also used by the subclass to access superclass methods or constructors.

QUESTION:

- ✓ If Animal's eat() is inherited to Dog? — Yes.
- ✓ How many members for Dog cls? — 5

WHEN METHOD OVERRIDING POSSIBLE?

- Method overriding is possible if and only if the super class method is inheritable

METHOD OVERLOADING VS METHOD OVERRIDING

1. Method Overloading

Two or more methods in the same class with the same name but different parameter lists.

Example:

```
Aclass Animal {  
    void eat() {  
        System.out.println("Animal eats");  
    }  
}  
  
class Dog extends Animal {  
    void eat(String food) { // Overloading with parameter  
        System.out.println("Dog eats milk, meat, biscuit...");  
    }  
}  
  
public class OverloadDemo {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.eat(); // Dog eats food  
        d.eat("bone"); // Dog eats bone  
    }  
}
```


2. Method Overriding

Subclass provides its own implementation of a method that is already defined in the superclass.

```
class Animal {  
    void eat() {  
        System.out.println("Animal eats");  
    }  
}
```

```
class Dog extends Animal {  
    void eat() { // Overriding parent class method  
        super.eat(); // call parent method  
        System.out.println("Dog eats milk, meat, biscuit...");  
    }  
}
```

```
public class OverrideDemo {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.eat();  
        // Animal eats  
        // Dog eats milk, meat, biscuit...  
    }  
}
```

HOW IS THE METHOD OVERRIDING OBTAINED?

LEGAL RULES FOR METHOD OVERRIDING :

1. Overridden method must share the same method signature.
2. Overridden method can retain the same access level or can have a broader access level.
3. Overridden methods can have the same return type or any subtype of the superclass return type (covariant return type).

```
public class Animal {  
    int lifespan;  
  
    public Number eat() {  
        System.out.println("Animals eat");  
        return 57;  
    }  
}
```

```
public class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barks loud... loud... loud...");  
    }  
  
    void wagTail() {  
        System.out.println("Dog wags its tail...");  
    }  
  
    public Integer eat() {  
        System.out.println("Dog eats milk, meat, biscuit...");  
        return 20;  
    }  
}
```

```
public class InheritanceDemo {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
    }  
}
```

```
        int age = d.eat();
        System.out.println(age);
    }
}
```

Output:

```
Dog eats milk, meat, biscuit...
20
```

NOTES:

- Order of access control: private, default, protected, public.

HOW PROTECTED WORKS IN PARENT VS CHILD:

Protected in Parent Class

```
class Animal {
    protected void eat() {           // protected method
        System.out.println("Animal eats");
    }
}
```

```
public class ProtectedParentDemo {
    public static void main(String[] args) {
        Animal a = new Animal();
        a.eat(); // Allowed (same package)
    }
}
```

Protected in Child Class (Access through Inheritance)

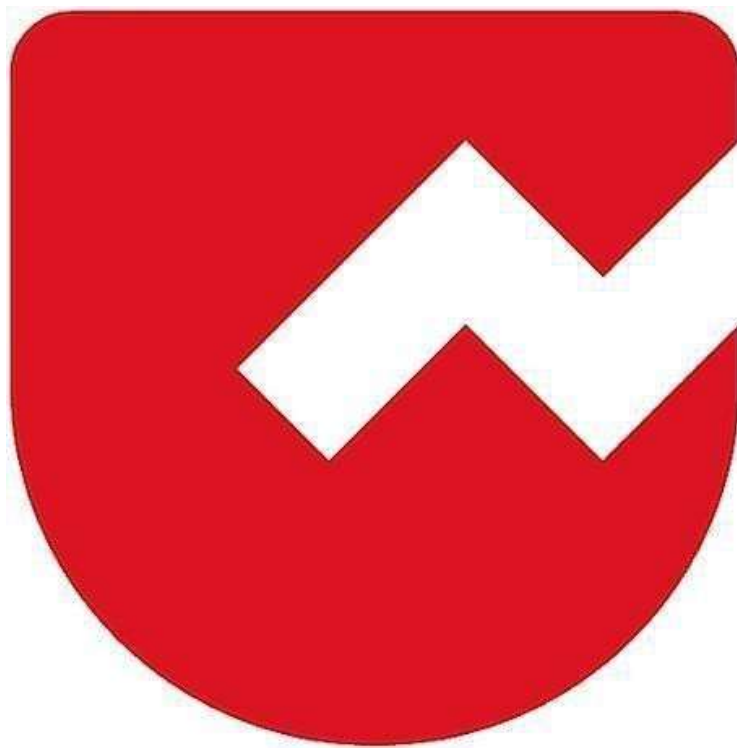
```
class Animal {  
    protected void eat() {  
        System.out.println("Animal eats");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barks");  
    }  
  
    void show() {  
        eat(); // Dog can access protected method from Animal  
        bark();  
    }  
}  
  
public class ProtectedChildDemo {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.show(); // Accessing protected method through child class  
    }  
}
```

protected allows access:

- Within the same package.
- In subclasses (even if in different package).
- But not accessible directly by unrelated classes in another package.



Zoho Schools for Graduate Studies



Notes Day-18

**Date:23/9/2025
Day:Tuesday**

Method Overriding

Overridden method :

The **super class** method that is subjected for **redefinition** is termed as overridden method.

Overridden method :

The **inherited sub-class** method that got **redefined** is termed as overridden method.

When to override a method ?

To capture the **unique child behaviour** that is different from the **parent**, we need to go for overriding.

Exceptions:

In Java, an exception is an event that occurs during the execution of a program that **disrupts the normal flow of instructions**. These exceptions can occur for various reasons, such as invalid user input, file not found, or division by zero. When an exception occurs, it is typically represented by an object of a subclass of the **java.lang.Exception class**.

There are two types of exceptions in java ,

- **Checked Exception**
- **Unchecked Exception**

Checked Exception :

Those exceptions that compiler must check for a **exception handler**, Except **Run-time exception** are all checked Exceptions and those exceptions must handled by the programmer with the Try and catch block , here **catch** block is a **exception handler**.

Some common checked exceptions are,

- **IOException** → File handling errors.
- **SQLException** → Database access errors.
- **ClassNotFoundException** → Class not found when loading dynamically.
- **InterruptedException** → Thread interruption.

Unchecked Exception :

Unchecked exceptions are exceptions that occur at **Run-time** and are not checked by the compiler during **compile-time**, Runtime exception and its subclass are all belongs to unchecked exceptions.

Some common unchecked exceptions are,

- **ArithmeticException** → Divide by zero.
- **NullPointerException** → Using a null object.
- **ArrayIndexOutOfBoundsException** → Invalid array index.
- **NumberFormatException** → Converting invalid string to number.
- **IllegalArgumentException** → Passing invalid argument to method.

Legal rules for Method Overriding :

1. Method overriding is possible if and only if the super class method is inheritable.
2. Overridden methods must share the same name and parameter list.
3. Overridden methods can have the same access level or a broader access level
4. Overridden methods can have the same return type or any sub-type of the super class return type (co-variant return types).
5. Methods that are marked as final or private or static cannot be overridden.
6. Constructors can be overloaded but cannot be overridden.
7. Overridden methods can throw any new unchecked Exception.
8. Overridden methods can eliminate throwing any unchecked Exceptions.
9. Overridden methods can throw any number of unchecked Exceptions.
10. Overridden methods cannot throw any new checked Exceptions.

11. Overridden methods can throw any sub-type of the super class checked Exception type.

12. Overridden methods can eliminate throwing any checked Exceptions.

Examples :

- **Rule 7 : Overridden methods can throw any new unchecked Exception.**

```
public class Animal // Base class or Parent class or Super class
{
    public Number eat() throws NullPointerException // overridden method
    {
        System.out.println("Animal eats");
        return 23;
    }
}

public class Dog extends Animal //Derived class or child class or sub class
{
    @Override // overridden method
    public Byte eat() throws NullPointerException , ArithmeticException
    { // method overriding
        System.out.println("Dog eats meat, milk, biscuit...");
        return 34;
    }
}
```

Explanation :

An overridden method in a **subclass is permitted to throw any new unchecked exceptions, even if the superclass method does not.** This is because unchecked exceptions are not required to be handled at compile time, providing flexibility in how subclasses handle runtime errors.

- **Rule 8 : Overridden methods can eliminate throwing any unchecked Exceptions.**

```
public class Animal // Base class or Parent class or Super class
{
    // overridden method
    public Number eat() throws NullPointerException , ArrayIndexOutOfBoundsException
    {
        System.out.println("Animal eats");
        return 23;
    }
}
```

```
public class Dog extends Animal //Derived class or child class or sub class
{
    @Override // overridden method
    public Byte eat() // method overriding
    {
        System.out.println("Dog eats meat, milk, biscuit...");
        return 34;
    }
}
```

Explanation :

The Dog class is overriding the eat() method from its parent class, Animal. The overridden method in Animal declares that it can **throw two unchecked exceptions**: NullPointerException and ArrayIndexOutOfBoundsException. However, the overridden method in **Dog does not declare any exceptions at all**. This is a perfectly legal and valid action in Java due to the rules of method overriding.

- **Rule 9 : Overridden methods can throw any number of unchecked Exceptions.**

```
public class Animal // Base class or Parent class or Super class
{
    // overridden method
    public Number eat()
    {
        System.out.println("Animal eats");
        return 23;
    }
}
```

```

public class Dog extends Animal //Derived class or child class or sub class
{
    @Override // overridden method
    public Byte eat() throws NullPointerException,
                          ArrayIndexOutOfBoundsException // method overriding
    {
        System.out.println("Dog eats meat, milk, biscuit...");
        return 34;
    }
}

```

Explanation :

unchecked exceptions are not part of the method's compile-time contract, **an overridden method can add as many of them as needed**. The Dog class's eat() method is within its rights to declare that it can throw these exceptions, **even though the parent method doesn't**. This doesn't violate the inheritance contract because clients of the Animal class are not required to handle these runtime exceptions in the first place.

- **Rule 10 : Overridden methods cannot throw any new checked Exceptions.**

```

public class Animal // Base class or Parent class or Super class
{
    // overridden method
    public Number eat()
    {
        System.out.println("Animal eats");
        return 23;
    }
}

public class Dog extends Animal //Derived class or child class or sub class
{
    @Override //overridden method
    public Byte eat() throws java.io.FileNotFoundException
    { // method overriding
        System.out.println("Dog eats meat, milk, biscuit...");
        return 34;
    }
}

```

Explanation :

The Dog class is overriding the eat() method from its parent, Animal. The overridden method in **Animal doesn't throw any exceptions**. However, **the overridden method in Dog attempts to throw a new checked exception**, FileNotFoundException. This will result in a **compile-time error**.

- **Rule 11 : Overridden methods can throw any sub-type of the super class checked Exception type.**

```
public class Animal // Base class or Parent class or Super class
{
    // overridden method
    public Number eat() throws java.io.IOException
    {
        System.out.println("Animal eats");
        return 23;
    }
}

public class Dog extends Animal //Derived class or child class or sub class
{
    @Override // overridden method
    public Byte eat()throws java.io.FileNotFoundException
    { // method overriding
        System.out.println("Dog eats meat, milk, biscuit...");
        return 34;
    }
}
```

Explanation :

The Dog class is overriding the eat() method from its parent, Animal. The overridden method in Animal throws a **checked exception**, IOException. The overridden method in Dog throws a **sub-type** of that exception, FileNotFoundException. This is a legal and valid action in Java.

- **Rule 12 : Overridden methods can eliminate throwing any checked Exceptions.**

```
public class Animal // Base class or Parent class or Super class
{
    // overridden method
    public Number eat() throws java.io.IOException, java.io.EOFException
    {
        System.out.println("Animal eats");
        return 23;
    }
}

public class Dog extends Animal //Derived class or child class or sub class
{
    @Override // overridden method
    public Byte eat() // method overriding
    {
        System.out.println("Dog eats meat, milk, biscuit...");
        return 34;
    }
}
```

Explanation :

The Dog class is overriding the eat() method from Animal. The overridden method in the **parent class Animal throws the checked exception IOException**. However, the overridden method in the **child class Dog does not throw any exceptions at all**.

Thank You 🍊



Zoho Schools for Graduate Studies



Notes Day-19

Constructor Invocation in Java

Super():

The **super()** keyword in Java is used to **call the constructor of the parent (superclass)** from a **child (subclass)**.

It ensures that the parent class is properly initialized before the child class adds its own initializations.

//Multilevel inheritance

ex.1

In Java, an implicit `super()` **is automatically inserted by the compiler** as the **first line of a child class constructor** if and only if the constructor doesn't have any form of `super()` or `this()` call explicitly by programmer.

note:

- implicitly provided **super()** call statement will always invoke the zero argument constructor of the super class.
- **super** keyword can be used by a sub class to access its super class members

// Parent class

```
class Game{
```

```
String type;
```

```
    Game(){
```

```
        System.out.println("Game"); // This prints "Game"
```

```
    }
```

```
}
```

// Child class of Game

```
class IndoorGame extends Game{
```

```
    IndoorGameO{
```

```
        // Implicit super() → calls Game() constructor
```

```
        System.out.println("IndoorGame"); // Prints "IndoorGame"
```

```
    }
```

```
}
```

// Child class of IndoorGame

```
class Chess extends IndoorGame{
```



```

    Chess(){

        super(); //explicitly calls IndoorGame() constructor

        System.out.println("Chess"); // Prints "Chess"

    }

}

public class InheritanceDemo{

    public static void main(String[] args){

        Chess c = new Chess(); // output :

                                Game
                                IndoorGame
                                Chess

    }

}

```

ex.2

Object class: (ultimate super class)

- Object is the **root class of all classes** in Java which is present in **java.lang** package.
- Every class in Java **directly or indirectly inherits** from Object.
- Even if you don't write extends Object explicitly. it is implicitly inherits by subclass.

note:

- in java, Orphan classes are not allowed except Object class
- if a class doesn't inherit any other class, then the class will be made to inherit Object class implicitly.
- in java, every class must inherit one another class except Object class.

// child class of Object class

class Game **extends** Object{// either implicit or explicitly inherited (Object class)

String type;

```

    Game(){

        // Implicit super() → calls Object() constructor

        System.out.println("Game");

    }

}

```

ex.3

this() : this() call is used to invoke another constructor within same class.

note:

- When you use this() in a constructor, it **must be explicitly written** by the programmer Unlike super(), which is **implicitly added** by the compiler if you don't write it, this() is not provided implicitly.
- **Must be the first statement** in the constructor.
- this() and super() call **cannot co-exist** in a constructor
- **this** is a reference to the **current instance** of the class. used to distinguish between instance variables and parameters.

// Parent class

```
class Game{  
    String type;  
    Game(){  
        System.out.println("Game"); // This prints "Game"  
    }  
}
```

// Child class of Game

```
class IndoorGame extends Game{  
    IndoorGame(){  
        // Implicit super() → calls Game() constructor  
        System.out.println("IndoorGame"); // Prints "IndoorGame"  
    }  
}
```

// Child class of IndoorGame

```
class Chess extends IndoorGame{  
    Chess(){  
        this(12); //call Chess() constructor which accepts int as an arguments within same class  
        System.out.println("Chess"); // Prints "Chess"  
    }  
    Chess(int x){  
        // Implicit super() → calls IndoorGame() constructor  
    }  
}
```

```

}

public class InheritanceDemo{

    public static void main(String[] args){

        Chess c = new Chess(); // output :

                                   Game
                                   IndoorGame
                                   Chess

    }

}

```

recursive constructor invocation

ex.4

```

class Chess extends IndoorGame{

    Chess(){

        this(12); //call Chess(int x) constructor which accepts int as an arguments within same class

        System.out.println("Chess")

    }

    Chess(int x){

        th-so: //call Chess() constructor which accepts zero arguments within same class

    }

}

```

```

public class InheritanceDemo{

    public static void main(String[] args){

        Chess c = new Chess(); // output :

        compile-time error - This creates an infinite loop of constructor calls.

    }

}

```

ex.5

// Parent class

```
class Game{  
    String type;  
    Game(){  
        System.out.println("Game"); // This prints "Game"  
    }  
}
```

// Child class of Game

```
class IndoorGame extends Game{  
    IndoorGame(){  
        // this constructor never been called by the sub class constructor  
        System.out.println("IndoorGame");  
    }  
    IndoorGame(int x){  
        super(); //--> calls Game() constructor  
    }  
}
```

// Child class of IndoorGame

```
class Chess extends IndoorGame{  
    Chess(){  
        this(12); //call Chess(int x) constructor within class  
        System.out.println("Chess"); // Prints "Chess"  
    }  
    Chess(int x){  
        super(S); //call IndoorGame(int x) constructor of super class  
    }  
}
```

```
public class InheritanceDemo{  
    public static void main(String[] args){
```

```
Chess c = new Chess(); // output :
```

```
Game
```

```
Chess
```

```
}
```

```
}
```

ex.6

```
class Chess extends IndoorGame{
```

```
    Chess( Strings){
```

```
        this(12); //call Chess(int x) constructor within class
```

```
        System.out.println("Chess");
```

```
    }
```

```
    Chess(int x){
```

```
        super(5); //call IndoorGame(int x) constructor of super class
```

```
    }
```

```
}
```

```
public class InheritanceDemo{
```

```
    public static void main(String[] args){
```

```
        Chess c = new Chess(); // output :
```

```
        compile-time error --> because in Chess class there is no parameter less constructor is created
```

```
    }
```

```
}
```

ex.7

```
// Child class of Game
```

```
class IndoorGame extends Game{
```

```
    IndoorGame( Strings){
```

```
        System.out.println("IndoorGame");
```

```
    }
```

```
}
```

// Child class of IndoorGame

```
class Chess extends IndoorGame{
```

```
    Chess(){
```

// super() __, doesn't call IndoorGame class constructor - because there is no zero argument constructor present in that class.

// want to call it explicitly with super(args) -->because IndoorGame class only has parameterized constructor

```
        System.out.println("Chess");
```

```
    }
```

```
}
```

```
public class InheritanceDemo{
```

```
    public static void main(String[] args){
```

```
        Chess c = new Chess(); // output :
```

compile-time error__, IndoorGame doesn't **have zero-argument constructor**, only IndoorGame{String s}.

```
    }
```

```
}
```

note:

- If a class has **only a parameterized constructor**, Java compiler **won't create a zero argument constructor** in a class.
 - **super()** works **only if the parent has a Zero argument constructor**.
 - To call a **parameterized constructor** in parent class, you must use **super(arguments)** explicitly.
-

Quiz:

what are all implicitly provided in a class?

1. class A{

}

ans: class A extends Object{

AO{

super();

}

}

2. class B{

void BO{

}

}

ans: class B extends Object{

B(){

super();

}

void BO{

}

}

3. class C extends D {

C (int x){

super();

}

}

ans: nothing will be provided implicitly



Zoho Schools for Graduate Studies



Notes **Day-20**

Date : 25/09/2025

Day : Thursday

Multiple Inheritance

1. What is Multiple Inheritance?

Multiple Inheritance means a class can inherit features (fields and methods) from **more than one parent class**.

2. Does Java Support's Multiple Inheritance (with Classes)?

Ans: No, Java designers intentionally **disallowed multiple inheritance of classes** to avoid the “**Diamond Problem**”.

Reasons why java doesn't support multiple inheritance :

1. Deadly Diamond Problem (Ambiguity)

- If two parent classes have the **same method**, and a child class inherits both:
 - The compiler cannot decide which version to use.
 - This creates **ambiguity** and leads to confusion in method resolution.

2. Complexity in Design

- Handling multiple parent classes complicates the **compiler design** and the **object model**.
- Java was designed to be **simple and easy to understand** → avoiding multiple inheritance keeps the language clean.

3. Constructor Invocation Issues

- In multiple inheritance, both parent classes may have constructors that need to be called.
- Deciding the **order of constructor calls** becomes tricky and can lead to inconsistency.

Sample code for Multiple Inheritance

Game.java

```
public class Game extends Object {  
    Game() {  
        System.out.println("Game");  
    }  
    void play() {  
        System.out.println("Game started...");  
    }  
}
```

IndoorGame.java

```
public class IndoorGame extends Game {  
    IndoorGame() {  
        System.out.println("Indoor Game");  
    }  
    void play() {  
        System.out.println("Indoor Game started...");  
    }  
}
```

OutdoorGame.java

```
public class OutdoorGame {  
    void play() {  
        System.out.println("Outdoor Game started...");  
    }  
}
```

Chess.java

// ✗ Trying to extend two classes (not allowed in Java)

```
public class Chess extends IndoorGame, OutdoorGame {  
    Chess() {  
        System.out.println("Chess");  
    }  
    void play() {  
        System.out.println("Chess Game started...");  
    }  
}
```

InheritanceDemo.java

```
public class InheritanceDemo {  
    public static void main(String[] args) {  
        Chess c = new Chess();  
        c.play();  
    }  
}
```

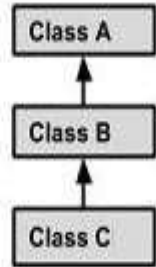
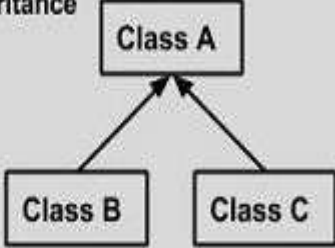
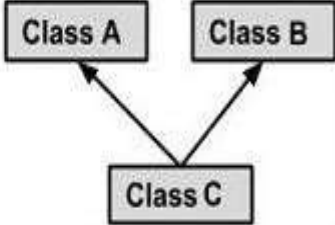
Output :

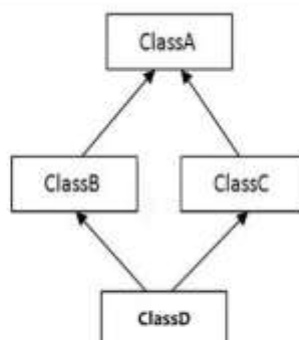
✗ Compile-time Error

When you try to compile Chess.java, you'll get:

```
error: '{' expected  
public class Chess extends IndoorGame, OutdoorGame {  
                                         ^  
error: class, interface, or enum expected  
public class Chess extends IndoorGame, OutdoorGame {
```

➤ Different Forms of Inheritance in Java

Single Inheritance  <pre> graph BT B[Class B] --> A[Class A] </pre>	<pre> public class A { } public class B extends A { } </pre>
Multi Level Inheritance  <pre> graph BT C[Class C] --> B[Class B] B --> A[Class A] </pre>	<pre> public class A { } public class B extends A { } public class C extends B { } </pre>
Hierarchical Inheritance  <pre> graph BT B[Class B] --> A[Class A] C[Class C] --> A </pre>	<pre> public class A { } public class B extends A { } public class C extends A { } </pre>
Multiple Inheritance  <pre> graph BT C[Class C] --> A[Class A] C --> B[Class B] </pre>	<pre> public class A { } public class B { } public class C extends A,B { } // Java does not support mutiple Inheritance </pre>



4) Hybrid Inheritance

➤ Rules for Representing Inheritance Diagrams

1. Use of Boxes for Classes

- Each class should be drawn as a box/rectangle.
- Write the class name clearly inside the box (sometimes with methods/attributes if needed).

2. Direction of Inheritance

- Use a Strong straight line with an arrow to show inheritance.
- The arrow points from the child class (subclass) to the parent class (superclass).

Example :



1. Single Inheritance :

- One subclass should have only one arrow pointing upward to its parent.

2. Multilevel Inheritance :

- The subclass of one class can itself be a parent for another class.
- Represented as a vertical chain of arrows.

3. Hierarchical Inheritance :

- A single parent class can have multiple child classes.
- Draw multiple arrows from the **same parent class** to its subclasses.

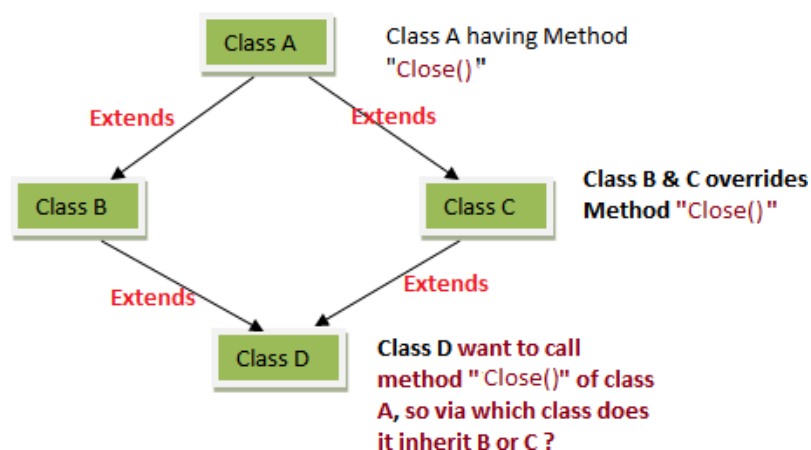
4. Hybrid Inheritance :

- **Definition:** Hybrid inheritance is a **combination of two or more types of inheritance** (e.g., single + multiple, or multilevel + hierarchical).
- It is called “hybrid” because it mixes different inheritance patterns.

5. Multiple Inheritance (Not in Java with classes) :

- A single subclass having arrows pointing to more than one parent class.
- In Java, show this only as a **conceptual diagram** (with a note: *not supported with classes*).
- Java allows it only via **interfaces**.

Deadly Diamond Problem



3. Does Java provide a **direct solution** to the **Deadly Diamond Problem**?

Ans : No, Java **does not support multiple inheritance with classes**.

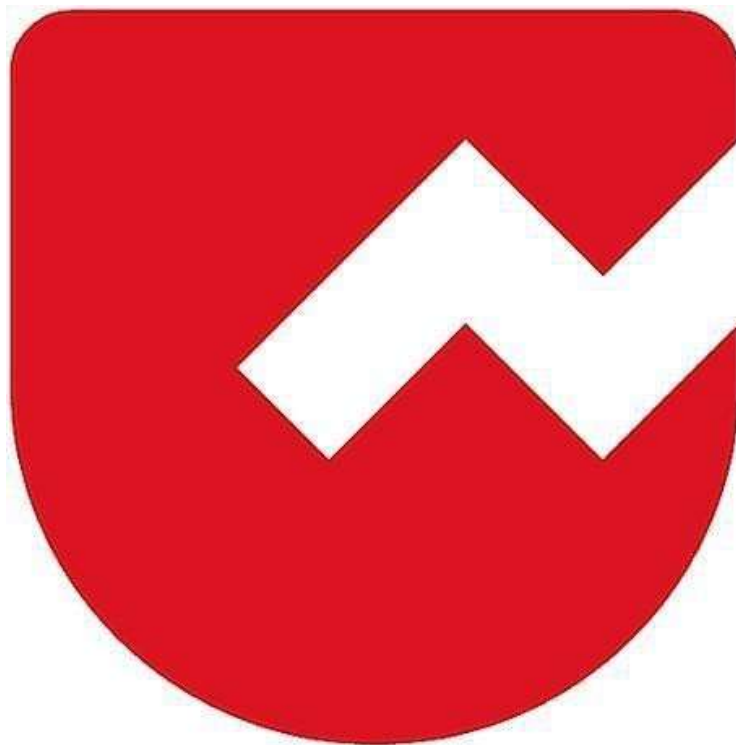
- This means: **the diamond problem cannot occur in Java's class-based inheritance model**.
- Instead of multiple inheritance, java provides alternative solution [\[Interfaces\]](#).

- Multiple inheritance is possible in Java through interfaces.

Today's Work : Design an **inheritance hierarchy** for calculating mileage of **Scooter, Bike, and Electric Scooter** in Java.



Zoho Schools for Graduate Studies



Notes

Day-21

Abstract methods

Abstract Method

- A method without implementation or definition or body is termed as an abstract method.

Concrete Method

- A method with implementation or definition or well-defined body is termed as a concrete method. (or) a well-defined method is a concrete method.

Abstract Class

- A class that cannot be instantiated is an abstract class.

Concrete Class

- A class with only well-defined members is a concrete class.

Note:

- If a class contains at least one abstract member, then the class must be declared as abstract.
- A class can be marked as abstract even if it contains only concrete members.
- Can a class be marked as abstract even if it contains only concrete members?
yes

- There must exist some concrete sub-class to provide implementation for the inherited abstract members.

(or)

- The first concrete sub-class must provide implementation for all the inherited abstract methods.

Abstract Class: Vehicle

```
abstract public class Vehicle
{
    private String name;
    private double distance;
    private double fuelConsumed;

    Vehicle(String name, double distance, double fuelConsumed)
    {
        this.name = name;
        this.distance = distance;
        this.fuelConsumed = fuelConsumed;
    }

    // Getters and setters...

    public String getName()
    {
        return name;
    }

    public void setName(String name)
    {
        this.name = name;
    }
}
```

```
public double getDistance()
{
    return distance;
}

public void setDistance(double distance)
{
    this.distance = distance;
}

public double getFuelConsumed()
{
    return fuelConsumed;
}

public void setFuelConsumed(double fuelConsumed)
{
    this.fuelConsumed = fuelConsumed;
}

abstract public double calculateMileage();
}
```

Explanation :

- Vehicle is an **abstract class** (you cannot create objects of it directly).
- It has **common properties**:
 - name → brand name of vehicle
 - distance → distance travelled (in km)
 - fuelConsumed → how much fuel/Battery it used (in litres/Kilowatts)
- It has an **abstract method** calculateMileage() → each child class must define how to calculate mileage.

Child Class: Bike

```
class Bike extends Vehicle
{
    Bike(String name, double distance, double fuelConsumed)
    {
        super(name, distance, fuelConsumed);
    }

    public double calculateMileage()
    {
        return getDistance() / getFuelConsumed();
    }
}
```

Explanation:

- Bike **extends** Vehicle.
- In the constructor, it calls `super(...)` to set name, distance, and fuelConsumed.
- Implements `calculateMileage()` as **distance ÷ fuelConsumed** (km per litre).

Child Class: ElectricVehicle

```
class ElectricVehicle extends Vehicle
{
    ElectricVehicle (String name, double distance, double fuelConsumed)
    {
        super(name, distance, fuelConsumed);
    }
}
```

```
public double calculateMileage()
{
    return getDistance() / getFuelConsumed();
}
}
```

Explanation:

- ElectricVehicle extends Vehicle
- Right now, it does the same calculation as Bike.
- Idea: In future, if electric vehicles calculate mileage differently (say, distance per kWh), we could override logic here.

Child Class: ElectricVehicle

```
class Scooter extends Vehicle
{
    Scooter(String name, double distance, double fuelConsumed)
    {
        super(name, distance, fuelConsumed);
    }

    public double calculateMileage()
    {
        return getDistance() / getFuelConsumed();
    }
}
```

Explanation:

- Scooter extends Vehicle.
- Same calculation for now, but written separately for clarity.
- Shows how **multi-level inheritance** works.

Main Class

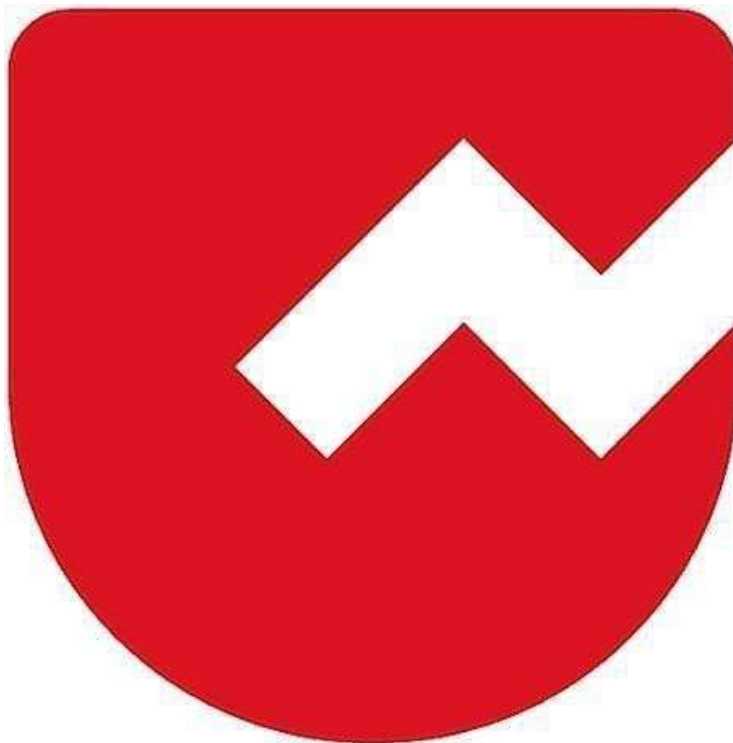
```
public class Main
{
    public static void main(String[] args)
    {
        Bike vobj1 = new Bike("Ducati", 40, 3.5);
        double bikeVal = vobj1.calculateMileage();

        Scooter vobj3 = new Scooter("Honda", 35, 8.3);
        double scootVal = vobj3.calculateMileage();

        System.out.println("Brand Name : " + vobj1.getName() + " Mileage : "
+ bikeVal + " km/l");
        System.out.println("Brand Name : " + vobj3.getName() + " Mileage : "
+ scootVal + " km/l");
    }
}
```



Zoho Schools for Graduate Studies



Notes Day-22

Date : 30/09/2025

Day : Tuesday

QUIZ - Methods

Question 1: Which of the following statement(s) describe the primary uses of methods in Java?

Options:

- A) It allows code re-usability (define once and use multiple times).
- B) You can break a complex program into smaller chunks of code.
- C) It increases code readability and debugging easier.
- D) All the above.

Correct Answer: D) All the above

Explanation:

Methods in Java improve **code reusability**, help **modularize complex programs**, and enhance **readability and debugging**, so all statements are correct.

Question 2: Which of the following statement is false?

Options:

- A) Functions are defined independently of any object.
- B) Methods are dependent and tied to an instance of a class (object) or the class itself.
- C) Functions are called directly by name, whereas methods are called on an object or class.
- D) None of the above.

Correct Answer: D) None of the above

Explanation:

All the given statements (A, B, C) are **true**, hence the false option is “**None of the above.**”

Question 3: What is the output of the following code?

```
public class First {  
    public static void main(String[] args) {  
        add(3, 2);  
    }  
    public int add(int a, int b) {
```



```
        return a + b;
    }
}
```

Options:

- A) 5
- B) No Output
- C) Compiler Error
- D) Runtime Error

Correct Answer: C) Compiler Error

Explanation:

The add() method is **non-static**, but it is being called directly from the static main() method without creating an object. This leads to a **compile-time error**.

Question 4: What is the output of the following code?

```
public class First {
    public static void main(String[] args) {
        sayHello("Hello");
    }
    public static void sayHello(String s) {
        System.out.println(s);
    }
}
```

Options:

- A) Hello
- B) No Output
- C) Compiler Error
- D) Runtime Error

Correct Answer: A) Hello

Explanation:

The method sayHello() is **static**, so it can be called directly from the static main() method. It correctly prints "Hello" to the console.

Question 5: What is the output of the following code?

```

public class First {
    public static void main(String[] args) {
        new First().cube(3);
    }
    public static int cube(int n) {
        return n * n * n;
    }
}

```

Options:

- A) 27
- B) No Output
- C) Compiler Error
- D) Runtime Error

Correct Answer: B) No Output

Explanation:

The method cube(3) is executed, and the value **27** is computed, but it is **not printed** or stored. Since there is no System.out.println() statement, nothing is displayed on the console.

Question 6: What is the output of the following code?

```

public class First {
    public static void main(String[] args) {
        new First().fo();
        System.out.print("1");
    }
    public void fo() {
        System.out.print("2");
        go();
    }
    public void go() {
        ho();
        System.out.print("3");
    }
    public void ho() {
        System.out.print("4");
    }
}

```

Options:

- A) 1234

- B) 2431
- C) 2341
- D) 2413

Correct Answer: B) 2431

Explanation:

fo() prints 2, then calls go().go() calls ho(), which prints 4. After returning from ho(), go() prints 3. Finally, control goes back to main(), which prints 1. So the output is 2431.

Question 7: Which of the following is true about the return statement in Java?

Options:

- A) It can only return integers.
- B) It can return multiple values.
- C) It must always be the last statement in a method.
- D) It can be used to return from a method at any point.

Correct Answer: D) It can be used to return from a method at any point.

Explanation:

In Java, the return statement is used to exit a method and optionally return a value. It can be placed anywhere in a method, not necessarily at the end. Methods can return only a single value (or an object/array containing multiple values), so options A, B, and C are incorrect.

Question 8: What is the output of the following code?

```
public class First {  
    int x = 2;  
    public static void main(String[] args) {  
        int x = 3;  
        int y = new First().go(x);  
        System.out.print(x + y);  
    }  
    public int go(int y) {  
        x = x + 2;  
        return x + y;  
    }  
}
```

Options:

- A) 5
- B) 10
- C) Compiler Error
- D) Runtime Error

Correct Answer: B) 10

Explanation:

- First has an instance variable $x = 2$.
- In main, local $x = 3$ (different from instance x).
- `go(x)` is called with $y = 3$. Inside `go`, $x = 2 + 2 = 4$.
- The method returns $x + y = 4 + 3 = 7$.
- Back in main, $x + y = 3 + 7 = 10$.

Hence, the output is **10**.

Question 9: What is the output of the following code?

```
public class First {  
    public static void main(String[] args) {  
        if (args.length != 0) {  
            System.out.println(args.length);  
        } else {  
            new First().go();  
        }  
    }  
    public static void go() {  
        ho();  
    }  
    public static void ho() {  
        main(new String[] { "one", "two", "Three Four", "Five" });  
    }  
}
```

Options:

- A) 0
- B) 4
- C) 5
- D) Infinite loop

Correct Answer: B) 4

Explanation:

- The program starts with main and an empty args array.
- Since `args.length == 0`, it calls `go()`.
- `go()` calls `ho()`, which calls main with a 4-element array: `{"one", "two", "Three Four", " Five"}`.
- Now `args.length != 0`, so it prints 4 and ends.
- No further recursion occurs.

Question 10: What is the output of the following code?

```
public class First {
    public static void main(String[] args) {
        System.out.println(go(go(go(3))));
    }
    public static int go(int n) {
        return 2 * n;
    }
}
```

Options:

- A) 6
- B) 12
- C) 24
- D) 48

Correct Answer: C) 24

Explanation:

- `go(3)` returns $2 * 3 = 6$.
- `go(go(3)) = go(6) = 2 * 6 = 12`.
- `go(go(go(3))) = go(12) = 2 * 12 = 24`.
- Hence, the output is **24**.

Question 11: Which of the following statement(s) is False?

Options:

- A) A parameter is a variable in the declaration of a function.
- B) An argument is the actual value that is passed to the function when it is called.
- C) Parameters act as placeholders for the actual values (arguments) that will be passed to the function when it is called.
- D) The number of arguments provided when calling a function must match the number of parameters defined in the function declaration.

E) Parameters and arguments mean the same.

Correct Answer: E) Parameters and arguments mean the same.

Explanation:

- A **parameter** is a variable in a function declaration, while an **argument** is the actual value passed when calling the function.
- They are related but **not the same**, so the last statement is false.

QUIZ - Iterative Statements

Question 1: How many times will the following loop execute?

```
int n = 1;
do {
    n *= 2;
} while (n < 100);
```

Options:

- A) 6
- B) 7
- C) 8
- D) 9

Correct Answer: B) 7

Explanation:

- n starts at 1 and doubles each time: $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 32 \rightarrow 64 \rightarrow 128$.
- The loop stops when $n < 100$ is false.
- The loop executes **7 times** (last value 64, then doubles to 128 and exits).

Question 2: What is the output of the following code snippet upon execution?

```
int x = 10;
while (x-- > 5) {
```

```
x -= 2;
}
System.out.println(x);
```

Options:

- A) 2
- B) 3
- C) 4
- D) 5

Correct Answer: B) 3

Explanation:

Step-by-step execution:

1. $x = 10$, condition $x-- > 5 \rightarrow 10 > 5$ true, then **x becomes 9** after post-decrement, inside loop $x -= 2 \rightarrow x = 7$.
2. $x = 7$, condition $x-- > 5 \rightarrow 7 > 5$ true, then **x becomes 6**, inside loop $x -= 2 \rightarrow x = 4$.
3. $x = 4$, condition $x-- > 5 \rightarrow 4 > 5$ false, loop exits.
 - The final value of x is **3** (because post-decrement reduces x by 1 after last condition check).

Question 3: What is the output of the following code snippet upon execution?

```
int count = 0;
for (int i = 0; i < 5; i++) {
    for (int j = i; j < 5; j++) {
        count++;
    }
}
System.out.println(count);
```

Options:

- A) 20
- B) 25
- C) 15
- D) None of the above

Correct Answer: C) 15

Explanation:

- Outer loop runs **i = 0 to 4** $\rightarrow$ 5 iterations.
- Inner loop runs from $j = i$ to 4 $\rightarrow$ number of iterations decreases as i increases:

- $i=0 \rightarrow 5$ iterations
- $i=1 \rightarrow 4$ iterations
- $i=2 \rightarrow 3$ iterations
- $i=3 \rightarrow 2$ iterations
- $i=4 \rightarrow 1$ iteration
- Total increments of count = $5 + 4 + 3 + 2 + 1 = 15$.

Question 4: How many times will the following loop execute?_____

```
int count=0;
for (int i = 0; i < 10; i++) {
    for (int j = 10; j > i; j--) {
        // Some code
    }
}
```

Options:

- A) 40
- B) 55
- C) 75
- D) None of the above

Correct Answer: B) 55

Explanation:

- Outer loop: $i = 0$ to $9 \rightarrow 10$ iterations.
- Inner loop: $j = 10$ down to $i+1 \rightarrow$ decreases with each i .
- Number of inner loop executions:
 - $i=0 \rightarrow 10$
 - $i=1 \rightarrow 9$
 - $i=2 \rightarrow 8$
 - ...
 - $i=9 \rightarrow 1$
- Total executions = $10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1 = 55$

Question 5: What is the output of the following code snippet upon execution?

```
int sum = 0;
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 2; j++) {
        if (i == j) {
            continue;
        }
    }
}
```



```

    }
    sum += i + j;
}
}
System.out.println(sum);

```

Options:

- A) 7
- B) 10
- C) 11
- D) None of the above

Correct Answer: A) 7

Explanation:

Step-by-step execution:

- i=0:
 - j=0 → i==j → continue → skipped
 - j=1 → sum += 0+1 = 1 → sum = 1
- i=1:
 - j=0 → sum += 1+0 = 1 → sum = 2
 - j=1 → i==j → continue → skipped
- i=2:
 - j=0 → sum += 2+0 = 2 → sum = 4
 - j=1 → sum += 2+1 = 3 → sum = 7

Question 6: What is the output of the following code snippet upon execution?

```

int i = 0;
for (System.out.print("A"); i < 2; System.out.print("B"), i++)
    System.out.print("C");

```

Options:

- A) Compiler Error
- B) ABCBC
- C) ACBCB
- D) None of the above

Correct Answer: C) ACBCB

Explanation:

Step-by-step execution:

1. **Initialization:** `System.out.print("A")` → prints A
2. **Condition check:** $i < 2 \rightarrow 0 < 2 \rightarrow \text{true}$
3. **Body:** `System.out.print("C")` → prints C
4. **Update:** `System.out.print("B"), i++` → prints B, then $i = 1$
5. **Condition check:** $i < 2 \rightarrow 1 < 2 \rightarrow \text{true}$
6. **Body:** `System.out.print("C")` → prints C
7. **Update:** `System.out.print("B"), i++` → prints B, then $i = 2$
8. **Condition check:** $i < 2 \rightarrow \text{false} \rightarrow \text{loop ends}$

Printed sequence: **ACBCB**

Question 7: What is the output of the following code snippet upon execution?

```
int sum = 0;
for (int i = 1; i <= 10; i++) {
    for (int j = 1; j <= i; j++) {
        sum += j;
    }
}
System.out.println(sum);
```

Options:

- A) 100
- B) 220
- C) 330
- D) None of the above

Correct Answer: B) 220

Explanation:

- Outer loop: $i = 1$ to 10
- Inner loop: $j = 1$ to $i \rightarrow$ sum of first i natural numbers is added in each iteration
- Total sum = $1 + 3 + 6 + 10 + 15 + 21 + 28 + 36 + 45 + 55 = 220$

Question 8: What is the output of the following code snippet upon execution?

```
int i = 0, j = 0, sum = 0;
for (; i < 5 && j < 3; i++, j++) {
    sum += i + j;
}
System.out.print(sum);
```

Options:

- A) 6
- B) 7
- C) 8
- D) None of the above

Correct Answer: A) 6

Explanation:

Loop trace:

- **Iteration 1:** $i=0, j=0 \rightarrow \text{sum} += 0+0 = 0$
- **Iteration 2:** $i=1, j=1 \rightarrow \text{sum} += 1+1 = 2$ (sum = 2)
- **Iteration 3:** $i=2, j=2 \rightarrow \text{sum} += 2+2 = 4$ (sum = 6)
- Next, $j=3$, condition $j < 3$ fails $\rightarrow$ loop ends.

Question 9: What is the output of the following code snippet upon execution?

```
int result = 0;

for (int i = 1; i <= 50; i++) {
    if (i % 2 == 0) continue;
    result += i;
}
System.out.println(result);
```

Options:

- A) 535
- B) 625
- C) 835
- D) None of the above

Correct Answer: B) 625

Explanation:

- The continue skips even numbers.
- So only odd numbers from 1 to 49 are added.
- Sum of first n odd numbers = n^2
- Here, number of odd terms = 25 (1, 3, 5, ..., 49).
- Sum = $25^2 = 625$.

Question 10: What is the output of the following code snippet upon execution?

```
int i = 0;
while (i < 3) {
    System.out.print(i);
    if (i == 2) break;
    i++;
}
```

Options:

- A) 012
- B) 0123
- C) 0122
- D) None of the above

Correct Answer: A) 012

Explanation:

- Start: i = 0 → prints 0, condition i == 2 false → i = 1.
- Next: i = 1 → prints 1, condition false → i = 2.
- Next: i = 2 → prints 2, condition true → break (loop ends).

Question 11: How many times will the innermost loop execute in the following code?

```
int count = 0;

for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 2; j++) {
        for (int k = 0; k < 2; k++) {
            count++;
        }
    }
}
```

Options:

- A) 12
- B) 24
- C) 10
- D) None of the above

Correct Answer: B) 12

Explanation:

- Outer loop (i) → runs 3 times.
- Middle loop (j) → runs 2 times for each i.
- Inner loop (k) → runs 2 times for each (i, j) pair.
- Total executions = $3 \times 2 \times 2 = 12$.

Question 12: What is the output of the following code snippet upon execution?

```
public class JumpStmt {
    public static void main(String []args) {
        int sum = 0;
        int j = 2;

        for (int i = 1; i <= 10; i++) {
            if (i % j == 0) {
                if (i % 5 == 0)
                    break;
                sum += i;
                System.out.print(i + " ");
            } else {
                continue;
            }
        }

        System.out.println("Sum=" + sum);
    }
}
```

Options:

- A) 30
- B) 20
- C) 10
- D) 0

Correct Answer: B) 20

Explanation:

- $j = 2$, so we only consider even numbers.
- Loop values:
 - $i=1 \rightarrow$ odd $\rightarrow$ skipped
 - $i=2 \rightarrow$ even, not multiple of 5 $\rightarrow$ $\text{sum} = 2$
 - $i=3 \rightarrow$ odd $\rightarrow$ skipped
 - $i=4 \rightarrow$ even, not multiple of 5 $\rightarrow$ $\text{sum} = 2+4=6$
 - $i=5 \rightarrow$ odd $\rightarrow$ skipped

- $i=6 \rightarrow$ even, not multiple of 5 $\rightarrow$ $\text{sum} = 6+6=12$
- $i=7 \rightarrow$ odd $\rightarrow$ skipped
- $i=8 \rightarrow$ even, not multiple of 5 $\rightarrow$ $\text{sum} = 12+8=20$
- $i=9 \rightarrow$ odd $\rightarrow$ skipped
- $i=10 \rightarrow$ even, divisible by 5 $\rightarrow$ **breaks loop**
- Final sum = **20**

Question 13: What is the output of the following code snippet upon execution?

```
public class JumpStmt {
    public static void main(String []args) {
        int i = 10;

        outer:
        while (true) {
            if (i == 0) {
                i--;
                break outer;
            }
            System.out.print(i + " ");
        }
    }
}
```

Options:

- A) Runtime Error
- B) Compiler Error
- C) Infinite Loop
- D) None of the above

Correct Answer: C) Infinite Loop

Explanation:

- i is initialized as 10.
- Inside the `while(true)` loop:
 - Condition `if (i == 0)` is never true because i is never decremented.
 - Hence, the `break outer;` is **never reached**.
- Loop keeps printing 10 10 10 ... forever.

Question 14: What is the output of the following code snippet upon execution?

```
int sum = 0;
for (int i = 0; i < 5; i++) {
```

```

        for (int j = 0; j < 5; j++) {
            if (i == j)
                break;
            else
                sum += i + j;
        }
    }

    System.out.println(sum);
}

```

Options:

- A) 25
- B) 40
- C) Infinite Loop
- D) None of the above

Correct Answer: B) 40

Explanation:

Let's trace the loops:

- **i = 0** → inner loop: j=0, condition i==j → break immediately → no addition.
- **i = 1** → inner loop: j=0 → sum += 1+0 = 1 → next j=1 → break → sum = 1.
- **i = 2** → inner loop: j=0 → sum=1+(2+0)=3, j=1 → sum=3+(2+1)=6, j=2 → break → sum=6.
- **i = 3** → inner loop: j=0 → sum=6+3=9, j=1 → sum=9+4=13, j=2 → sum=13+5=18, j=3 → break → sum=18.
- **i = 4** → inner loop: j=0 → sum=18+4=22, j=1 → sum=22+5=27, j=2 → sum=27+6=33, j=3 → sum=33+7=40, j=4 → break → sum=40.

Question 15: What is the output of the following code snippet upon execution?

```

public class JumpStmt {
    public static void main(String[] args) {
        for (int i = 1; i < 5; i++) {
            if (i % (i - 1) == 1)
                break;
            System.out.println(i);
        }
    }
}

```

Options:

- A) Compiler Error
- B) Runtime Error
- C) Infinite Loop
- D) None of the above

Correct Answer: [B\) Runtime Error](#)

Explanation:

- When $i = 1$, the condition $i \% (i - 1)$ becomes $1 \% 0$, which **divides by zero**.
- In Java, dividing by zero during runtime throws an **ArithmeticException**.
- Hence, the program terminates with a **Runtime Error**.



Zoho Schools for Graduate Studies



Notes

Day-23

Interface

- An Interface is defined using the keyword interface.
- Interface Name begins with uppercase.

Implements

- It will inherit all the members in the interface.
- It has to provide implementation for abstract method defined in the interface.

Prior to Java 8 :

- An interface is an 100% abstract class, .i.e. it contains only abstract Methods.
- It can have constants.

From java 8:

- It can contain abstract methods.
- It can contain only those concrete method that are marked as default, static, private.
- It can have constants.

In Java, interfaces can have data members, but they are implicitly public, static, and final. This means they are effectively constants.

Here's what that implies:

Public: These data members are accessible from anywhere.

Static: They belong to the interface itself, not to any specific instance of a class implementing the interface. You access them using the interface name (e.g. `MyInterface.MY_CONSTANT`).

Final: Their values cannot be changed after initialization. They must be assigned a value when declared.

Note:

- By default, method declarations inside an interface are abstract & public.
- By default variables declared inside an interface are public, static & final.
- An interface can inherit (extend) multiple interfaces.
- An interface cannot implement another interface
- A class can extend only one another class except object class.
- A class can implement multiple interfaces.
- A class cannot extend another interfaces.
- A class cannot implement another class.

1) Who's responsibility to provide implementation for abstract method?

Ans: Subclass

2) Why error occurred ?

```

1 package part2;
2
3 public interface Display {
4     final int x=5;
5     void studentDetails(); // abstract method
6 }
7

```

```

package part2;

```

```

public interface Display {
    final int x;
    void studentDetails(); // abstract method
}

```

Ans: Being a constant we should initialize it at the place of declaration.

3) why error occurred ?

Ans:

1. By default, method declarations inside an interface are abstract & public.

```

1 package part2;
2
3 public class DisplayDemo implements Display{
4     void studentDetails() {
5         |
6     }
7     public static void main(String[] args) {
8         // TODO Auto-generated method stub
9
10    }
11
12 }
13

```

2. Rule of inheritance , overloaded methods can share same access level or can have a broader access level.(In interface the jvm will provide public as access level for abstract method).

4) A class implements interface but unwilling to provide implementation for those abstract methods then what can do?

Ans: To declare the class under abstract .

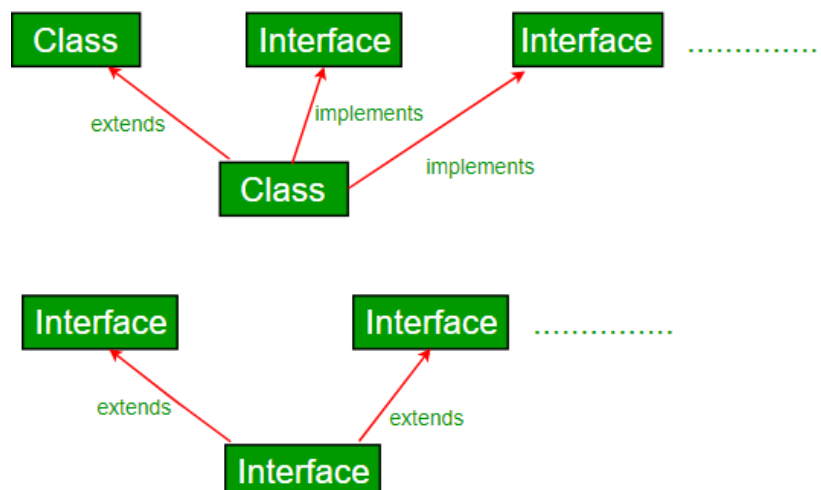
```

1 package part2;
2
3 abstract public class DisplayDemo implements Display{
4     /*public void studentDetails() {
5         System.out.println("Student details...");
6     }*/
7     public static void main(String[] args) {
8         DisplayDemo d=new DisplayDemo();
9         d.studentDetails();
10    }
11 }
12 }

```

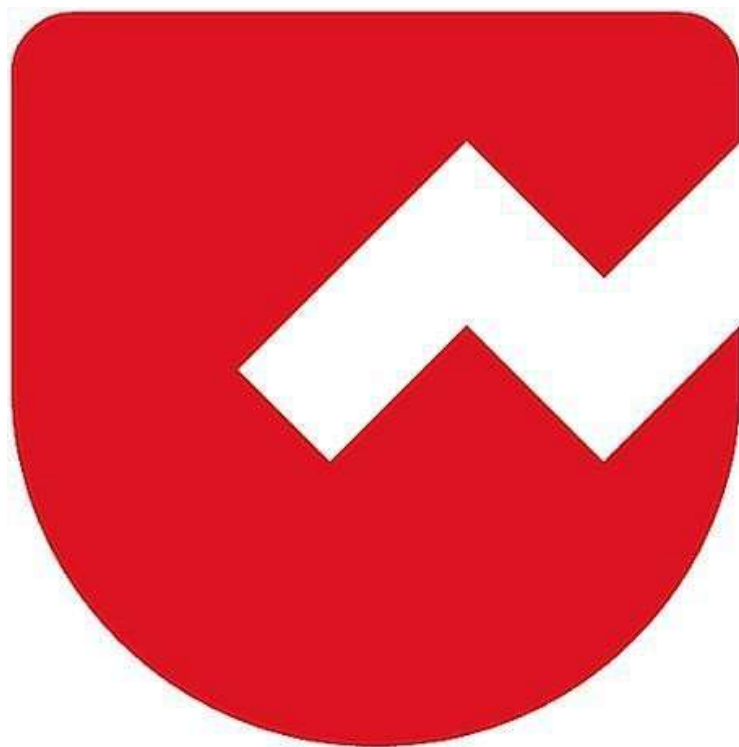
5) Attributes that declare inside interface when they get memory allocation and Why those members are static?

Ans: At the time of class Loading the memory will allocate to the attributes.





Zoho Schools for Graduate Studies



Notes

Day-24

Interface

Define Interface?

- An interface defines a set of abstract methods (methods without a body) which a class must implement, if it chooses to implement the interface.
- It can contain only those concrete methods that are marked as default or static or private. Otherwise, an interface cannot have a normal concrete method.
- It can have constants.

// InterfaceDemo

```
public interface Payment {  
    void pay();  
    public abstract void checkBalance();  
}  
  
abstract public class UPI implements Payment {  
    public void checkBalance() {  
        System.out.println("Checking the Balance");  
    }  
}
```

```
public class GooglePay extends UPI {  
    public void pay() {  
        System.out.println("Payment is made through GPay!");  
    }  
}  
  
public class InterfaceDemo {  
    public static void main(String[] args) {  
        Payment g = new GooglePay();  
        g.checkBalance();  
        g.pay();  
    }  
}
```

Notes:

1. Interfaces are implicitly abstract. Hence they cannot be instantiated. Explicit declaration of the abstract modifier is optional.
2. Interfaces are assumed to have either public or default access level. Otherwise, they cannot be marked as private, protected, or final.
3. Constants declared inside an interface must be initialized at the place of declaration.

4. Abstract methods inside an interface cannot be marked as static or final or protected or private.
5. The first concrete subclass should ensure that the implementations for all the Inherited abstract methods are available.
6. Interfaces can be implemented by any class from any inheritance tree.
7. Method declarations inside an interface are implicitly abstract and public.

Explicit declaration of these modifiers are optional.
8. Constants declared inside an interface are implicitly public, static, and final.

Explicit declarations of these modifiers are optional in any combination.
9. An interface can inherited (extend) multiple interfaces.
10. An interface cannot implement another interface or class.
11. An interface cannot extend another class.
12. A class can extend only one another class except Object class. i.e. a class cannot inherit multiple classes.

13. A class can implement multiple interfaces.
14. A class cannot extend another interface.
15. A class cannot implement another class.
16. If a class extends another class and also implements an interface, then the keyword 'extends' must appear prior to the keyword 'implements'.
17. A class implementing an interface can itself be abstract.
18. An abstract implementing class need not have to implement all the interface methods.
19. A legal (concrete subclass) non-abstract implementing class should ensure that the implementations for all the inherited abstract methods are available.
20. The legal non-abstract implementing class must follow all the legal override rules for the methods it implements.
21. Default methods can only be defined inside an interface and cannot be defined inside a class.
22. Default method is a way to provide default implementation for any abstract methods by an interface.

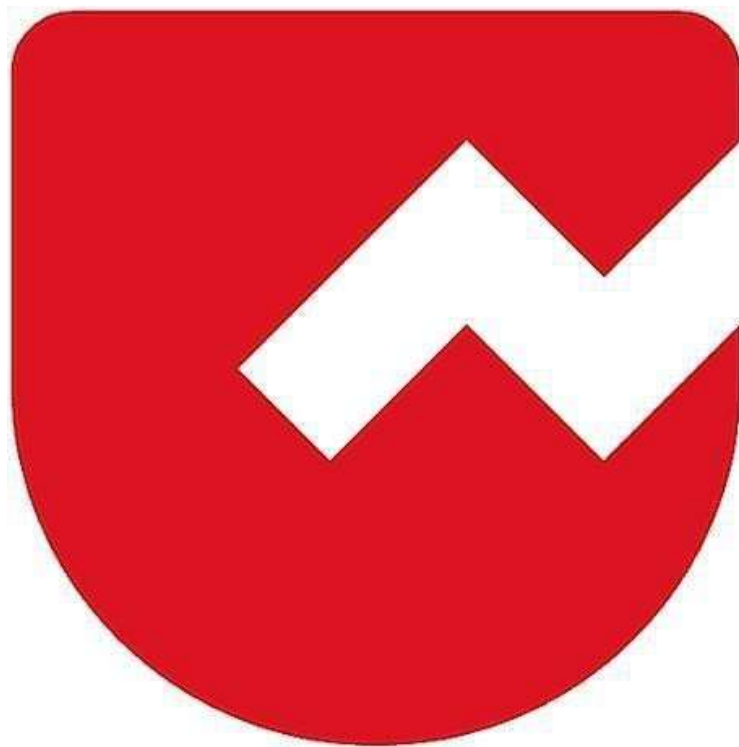
23. Static methods defined in an interface are not inherited by any class that implements the interface.

24. Interface reference names can be used polymorphically, i.e., an object type of the class that implements the interface can be assigned to the interface reference name

What is a marker interface?

- An empty interface is called as a marker interface.
- Example: Cloneable, Serializable

Zoho Schools for Graduate Studies



Notes

Day-25

PACKAGES

Define Packages?

Packages are **collection of related classes and interfaces**.

How to declare a package?

Packages are declared using the keyword **package** followed by package name.

Syntax :

```
package <packageName>;
```

Where in a program, a package statement can appear?

If present, package names must appear as the **first statement in a source file**.

Why packages? Or What are the advantages of package?

- Ease of access and maintenance.
- Better name space management.
(No need to have unique names across packages)
- To obtain private package access control or package access control or default access control.

What is a fully qualified class name?

Qualifying the **class names with their respective package name** is referred as fully qualified class name.

Example :

```
public class Test {  
    public static void main(String[] args) {  
        double result = java.lang.Math.min(25,8);  
        System.out.println(result); // Output : 8  
    } //main()  
} //class
```

What are root package in java?

Java and jdk.

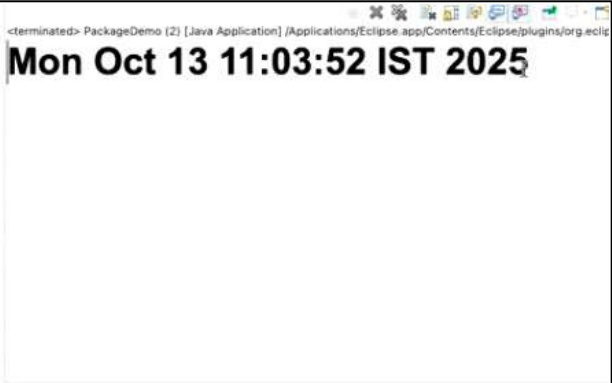
Which package get import by default?

The only package the gets imported by default is **java.lang**.

Can all java program be written without import statement?

Yes. One should use fully qualified class names instead of the import statements.

```
package part2;  
  
import java.util.Date;  
  
public class PackageDemo {  
    public static void main(String[] args) {  
        System.out.println(new Date());  
    }  
}
```



```
package part2;

//import java.util.Date;

public class PackageDemo {
    public static void main(String[] args) {

        System.out.println(new java.util.Date());

    }
}
```

<terminated> PackageDemo (2) [Java Application] /Applications/Eclipse.app/Contents/Eclipse/plugins/org.eclig
Mon Oct 13 11:05:23 IST 2025

What is the equivalent of import statement in ‘c’ language?

Macro => #define pi 3.14;

What is the significance of import statement?

- Import statement are used to **specify the location of the java library.**
- It replaces class names with their fully qualified class name.

What is the advantages of import statement?

It provides typing convenience to a programmer. i.e. programmer is free from writing the fully qualified class names whenever they use the corresponding import statement.

Where in a program, a import statement can appear?

Import statements, must appear between package declaration statements and class declaration.

What is the usage of static import?

It is used to import all the static members of a class.

//Without static import

```
import java.lang.Math;

public class Test {

    public static void main(String[] args) {

        System.out.println( Math.sqrt(25); // Output : 5.0

    }

}
```

//With static import

```
import static java.lang.Math.sqrt;

public class Test {

    public static void main(String[] args) {

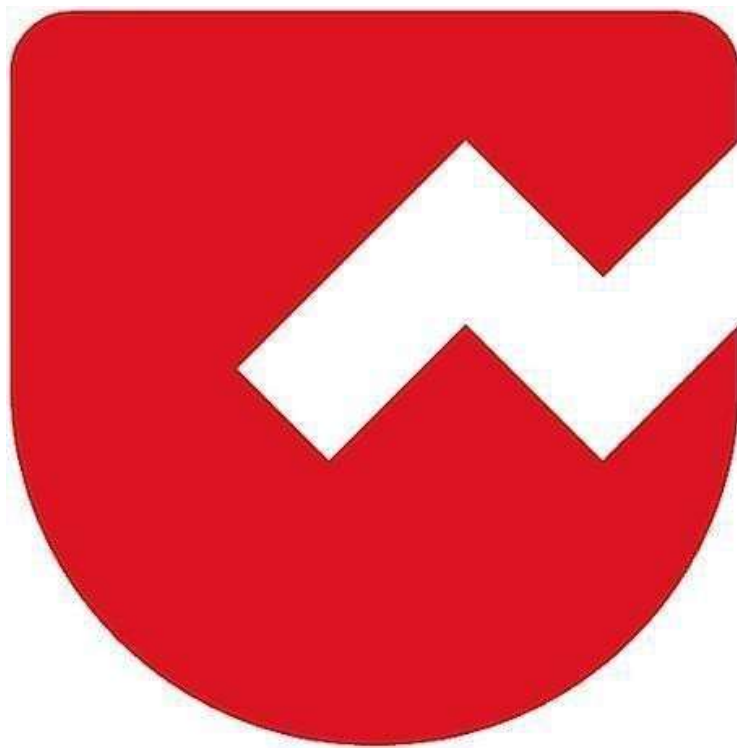
        System.out.println(sqrt(25); // Output : 5.0

    }

}
```




Zoho Schools for Graduate Studies



Notes Day-26

**Date: 23/10/2025
Day: Thursday**

EXCEPTION HANDLING

1. What is an Exception?

- An exception is an unexpected event or error that disrupts the normal flow of instructions in a program.
- It can usually be anticipated and handled by the programmer using specific code structures (like try-catch blocks).
- Common examples are trying to divide by zero or accessing a non-existent file.
- Handling exceptions makes programs more robust and user-friendly, preventing crashes.

2. What is an Error?

- An error is a serious problem in a program, often caused by the environment or system.
- Errors usually cannot be handled or recovered from by the code itself.
- Examples include running out of memory, stack overflow, or hardware failure.
- When an error occurs, the program often must terminate.

3. What is the effect of an Exception?

- When an exception occurs, it interrupts the normal sequence of execution in a program.
- If not caught, the program may abruptly terminate.
- If handled, control can be transferred to code that manages the issue, allowing the program to recover or exit cleanly.
- Unhandled exceptions often generate error messages or logs.

4. What is the impact of Exception?

- Improperly handled exceptions can cause a program to crash, lose data, or behave unexpectedly.
- This can result in poor user experience or loss of trust in software.
- Important processes may be left incomplete, causing system instability.
- Financial/business losses can occur if exceptions shut down critical software.

5. Who gets most affected because of an Exception?

- The primary party affected is the organization that depends on the software.
- Users may experience inconvenience, lost work, or failed transactions.
- The organization may suffer reputational harm, lost revenue, or increased support costs.
- Developers and IT support may also face extra workload fixing and responding to issues.

Exception Example:

```
public class Main {  
    public static void main(String[ ] args) {  
        int[] myNumbers = {1, 2, 3};  
        System.out.println(myNumbers[10]); // error!  
    }  
}
```

The Output:

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 10  
    at Main.main(Main.java:4)
```

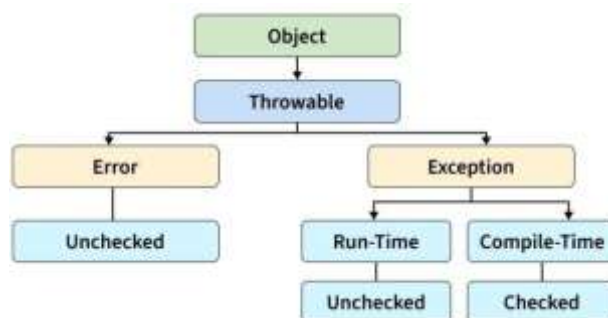
Note: `ArrayIndexOutOfBoundsException` occurs when you try to access an index number that does not exist.

Java Exception Hierarchy:

Java Exception class is the parent class of all exceptions in Java. If you catch an Exception object, you can be sure that you catch all the exceptions. The Java Exception class has many subclasses that represent the different types of exceptions.

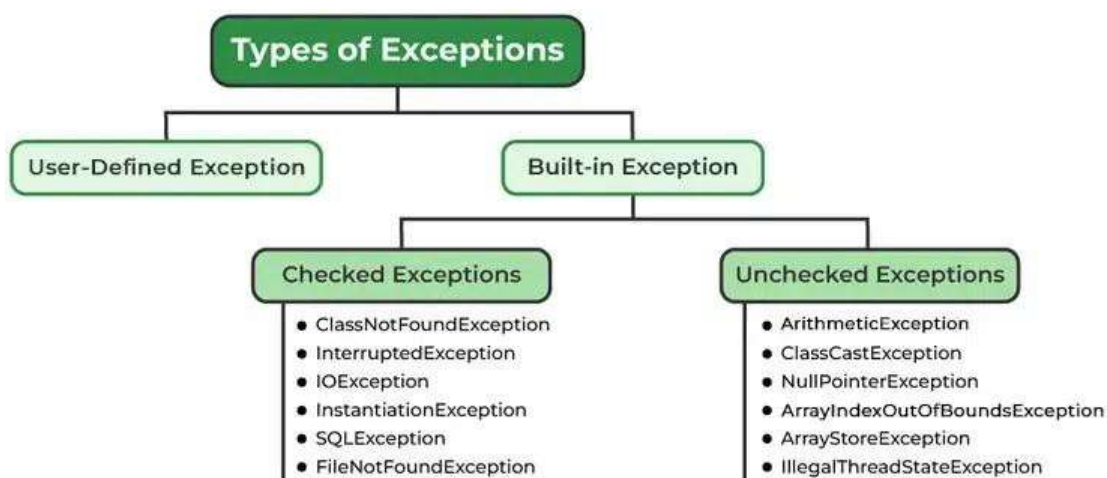
Here is a hierarchy of Java Exception classes:

- **Throwable:** Throwable is the parent class of Java Exceptions hierarchy and it has two child objects, Error and Exception. The Throwable class has two subclasses:
 - **Error:** The errors are exceptional scenarios that are out of the scope of application and it's not possible to anticipate and recover from them, for example, hardware failure, JVM crash, etc.
 - **Exception:** The exceptions are the exceptional scenarios that can be caught and recovered from, for example, FileNotFoundException, SQLException, etc.



Types of Java Exceptions

Java defines several types of exceptions that relate to its various class libraries. Java also allows users to define their exceptions.



What is Exception Handling?

Exception handling is a programming technique used to manage and respond to unexpected events (exceptions) that could disrupt the normal flow of a program. Here's what it means:

- **Detecting errors:** Exception handling involves identifying situations where things go wrong in a program, like trying to access a missing file or dividing by zero.
- **Responding safely:** Instead of letting the program crash, exception handling lets you write code to deal with these problems—often using special keywords like try, catch, or finally.
- **Maintaining flow:** With proper exception handling, your application can recover gracefully and continue running or exit cleanly, rather than causing a system-wide crash.
- **Improving code reliability:** Handling exceptions makes your software more reliable and user-friendly, as it prepares for and manages possible failures.

Exception Handling Example:

```
public class Main {  
    public static void main(String[] args) {  
        try {  
            int[] myNumbers = {1, 2, 3};  
            System.out.println(myNumbers[10]);  
        } catch (Exception e) {  
            System.out.println("Something went wrong.");  
        }  
    }  
}
```

Output: Something went wrong.

Java Exception Keywords

There are five keywords used to handle exceptions in Java:

- **try:** The try block is used to enclose the code that might throw an exception.
- **catch:** The catch block is used to handle the exception. It must be used after the try block only.

- **finally:** The finally block is used to execute the important code of the program.

It is executed whether an exception is handled or not.

- **throw:** The throw keyword is used to throw an exception.
- **throws:** The throws keyword is used to declare an exception.

Ways of Organizing TRY-CATCH-FINALLY:

1. try {}
catch(e) {}

2. try {}
finally {}

3. try {}
catch(e) {}
finally {}

4. try {}
catch(e) {}
catch(e) {}
catch(e) {}

5. try {}
catch(e) {}
catch(e) {}
catch(e) {}
finally {}

Here's a simple Java program that demonstrates the use of multiple catch blocks with a finally block:

```
public class MultipleCatchDemo
{
    public static void main(String[] args) \
    {
        try
        {
            int[] numbers = {10, 5, 0};
            System.out.println("Result: " + (numbers[0] / numbers[2]));
            //Division by zero
            System.out.println("Access value: " + numbers[3]);
            //Array index out of bounds
        }
        catch (ArithmeticException e)
        {
            System.out.println("Arithmetic error: " + e.getMessage());
        }
        catch (ArrayIndexOutOfBoundsException e)
        {
            System.out.println("Array index error: " + e.getMessage());
        }
        finally
        {
            System.out.println("Finally block executed. Resource cleanup
if needed.");
        }
    }
}
```

Explanation:

- The try block contains code that may cause different types of exceptions.
- There are two catch blocks:
- One handles arithmetic errors (like division by zero).
- The other handles array index errors (accessing an index that doesn't exist).
- The finally block will execute no matter what, and is used for cleanup actions.

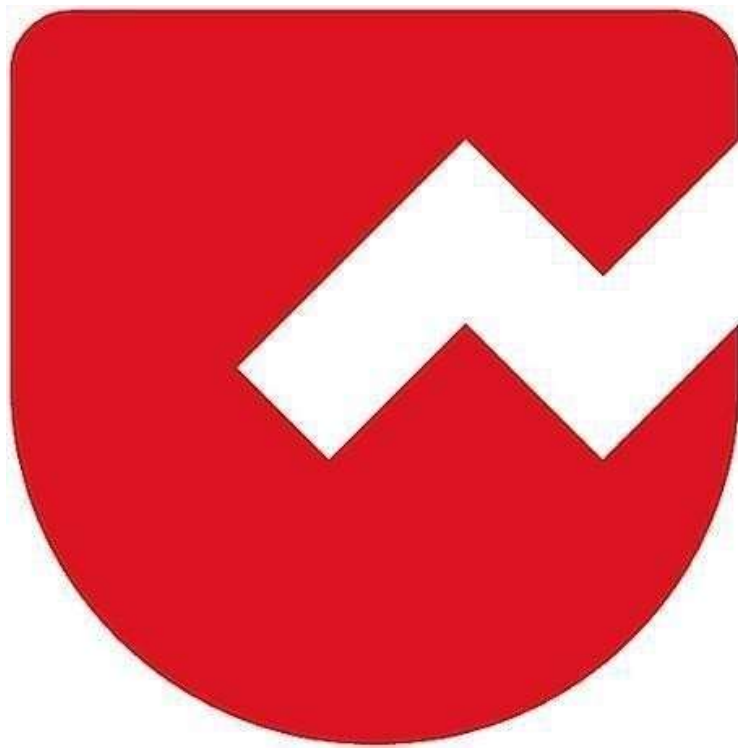
Key Notes:

1. 'try' block cannot exist independently. It must be followed either by a catch block or a finally block
2. Whenever an exception occurs, the flow of control gets transferred to the appropriate catch block.
3. The order in which catch blocks are placed is important.
i.e sub-type catch blocks must appear prior to super-type catch blocks.
4. finally block will get executed either after a try block or a catch block. One can have several lines of executable code in a finally block.
5. try block or catch block or finally block must be enclosed within a pair of braces even if the block contains only one statement.

Thank You 🍷



Zoho Schools for Graduate Studies



Notes Day-27

**Date: 24/10/2025
Day: Friday**

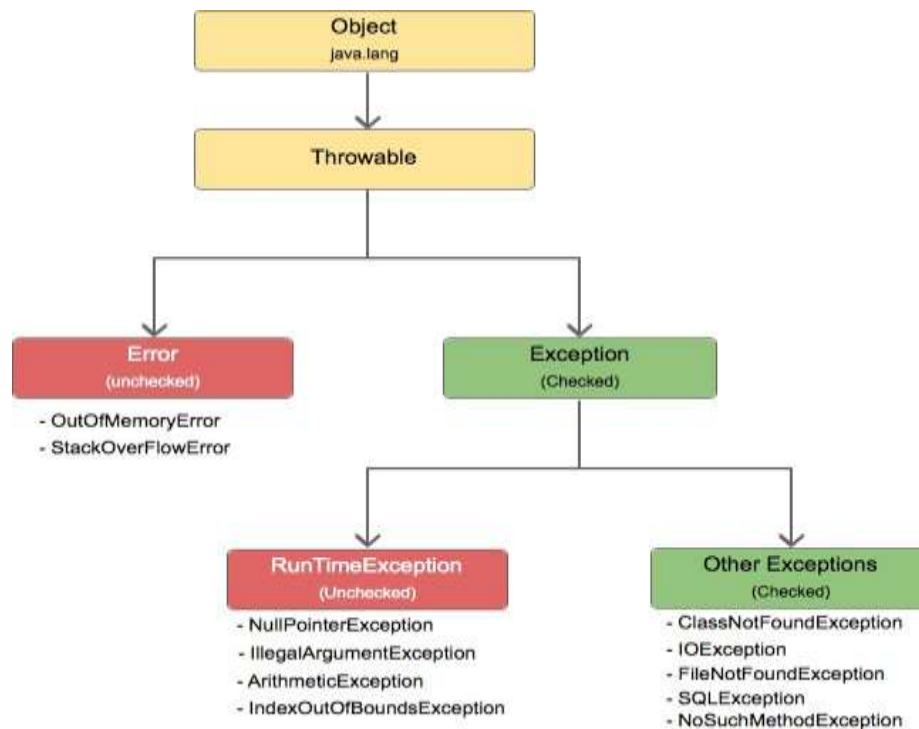
EXCEPTION HANDLING

Part – 2

throw and throws

1. Throw

- Throw keyword explicitly throw the Exception.
- Generally it is used to throw user defined exception , custom based exception.



EXCEPTION HIERARCHY

- The Argument for Catch Block must be an Exception Type that must have either throwable or any subclass of throwable.

```
try {
    throw new Throwable();
}
catch (Throwable e) {
    System.out.println("Exception Caught");
}
finally {
    System.out.println("Finally");
}
```

How to create a User defined Exception ?

- User defined exceptions are created by inheriting (extending) Exception Class.

```
public class ExcDemo {
    public static void main(String[] args) {
        try {
            int n = Integer.parseInt("-10");
            if (n < 0)
                throw new NegativeNumberException();
            else
                System.out.println(n);
        }
        catch (NegativeNumberException re) {
            System.out.println("NNEException Caught");
        }
        finally {
            System.out.println("Finally!");
        }
    }
}
```

What are Unchecked Exceptions ?

- Those exceptions for which compiler doesn't check(expect) for the presence of an exception handler are called as unchecked exceptions.
- RuntimeException and its sub-classes are unchecked exceptions.

What are checked Exceptions ?

- Those exceptions for which compiler checks for the presence of an exception handler are called as checked exceptions.
- All subclasses of Exception except RuntimeException and its subclasses are checked exceptions.

When to use throws clause ?

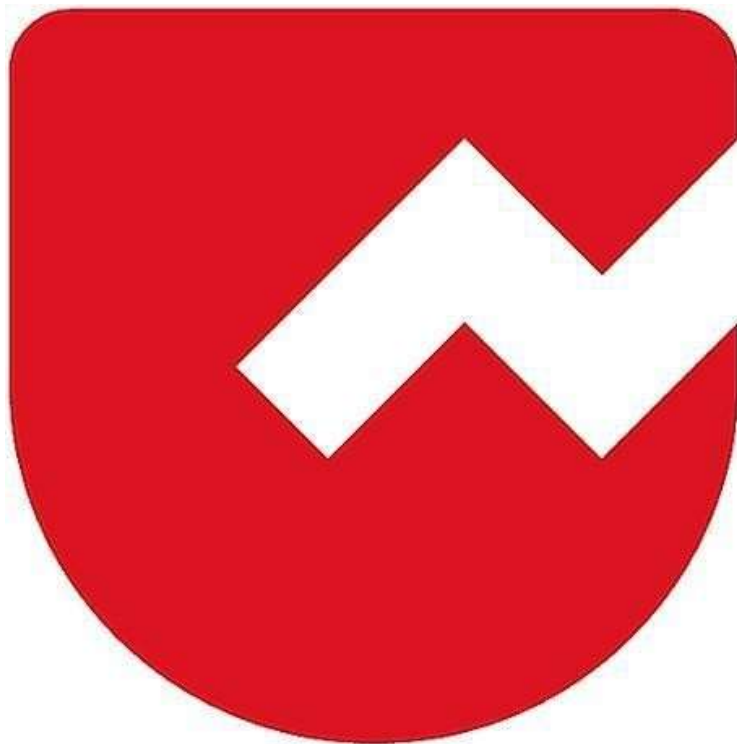
- Throws clause is used to declare the exceptions that might be thrown by the method in the absence of an exception handler for checked Exception.

Notes :

- It is Possible for a catch block to capture multiple Exception using logical 'or' operator (' | ')
- Checked Exception must be handled otherwise compiler will signal an error.
- The Argument for Catch Block must be an Exception Type that must have either throwable or any subclass of throwable.



Zoho Schools for Graduate Studies



Notes

Day-28

FLAVOURS AND SUB-FLAVOURS OF COLLECTION

CORE KEY INTERFACES OF THE COLLECTIONS FRAMEWORK :

1. Collection
2. List
3. Queue
4. Set
5. Map
6. Navigable Set
7. Sorted Set
8. Sorted Map
9. Navigable Map

Note : Collection is the root Interface in the collection hierarchy

BASIC FLAVOURS OF COLLECTION :

Collections come in four basic flavours :

1. Lists - Classes that implements List
2. Sets - Classes that implements Set
3. Maps - Classes that implements Map
4. Queue - Classes that implements Queue

THE CORE CONCRETE CLASSES OF THE COLLECTIONS FRAMEWORK :

Lists	Sets	Queues	Maps	Utilities
ArrayList	HashSet	Priority Queue	HashMap	Collections
Vector	LinkedHashSet	ArrayDeque	HashTable	Arrays
LinkedList	TreeSet		TreeMap	
Stack			LinkedHash Map	

ORDERED vs SORTED :

- An **ordered collection** means that the elements of the collection have a specific order. The Order is independent of the value. A **List** is an example.
- A **sorted collection** means that not only does the collection have order, but the order depends on the value of the element. A **SortedSet** is an example.
- In Contrast, a **collection without any order** can maintain the elements in any order. A **Set** is an example.

LIST IMPLEMENTATION CLASSES :

List	Ordered	Ordered by	Sorted
ArrayList	Yes	Index	No
Vector	Yes	Index	No
LinkedList	Yes	Index	No

SET IMPLEMENTATION CLASSES :

Set	Ordered	Ordered by	Sorted
HashSet	No		No
LinkedHashSet	Yes	Insertion Order	No
TreeSet	Yes	Sorted	By natural order or custom order

MAP IMPLEMENTATION CLASSES :

Map	Ordered	Ordered by	Sorted
HashMap	No		No
HashTable	No		No
TreeMap	Sorted		By Natural Order or Custom Order
LinkedHashMap	Yes	Insertion Order	No

QUEUE IMPLEMENTATION CLASSES :

Queue	Ordered	Ordered by	Sorted
Priority Queue	Sorted		By Natural Order or Custom Order
Array Deque	Yes	Position	No

ARRAYS VS COLLECTIONS :

Key	Arrays	Collection
Size	Arrays or Fixed in size i.e. static	Dynamic in size
Memory Usage	Inefficient use of memory	Efficient use of memory
Data Type	Stores only homogeneous data	Can store both homogeneous and heterogeneous data
Primitive Storage	Can store both objects and primitives	Can store only object types
Performance	Better in Performance	Relatively low in Performance

ArrayList – Complete Notes

What is an ArrayList?

- ArrayList is part of the Java Collection Framework and implements the List interface.
- Elements are ordered using index values (0-based indexing).
- Internally uses a dynamic resizable array.
- Allows duplicates and null values.
- Default initial capacity = 10.

When to Use ArrayList?

- ✓ Fast retrieval – $O(1)$
- ✓ Good for frequent access operations

When NOT to Use ArrayList?

- ✗ Avoid when many insertions/deletions happen in the middle (shifting occurs – $O(n)$)

Important Methods

add(value), remove(index), get(index), set(index, value), clear(), subList(start, end), toArray(), trimToSize()

LinkedList – Complete Notes

What is LinkedList?

- Linear collection of nodes linked by pointers.
- Not stored in continuous memory.
- Fast insertion/deletion, slow retrieval.

Node Structure

Singly LL → Data + Next Pointer

Doubly LL → Data + Prev + Next Pointer

Features

- Supports duplicates & nulls
- Not synchronized
- Faster insertion/deletion
- Slow random access

LinkedList vs ArrayList Comparison

Index Support: LinkedList – No | ArrayList – Yes

Random Access: LinkedList – No | ArrayList – Yes

Most Expensive Op: LinkedList – Retrieval | ArrayList – Insert/Delete

Singly vs Doubly LL

Space: Singly – Less | Doubly – More

Implementation: Singly – Easy | Doubly – Hard

Insert/Delete Time: Singly – Less | Doubly – More

Navigation: Singly – One-way | Doubly – Two-way

Quiz Section

Quiz Section

1. Default size of LinkedList?

Answer: 0

2. Why doesn't LinkedList support random access?

Answer: It does not implement RandomAccess and requires traversal.

3. Worst-case insertion time in doubly LL?

Answer: $O(n)$ if traversal needed, $O(1)$ if reference known.

4. Method to trim ArrayList capacity?

Answer: trimToSize()

This document was generated with Writer, and the link to code snippets is provided [here](#).

List Creation with List.of

```
1 import java.util.List;
2
3 public class ListDemo {
4     public static void main(String[] args) {
5         List<String> wordList =
List.of("Bad", "Dad", "Dead");
6
7         wordList.add("Road");
8         wordList.remove("Dead");
9     }
10 }
```

When list is created using [List.of\(\)](#) or [List.copyOf\(\)](#), the list will be become immutable. The above code will throw a [UnsupportedOperationException](#). If you want to do it. Here's another version of the code.

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class ListDemo {
5     public static void main(String[] args) {
6         List<String> wordList =
List.of("Bad", "Dad", "Dead");
7         List<String> mutableWordList = new
ArrayList<>(wordList);
8         mutableWordList.add("Road");
9         mutableWordList.remove("Dead");
10
11         System.out.println(mutableWordList);
```

```
12     }  
13 }
```

The above will do, if the list needs modification. There's also another case when the list will be immutable when using [Arrays.asList\(\)](#).

Traversing Collections

Traversing with Enhanced For Loop,

```
1 import java.util.ArrayList;  
2 import java.util.List;  
3  
4 public class ListDemo {  
5     public static void main(String[] args) {  
6         List<String> wordList =  
List.of("Bad", "Dad", "Dead");  
7         List<String> mutableWordList = new  
ArrayList<>(wordList);  
8         mutableWordList.add("Road");  
9         mutableWordList.remove("Dead");  
10  
11         for(String word : mutableWordList) {  
12 //             if(word.endsWith("ad"))  
13 //                 mutableWordList.remove(word);  
14 //             this will throw  
ConcurrentModificationException  
15                 System.out.print(word + " ");  
16         }  
17     }  
18 }
```

Traversing with For loop

```
1 import java.util.ArrayList;
```

```

2 import java.util.List;
3
4 public class ListDemo {
5     public static void main(String[] args) {
6         List<String> wordList =
List.of("Bad", "Dad", "Dead");
7         List<String> mutableWordList = new
ArrayList<>(wordList);
8         mutableWordList.add("Road");
9         mutableWordList.remove("Dead");
10        int length = mutableWordList.size();
11
12        for (int i = 0; i < length; i++) {
13            System.out.print(mutableWordList.get(i) + "
");
14        }
15    }
16 }

```

Traversing with Iterator

```

1 import java.util.ArrayList;
2 import java.util.Iterator;
3 import java.util.List;
4
5
6 public class ListDemo {
7     public static void main(String[] args) {
8         List<String> wordList =
List.of("Bad", "Dad", "Dead");
9         List<String> mutableWordList = new
ArrayList<>(wordList);
10        mutableWordList.add("Road");
11        mutableWordList.remove("Dead");
12        Iterator<String> iterator =

```



```

mutableWordList.iterator();
13
14     while(iterator.hasNext()) {
15         String word = iterator.next();
16         if(word.endsWith("ad"))
17             iterator.remove();
18     }
19
20     System.out.println("After removal : " +
mutableWordList);
21 }
22 }

```

[Iterator](#) is a interface in java.util package. With Iterator it is possible to avoid ConcurrentModificationException. insertion is not possible because Iterator provides only the following methods

- hasNext()
- next()
- remove()
- forEachRemaining(Consumer<? super E> action)

Vector

The [Vector](#) class provides a dynamic array that stores objects, allowing elements to be accessed via integer indices, similar to a standard array. Unlike a fixed-size array, its capacity can expand or contract as items are added or removed after creation.

All Implemented Interfaces:

[Serializable](#), [Cloneable](#), [Iterable](#)<E>, [Collection](#)<E>, [List](#)<E>, [RandomAccess](#)

The initial capacity is 10 and doubles in capacity by default, the capacity and capacityIncrement can also be provided in the constructor.

Vector

1. *Vector is a collection that implements List interface.*
2. *Elements are ordered by index and index is zero based.*
3. *It is built on a data structure called as a resizable array or growable array.*
4. *Null insertion is possible.*
5. *Default capacity is 10.*
6. *Preferred when retrieval is the frequent operation*
7. *It will be a worst choice when insertion/deletion becomes frequent operation*

20

Vector

8. *Vector doesn't have default initialization.*
9. *Vector can be multi-dimensional.*
10. *Methods in an Vector are synchronized.
Hence, they are thread safe.*
11. *Vector implements RandomAccess Interface.*

21

ArrayList Vs Vector

ArrayList Vs Vector

ArrayList	Vector
ArrayList is not synchronized.	Vector is synchronized.
ArrayList increments 50% whenever initial capacity exceeds.	Vector increments 100%
ArrayList is not a legacy class.	It is a legacy class.
It is relatively faster	It is slow
ArrayList can use either Iterator interface or ListIterator to traverse the elements.	A Vector can use either the Iterator interface or Enumeration interface

22

VectorDemo

```
1 package part3;
2
3 import java.util.*;
4
5 public class VectorDemo {
6     public static void main(String[] args) {
7         Vector v=new Vector();
8         for(int i=1;i<=10;i++)
9             v.addElement(i);
10        System.out.println(v);
11    }
12 }
13
14 }
15
```

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Vector Changing Capacity

```
1 package part3;
2
3 import java.util.*;
4
5 public class VectorDemo {
6     public static void main(String[] args) {
7         Vector v=new Vector();
8         System.out.println(v.capacity());
9         for(int i=1;i<=10;i++)
10             v.addElement(i);
11         System.out.println(v);
12         v.add(11);
13         System.out.println(v.capacity());
14     }
15 }
16
17 }
18
```

10
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
20

Enumeration in Vector

```
1 package part3;
2
3 import java.util.*;
4
5 public class VectorDemo {
6     public static void main(String[] args) {
7         Vector v=new Vector();
8         System.out.println(v.capacity());
9         for(int i=1;i<=10;i++)
10             v.addElement(i);
11         System.out.println(v);
12         v.add(11);
13         System.out.println(v.capacity());
14
15         for (Enumeration<E> e = v.elements(); e.hasMoreElements();){
16             System.out.println(e.nextElement());
17         }
18     }
19 }
20 }
```

Stack

Stack

- **Stack** is a linear data structure which follows a LIFO or FILO order in which the operations are performed.

[Stack](#) is a subclass of Vector in java. The standard version of Stack utilization in java is,

```
Stack<?> stack = new Stack<>();
```

Even though the above works for most of the cases in data structures. A more complete and consistent set of LIFO stack operations is provided by the [Deque](#) interface and its implementations, which should be used in

preference to this class. For example:

```
Deque<Integer> stack = new ArrayDeque<Integer>();
```

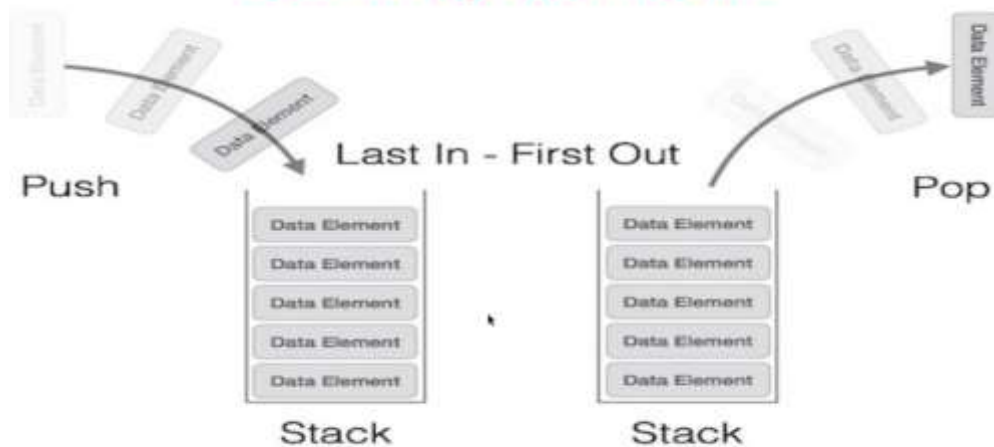
Stack Hierarchy

Hierarchy

- <Collection> (1.2)
 - <List> (1.2)
 - ArrayList (1.2)
 - LinkedList (1.2)
 - Vector (1.0) [legacy classes]
 - Stack (1.0)

Stack Representation

Stack Representation



Quiz Answers

Quiz 1 -Answer

Does ArrayList have default initialization like an Array?

- a) Yes
- b) No

Quiz 2 - Answer

*ArrayList can be multi-dimensional like an Array?
State True or False.*

- a) True
- b) False

Quiz 3 - Answer

LinkedList doesn't implement RandomAccess interface. State True or False.

- a) *True*
- b) *False*

9

Quiz4

What is the output of the code?

```
public class ArrayListDemo (  
public static void main(StringO args) {  
List<String> list =new ArrayList<>();  
list.add("Mondray'J;  
list.add(""?;  
list.add("Tuesday'□,;  
list.remove(O) •  
System.out.println(list.get(O));  
}}
```


Quiz 4 - Answer

- a) *An empty String*
- b) *Tuesday*
- c) *The code doesn't compile*
- d) *The code compiles but throws an exception*

Quiz5

Default capacity of Vector is-----

10

Quiz6

*Vector allows the storage of null values.
State True or False.*

- a) *True*
- b) *False*

Quiz 7 - Answer

What is the main difference between ArrayList and Vector.

*Vector is **synchronized** (thread safe) whereas ArrayList is **not synchronized** (thread s,afe).*

Quiz 1 -Answer

Does Stack class implements List Interface?

- a) Yes
- b) No

Quiz 2 - Answer

Stack class implements Random Access Interface. State True or False.

- a) *True*
- b) *False*

6

Quiz 3 - Answer

LIFO stands for "Last In First Out" strategy

Quiz 4 - Answer

What is the immediate super class for Stack class.

- a) *ArrayList*
- b) *LinkedList*
- c) *Vector*
- d) *List*

14

Quiz 5 - Answer

The output is 1100

```
public class Test{
    public static void main(String[] args){
        Stack<Integer> stack= new Stack<Integer>();
        int n = 12;
        while (n > 0){
            stack.push(n%2);
            n = n/2;}
        String result = "";
        while (!stack.isEmpty())
            result += stack.pop();
        System.out.println(result)}
    }
```



Zoho Schools for Graduate Studies



Notes

Generics:

Generics allow classes and methods to operate on different data types while providing type safety.

Example:

The CustomList<T> class is a generic class that can store elements of any type. By using <T>, the same class works for String, Integer, Double, or any custom class.

```
1 package part3;
2
3 import java.util.*;
4
5 public class CustomList<T> {
6     List<T> al=new ArrayList<T>();
7
8     public void addElement(T s) {
9         al.add(s);
10    }
11
12    public void removeElement(T s) {
13        al.remove(s);
14    }
15
16    public String toString() {
17        return al.toString();
18    }
```

- Without overriding toString(), printing the object shows its memory reference.

```
2
3 public class CustomListDemo {
4     public static void main(String[] args) {
5         CustomList<String> cl=new CustomList<>()
6         cl.addElement("one");
7         cl.addElement("two");
8         cl.addElement("three");
9         cl.removeElement("two");
10        System.out.println(cl); // [one, two, three]
11
12        CustomList<Integer> cl1=new CustomList<>();
13        cl1.addElement(12); // auto-boxing
14        cl1.addElement(new Integer(22)); // auto-boxing
15        cl1.addElement(Integer.valueOf(30));
16        cl1.removeElement(12);
17        System.out.println(cl1);
18    }
19 }
```

Generic method:

A **generic method** is a method that can work with **different data types** using a **type parameter**.

Example:

```
static public <T> T getData(T obj) {  
    return obj;  
}
```

Explanation:

- **<T>** → Type parameter.
It tells Java: “This method uses a type T.”
- **T getData(...)** → Return type is T.
The method returns the same type that it receives.
- **getData(T obj)** → Takes an argument of type T.
- **return obj** → Returns the same value.

In Java, we can restrict a generic type to only accept certain types using an **upper bound**.

Example:

```
3 import java.util.*;  
4  
5 public class CustomList<T extends List> {  
6     List<T> al=new ArrayList<T>();  
7  
8     public void addElement(T s) {  
9         al.add(s);  
10    }  
11  
12    public void removeElement(T s) {  
13        al.remove(s);  
14    }  
15  
16    public T get(int index) {  
17        return al.get(index);  
18    }
```

In this, **T extends List**, means T must be a List type.

So T can only be:

- ArrayList
- LinkedList
- Vector

- Any class that implements **java.util.List**

```
public class Main {  
    public static void main(String[] args) {  
  
        CustomList<ArrayList<Integer>> cl = new CustomList<>();  
  
        ArrayList<Integer> l1 = new ArrayList<>();  
        l1.add(10);  
        l1.add(20);  
  
        ArrayList<Integer> l2 = new ArrayList<>();  
        l2.add(100);  
        l2.add(200);  
  
        cl.add(l1);  
        cl.add(l2);  
  
        System.out.println(cl); // [[10, 20], [100, 200]]  
        System.out.println(cl.get(0)); // [10, 20]  
    }  
}
```




Zoho Schools for Graduate Studies



Notes Queue Day-30

27/11/2025

Understanding the Queue Data Structure

Definition

A Queue is a linear data structure that follows the FIFO (First-In, First-Out) principle. This means the element that has been in the queue the longest is the next one to be removed.

Core Operations

The primary operations for managing elements within a Queue have specific terminology:

- Enqueue: The action of inserting an element into a queue.
- Dequeue: The action of removing or deleting an element from a queue.

Note: The core implementation classes for Queues shown in the Collections table are `PriorityQueue` and `ArrayDeque`.

The time complexity for the fundamental operations of a Queue depends on the specific implementation used. The most common implementations in standard libraries (like Java's `ArrayDeque` OR `LinkedList`) are highly optimized.

Here is a breakdown of the time complexity for the core Queue operations:

Queue Operation Time Complexity

Operation	Implementation	Time Complexity	Notes
Enqueue (Insert at Rear)	Array-based (e.g., <code>ArrayDeque</code>)	$O(1)$	Constant time, as the pointer/index to the rear is simply incremented.

Operation	Implementation	Time Complexity	Notes
	Linked List-based (e.g., LinkedList)	O(1)	Constant time, as a new node is simply attached to the tail of the list.
Dequeue (Remove from Front)	Array-based (e.g., ArrayDeque)	O(1)	Constant time, as the pointer/index to the front is simply incremented.
	Linked List-based (e.g., LinkedList)	O(1)	Constant time, as the head pointer is moved to the next node.
Peek (Get Front Element)	All Implementations	O(1)	Constant time, as it only involves reading the value at the known front index/pointer.

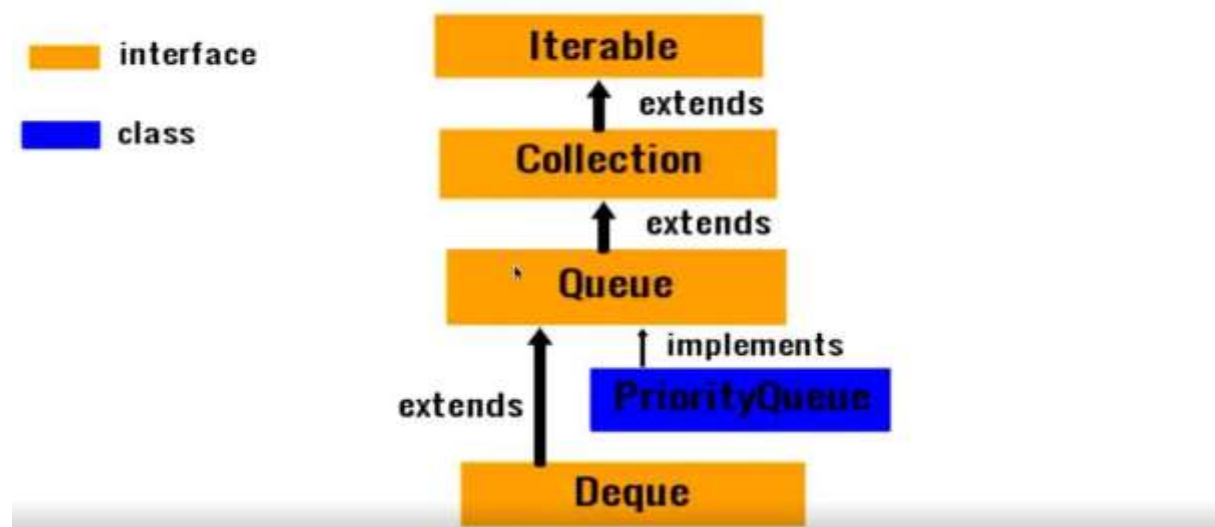
Difference in Add & offer method in Queues

Both the `add(e)` and `offer(e)` methods are used to insert an element (`e`) into the Queue. The primary distinction lies in their behavior when the insertion fails, particularly in capacity-restricted (or "bounded") Queue implementations.

Method	Behavior on Successful Insertion	Behavior on Insertion Failure (Queue is Full)	Return Value Type
<code>add(e)</code>	Inserts the element at the tail of the queue.	Throws an <code>IllegalStateException</code> (an exception).	boolean (usually returns true).
<code>offer(e)</code>	Inserts the element at the tail of the queue.	Returns the special value <code>false</code> .	boolean

QUEUE HIERARCHY

Quiz 3 – Queue Hierarchy - Explanation



Element	Type	Relationship
Iterable	Interface (Orange)	The root of all collections that can be iterated.
Collection	Interface (Orange)	Extends <code>Iterable</code> . The base interface for all collection types (List, Set, Queue).
Queue	Interface (Orange)	Extends <code>Collection</code> . Designed for holding elements prior to processing, usually in a FIFO order.
Deque	Interface (Orange)	Extends <code>Queue</code> . Represents a Double-Ended Queue, allowing element insertion and removal at both ends (front and rear).

Element	Type	Relationship
PriorityQueue	Class (Blue)	Implements <code>Queue</code> . A concrete implementation class that provides priority-based ordering.

Priority Queue Explanation

A Priority Queue is a specialized extension of the standard Queue data structure, offering priority-based ordering instead of strict FIFO ordering.

Characteristics

- **Priority Association:** Every element inserted into a priority queue has a priority associated with it.
- **Deletion Order:** The element with the higher priority will be deleted (removed or polled) *before* elements with lesser priority.
- **Tie-Breaker:** If two or more elements have the same priority, their relative order is determined by the FIFO (First-In, First-Out) principle.
- **Internal Structure:** The `PriorityQueue` class in Java is typically implemented using a Min-Heap, where the element with the highest priority (often the smallest value) is always at the root, making its retrieval $O(1)$.

Priority Queue Ordering: Comparable and Iterator

The `PriorityQueue` class relies on two key mechanisms for element ordering and traversal.

1. Ordering and Comparable / Comparator

The `PriorityQueue` determines the priority of elements using either the elements' natural ordering or a custom ordering provided at construction time.

- Natural Ordering (via Comparable):
 - If no custom comparator is provided, the elements stored in the PriorityQueue must implement the `java.lang.Comparable` interface.
 - This interface forces the element class to define its own "natural" comparison logic using the `compareTo()` method.
 - Example: Standard wrapper classes like `Integer` and `String` already implement `Comparable`.
- Custom Ordering (via Comparator):
 - The PriorityQueue can be constructed with an instance of the `java.util.Comparator` interface.
 - This is typically used when you need a sorting order different from the natural one, or when the element class is a custom type that doesn't implement `Comparable`.
 - The Comparator provides the comparison logic externally via the `compare(T o1, T o2)` method.

2. Traversal and Iterator method

- Implements Iterable: Since PriorityQueue implements the Queue interface, which extends Collection, which extends Iterable, the PriorityQueue provides an `iterator()` method.
- Iteration Order: The `iterator()` method does not guarantee traversing the elements in their priority-sorted order. The iterator traverses the elements in the order they are stored in the underlying heap structure (often a level-order traversal), not the sorted logical order.
- For Sorted Traversal: If you require elements in strict priority order, you must repeatedly call the `poll()` method, which always retrieves and removes the highest-priority element.

Small Quiz on Priority Queue (try on yourself)

```
package part3;
import java.util.*;
public class QueueDemo {
    public static void main(String[] args) {
        PriorityQueue<String>pq=new PriorityQueue<>();
        pq.add("2");pq.add("4");
        System.out.print(pq.peek() + " ");
        pq.offer("1");
        pq.add("3");pq.remove("1");
        System.out.print(pq.poll() + " ");
        if(pq.remove("2"))
            System.out.print(pq.poll() + " ");
        System.out.print(pq.poll() + " " + pq.peek() + " ");
    }
}
```

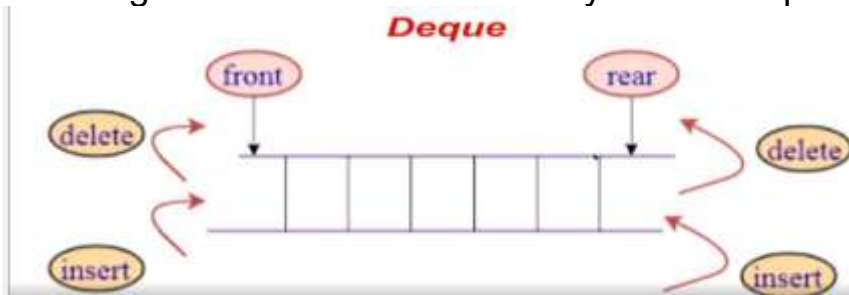
Deque (Double-Ended Queue) Explanation

A Deque (pronounced "deck" or sometimes "D.E.Q.") stands for Double-Ended Queue. It is a special type of queue that supports the addition and removal of elements from both ends—the front and the rear.

Core Concept

Unlike a standard Queue (which enforces FIFO—First-In, First-Out) or a Stack (which enforces LIFO—Last-In, First-Out), a Deque is more flexible. It provides the functionality of both a queue and a stack in a single data structure, depending on how you choose to use its operations.

The diagram illustrates the flexibility of the Deque:



Key Operations

A Deque allows four primary operations, enabling it to act as a double-ended structure:

Operation	Action	End Affected	Standard Queue/Stack Analog
Insert at Front	Add an element to the beginning.	Front	Stack's Push (LIFO insertion)
Delete from Front	Remove an element from the beginning.	Front	Queue's Dequeue (FIFO removal)
Insert at Rear	Add an element to the end.	Rear	Queue's Enqueue (FIFO insertion)
Delete from Rear	Remove an element from the end.	Rear	Stack's Pop (LIFO removal)

Implementations in Java

In the Java Collections Framework, the `Deque` is an interface that extends the `Queue` interface. Common concrete classes that implement `Deque` include:

1. `ArrayDeque`: An array-based implementation that resizes dynamically (like `ArrayList`). It is generally the preferred and most efficient implementation for `Deques` and `Queues` in Java.
2. `LinkedList`: A doubly-linked list implementation. While it supports the `Deque` interface, `ArrayDeque` is often faster for most `Deque` operations.

Time Complexity

The major advantage of well-implemented `Deques` (like `ArrayDeque`) is that all core insertion and deletion operations at both the front and rear are performed in constant time, $O(1)$

Array Deque

- Unlike `Queue`, one can add or remove elements from both sides.
- Null elements are not allowed.
- It is not thread safe, in the absence of external synchronization.
- It has no capacity restrictions.
- It is faster than `LinkedList` and `Stack`.

Expected Output:

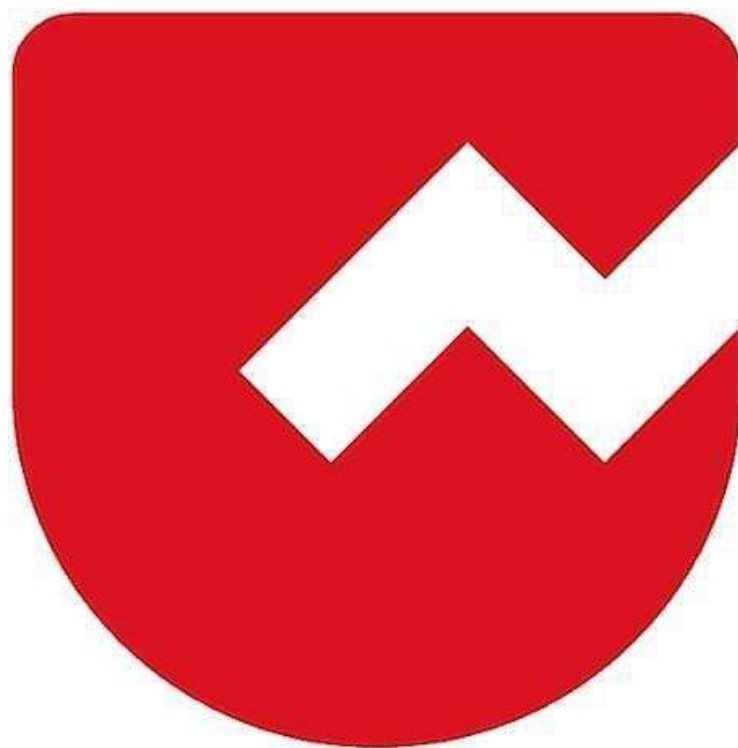
```
import java.util.ArrayDeque;
class Main {
    public static void main(String[] args) {
        ArrayDeque<String> ad = new ArrayDeque<>();
        ad.add("2");
        ad.add("4");
        System.out.print(ad.peek() + ", ");
        ad.offer("1");
        ad.add("3");
        ad.remove();
        System.out.print(ad.peek() + ", ");
        If(ad.peek().equals("2")){
            System.out.println(ad.poll() + " ");
            System.out.print(ad.poll() + " " + ad.peek() + " ");
        }
    }
}
```

Output:

2 4 1 3



Zoho Schools for Graduate Studies



Notes
Day-31

Bound wildcards in Generics

Session Date: December 3, 2025

1. Core Concepts

Generics vs. Arrays (Invariance vs. Covariance)

- **Arrays are Covariant:** `Integer[]` is a subtype of `Number[]`. You can assign an `Integer` array to a `Number` array reference.
- **Generics are Invariant:** `List<Integer>` is **NOT** a subtype of `List<Number>`.
 - *Reasoning:* If `List<Integer>` were a subtype of `List<Number>`, you could dangerously add a `Double` to a list of `Integers` via the `Number` reference, causing runtime errors.

Type Erasure

- **Definition:** Generics are a **compile-time** safety feature. The Java Virtual Machine (JVM) does not know about generic types at runtime.
- **Mechanism:** During compilation, the Java compiler validates type safety and then "erases" the specific type parameters (e.g., `<Integer>`), replacing them with their bound (e.g., `Number`) or `Object`.
- **Implication:** You cannot use generic types in runtime checks like `instanceof T` or create new instances like `new T()`.

2. Bounded Type Parameters

To write flexible code that works with multiple specific types (like `Integer`, `Double`, `Float`), we use **Bounded Type Parameters**.

Syntax

```
public <T extends Number> void method(List<T> list)
```

- **<T extends Number>**: This declares a generic type `T` that is restricted to `Number` or its subclasses.
- **Benefit:** This allows you to call methods defined in the `Number` class (like `doubleValue()`, `intValue()`) on the generic objects inside the method.

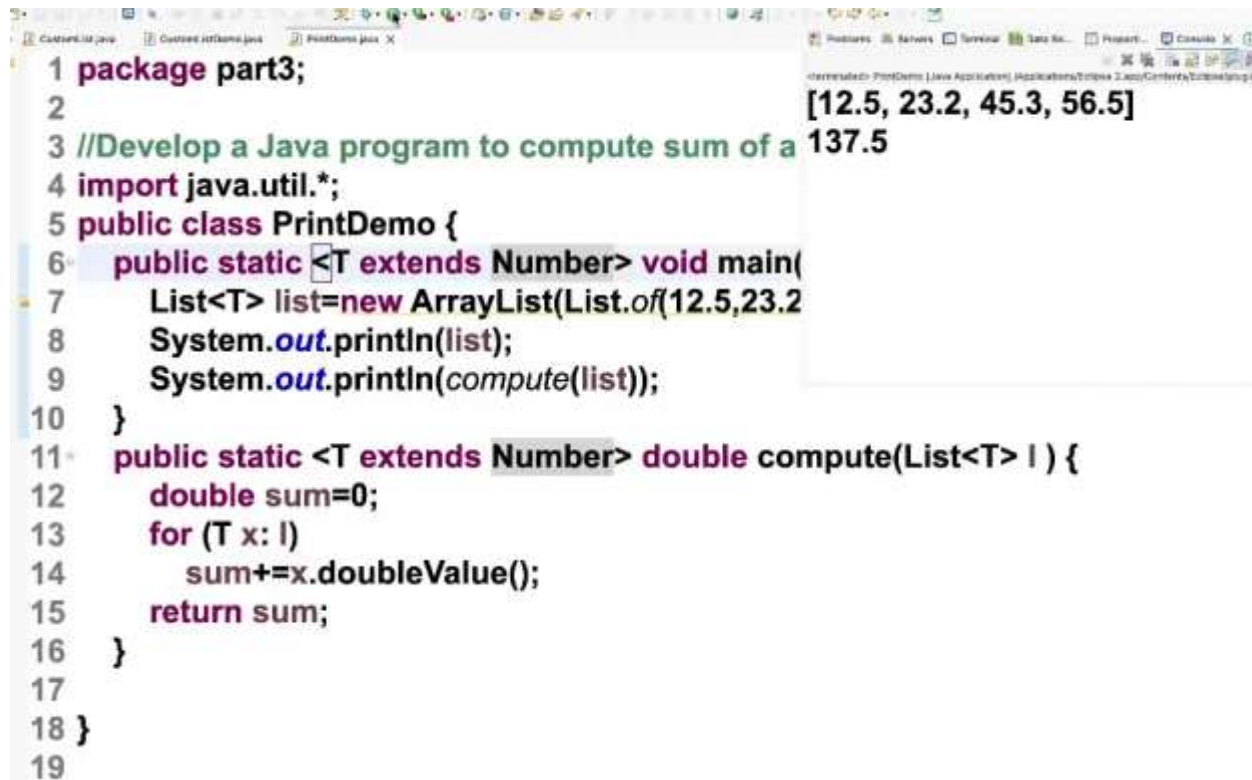
Unbounded vs. Bounded

- **Unbounded (<T> or <?>):** Accepts any Java Object. Treated as `Object` inside the method.
- **Upper Bounded (<T extends Number> or <? extends Number>):** Accepts `Number` and its subclasses. Treated as `Number` inside the method.

3. Practical Code Example: Sum of Numbers

Objective: Write a single method compute that calculates the sum of a list of numbers, regardless of whether they are Integers, Doubles, or Longs.

The Solution



```
1 package part3;
2
3 //Develop a Java program to compute sum of a
4 import java.util.*;
5 public class PrintDemo {
6     public static <T extends Number> void main(
7         List<T> list=new ArrayList(List.of(12.5,23.2
8         System.out.println(list);
9         System.out.println(compute(list));
10    }
11    public static <T extends Number> double compute(List<T> l ) {
12        double sum=0;
13        for (T x: l)
14            sum+=x.doubleValue();
15        return sum;
16    }
17
18 }
19
```

Output: [12.5, 23.2, 45.3, 56.5]
137.5

Key Rules for Implementation

1. **Primitives are Forbidden:** You cannot use List<int> or List<double>. Generics only work with Reference Types (Objects). You must use wrapper classes like List<Integer> and List<Double>.
2. **Generic Declaration:** The <T extends Number> declaration must appear **before** the return type of the method.
3. The "Wildcard" Alternative: The method signature could also be written using wildcards:
`public static double compute(List<? extends Number> list)`

4. Wildcards

Wildcards (?) allow for more flexible method parameters when the exact type is unknown or irrelevant.

Types of Wildcards

1. Unbounded Wildcard (<?>):

- Represents a list of *any* type.
- Useful when the method only uses methods from the Object class or doesn't depend on the specific type parameter.
- Example: `void printList(List<?> list)` can print a list of anything.

2. Upper Bounded Wildcard (<? extends Number>):

- Restricts the unknown type to be a specific type or a **subtype** of that type.
- Usage: Useful for "reading" from a list where you know everything in the list is at least a Number.
- Example: `double sum(List<? extends Number> list)` allows List<Integer>, List<Double>, etc.

```
package part3;
```

```
//Develop a Java program to compute sum of a given list of numbers
```

```
import java.util.*;
```

```
import java.io.*;
```

```
public class PrintDemo {
```

```
    public static void main(String[] args) {
```

```
        List<? extends Number> list=new ArrayList(List.of(12L,23L,45L,56L));
```

```
        System.out.println(list);
```

```
        System.out.println(compute(list));
```

```
    }
```

```
    public static double compute(List<? extends Number> l ) {
```

```
        double sum=0;
```

```
        for (Number x: l)
```

```
            sum+=x.doubleValue();
```

```
        return sum;
```

```
    }
```

3. Lower Bounded Wildcard (<? super Integer>):

- Restricts the unknown type to be a specific type or a **super** type of that type.
- *Usage:* Useful for "writing" to a list (e.g., adding Integers to a List<Number> or List<Object>).
- *Example:* `void addNumbers(List<? super Integer> list)` allows List<Integer>, List<Number>, List<Object>.

Wildcards vs. Bounded Type Parameters

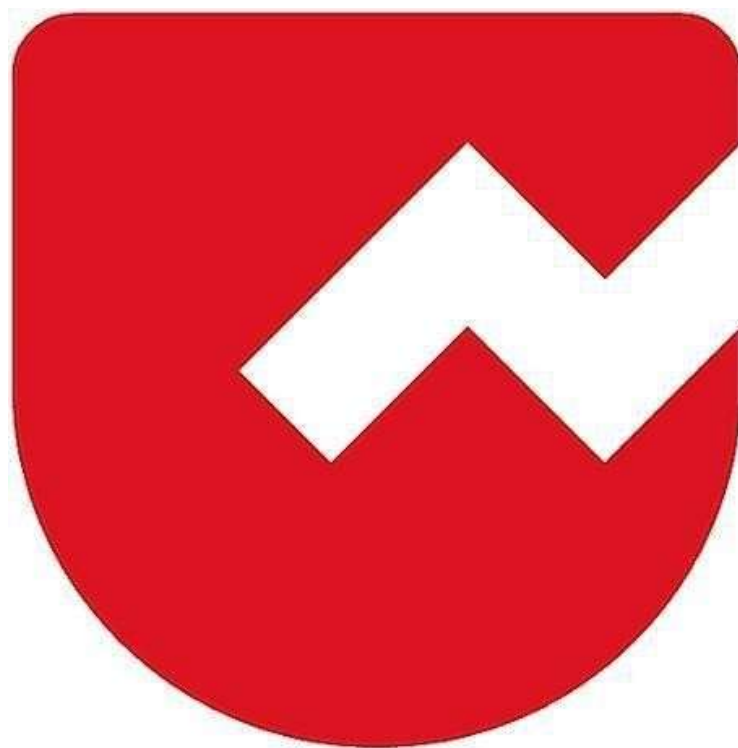
- **Bounded Parameter (<T extends Number>):** Use when you need to refer to the type T multiple times (e.g., strictly ensuring the return type is T, or using T to declare variables inside the method).
- **Wildcard (<? extends Number>):** Use when you only need to access the list and don't care about the specific type name inside the method body (often cleaner syntax).

5. Key Takeaways

1. **Code Reuse:** Generics prevent code replication (avoiding separate methods for `sum(List<Integer>)`, `sum(List<Double>)`, etc.).
2. **Type Safety:** Generics move type checking from runtime to compile-time, preventing `ClassCastException`.
3. **The "Number" Hierarchy:** Remember that `Byte`, `Short`, `Integer`, `Long`, `Float`, and `Double` all extend the abstract class `Number`.



Zoho Schools for Graduate Studies



Notes

Day-32

Java I/O Sources

Session Date: December 4, 2025

1. Introduction to Java I/O

Java I/O (Input/Output) is a system that allows programs to read data from Sources and write data to destinations. These sources can include files, keyboard input, network connections, memory, etc.

2. Streams in Java

Streams represent a flow of data. They are of two main types:

A. Byte Streams (8-bit data)

- Classes: FileInputStream, FileOutputStream
- Used for binary data such as images, audio files, PDFs, etc.

B. Character Streams (16-bit Unicode)

- Classes: FileReader, FileWriter
- Used for reading and writing text files.

3. Types of I/O Sources

Java can perform I/O operations on multiple sources:

A. Console / Keyboard

- Using System.in, System.out, Scanner
- Reads user input from the terminal.

B. Files

- FileInputStream, FileOutputStream, BufferedReader, FileWriter
- Most commonly used for persistent data storage.

C. Network

- Using Sockets (TCP/UDP)
- `socket.getInputStream()`, `socket.getOutputStream()`

D. Memory Streams

- `ByteArrayInputStream`
- `CharArrayReader`
- Used when data is already present in memory.

The Solution

```
public class ByteArrayDemo {  
    public static void main(String[] args) {  
        byte[] b="RajaGanapathy likes Briyani".getBytes();  
        ByteArrayInputStream bis=new ByteArrayInputStream(b);  
        String destPath="/Users/selvam-8490/Desktop/EclipseProjects/GS20252/src/  
        FileOutputStream fos=new FileOutputStream(destPath);  
        int value=bis.read();  
        while(value!=-1) {  
            fos.write(value);  
            value=bis.read();  
        }  
        bis.close();  
        fos.close();  
        System.out.println("Byte Array contents are copied successfully!");  
    }  
}
```

4. Buffered Streams (Decorator Pattern)

Buffered streams improve performance by reducing the number of I/O operations.

Examples:

- BufferedInputStream
- BufferedOutputStream
- BufferedReader
- BufferedWriter

They wrap around basic streams and make reading and writing faster.

5. Difference Between Stream Types

- InputStream / OutputStream → handles raw bytes
- Reader / Writer → handles characters (text)

Examples:

- To read a text file: use FileReader
- To read a binary file: use FileInputStream

```
import java.io.*;

// Develop a Java program to copy the contents of a file

public class FileCopyDemo {
    public static void main(String[] args) throws Exception {
        String srcPath="/Users/selvam-8490/Desktop/EclipseProjects/GS20252/src/p
        FileInputStream fis=new FileInputStream(srcPath);
        FileOutputStream fos=new FileOutputStream("t2.txt");
        int value=fis.read();
        while(value!=-1) {
            fos.write(value);
            value=fis.read();
        }
        fis.close();
        fos.close();
        System.out.println("File contents are copied successfully!");
    }
}
```

Input source – Generates Stream

- File (FileInputStream)
- Keyboard (System.in)
- Byte Array (ByteArrayInputStream)
- Object (ObjectInputStream)

Output Source – Consumes Stream

- Monitor (System.out)
- File (FileOutputStream)
- ObjectOutputStream

Stream : A sequential flow of data

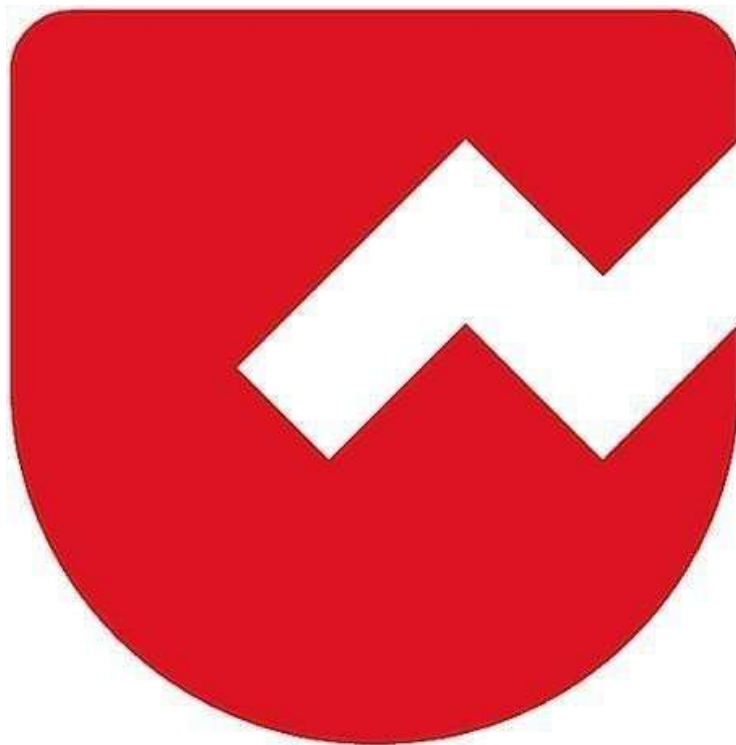
Classification of Stream:

- Byte Oriented Stream → read/write byte by byte
- Character Oriented Stream → read/write character by character

6. Important Points

- Streams are the core mechanism for data flow in Java
- Java supports both byte and character operations
- Multiple I/O sources: console, file, network, memory
- Buffers significantly speed up I/O processing

Zoho Schools for Graduate Studies



Notes

Day-33

Streams in Java

A **Stream** in Java is a **flow of data** between a **source** and a **destination**.

They are of two main types:

1. Byte Streams (8-bit data)

- Classes: FileInputStream, FileOutputStream
- Used for binary data such as images, audio files, PDFs, etc.

2. Character Streams (16-bit Unicode)

- Classes: FileReader, FileWriter
- Used for reading and writing text files.

File Read Time Measurement Using FileInputStream

```
import java.io.FileInputStream;
import java.io.FileOutputStream;

public class FileReadTimeTakenDemo {
    public static void main(String[] args) throws Exception {
        String srcPath="/Users/selvam-8490/Desktop/EclipseProjects/GS20252/src/p
        FileInputStream fis=new FileInputStream(srcPath);
        long t1=System.currentTimeMillis();
        int value=fis.read();
        while(value!=-1) {
            value=fis.read();
        }
        long t2=System.currentTimeMillis();
        fis.close();
        System.out.println("Time Taken=" + (t2-t1)/(float)1000 + "seconds");
    }
}
```

Output: Time Taken = 6.399 Seconds

FileInputStream

FileInputStream is a Java class used to **read data from a file in the form of bytes**. It belongs to the **java.io** package and is mainly used for reading **binary files** like images, videos, audio, or even text files byte-by-byte.

System.currentTimeMillis()

System.currentTimeMillis() is a Java method that returns the **current time in milliseconds** (1 second = 1000 milliseconds) from the **Unix epoch** — which is **January 1, 1970, 00:00:00 UTC**.

This program measures the time taken to read a file using **FileInputStream**, which reads data **one byte at a time**. First, the program records the start time using System.currentTimeMillis(), then it continuously reads bytes from the file until read() returns **-1**, indicating the end of the file. After the file is fully read, it records the end time and calculates the total time taken by subtracting the start time from the end time. Finally, the program prints the time taken in seconds.

What is a Buffer?

A **buffer** is a temporary memory area used to store data before it is processed or transferred. In Java file handling, a buffer helps in **reading or writing data faster** by reducing the number of direct interactions with the file system.

BufferedReader

```
public class BufferedReadDemo {  
    public static void main(String[] args) throws Exception {  
        String srcPath="/Users/selvam-8490/Desktop/EclipseProjects/GS20252/src/p  
        FileInputStream fis=new FileInputStream(srcPath);  
        BufferedInputStream bis=new BufferedInputStream(fis);  
        long t1=System.currentTimeMillis();  
        int value=bis.read();  
        while(value!=-1) {  
            value=bis.read();  
        }  
        long t2=System.currentTimeMillis();  
        fis.close();  
        bis.close();  
        System.out.println("Time Taken=" + (t2-t1)/(float)1000 + "seconds");  
    }  
}
```

Out Put : Time Taken = 0.099 Seconds

Difference Between FileInputStream & BufferedInputStream

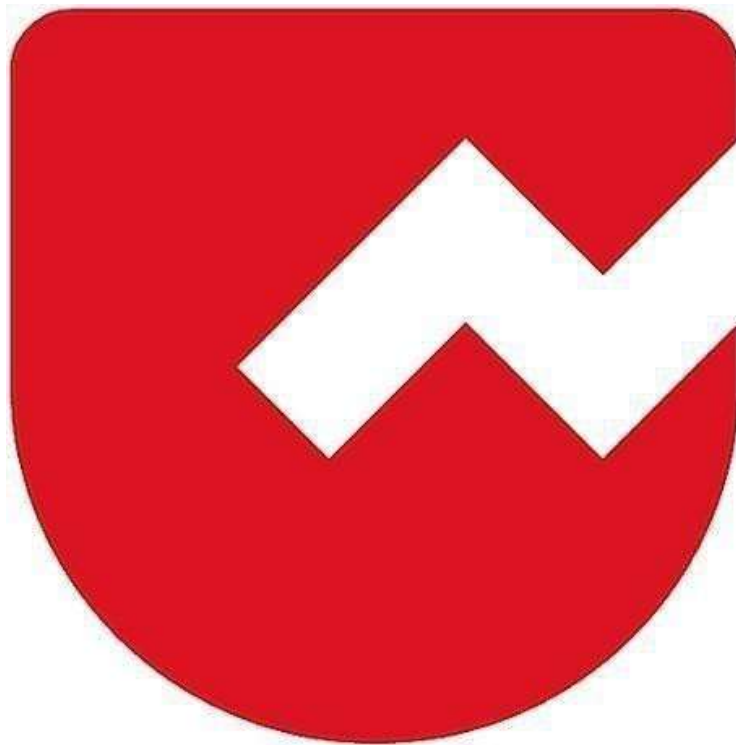
Feature	FileInputStream	BufferInputStream
Stream Used	FileInputStream	BufferedInputStream (with FileInputStream)
Reads From	From internal buffer	From internal buffer
Speed	Every byte requires disk I/O	Disk accessed rarely
Efficiency	Low	High
Best For	Small files	Large files

Task

//Develop a Java program to read line by line and write line by line|

```
public class ReadLineDemo {  
  
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
  
    }  
  
}
```


Zoho Schools for Graduate Studies



Notes

Day-34

Java Program to read line by line and write line by line

```
import java.io.*;

public class ReadLineDemo {

    public static void main(String[] args) throws Exception {

        String srcPath = "input.txt";

        FileReader fr = new FileReader(srcPath);

        BufferedReader br = new BufferedReader(fr);


        String destPath = "output.txt";

        FileOutputStream fos = new FileOutputStream(destPath);

        PrintWriter pw = new PrintWriter(fos);


        String value = br.readLine();

        while (value != null) {

            pw.println(value);

            value = br.readLine();

        }

        fr.close();

        br.close();

        fos.close();

        pw.close();

        System.out.println("File content are copied line by line.");

        Runtime rt = Runtime.getRuntime();

        String command = "open " + srcPath;

        Process p = rt.exec(command);

    }

}
```

FileReader:

- FileReader is a high-level character input stream used to read characters from a file. It reads data character by character.
- Input: A file path (String) or a File object.
- FileReader is used here to open the source file for reading.

BufferedReader:

- BufferedReader wraps around a Reader (like FileReader) and provides buffering, which makes reading efficient. It also provides the readLine() method to read one line at a time.
- Input: A Reader object (FileReader).
- readLine() → reads a line of text
- read() → reads a single character

PrintWriter:

- PrintWriter is a high-level character output stream used to write text to files. It provides convenient methods like print() and println().
- Input: OutputStream like FileOutputStream.
- print(String s) → writes text without newline.
- println(String s) → writes text with newline.
- close() → flushes data and closes the stream.

Runtime:

Runtime class in the java.lang package. The Runtime class represents the Java application's runtime environment. Every Java program has one instance of Runtime, which can be obtained using the method

```
Runtime rt = Runtime.getRuntime();
```

The Runtime object allows the program to perform system-level operations, such as executing external commands. Runtime acts as a bridge between your Java program and the operating system. You cannot create multiple Runtime objects; you always use the single instance provided by `getRuntime()`.

exec():

- The `exec()` method is a **member of the Runtime class**. It is used to **run system commands** from within a Java program.
- You pass a **command string** to `exec()`, for example "open input.txt" (on Mac) or "notepad input.txt" (on Windows).
- The command is executed by the operating system.
- The method returns a **Process object** representing the program or command that is running.

```
Process p = rt.exec("open input.txt");
```

Java Program to play a video or audio file

```
import java.io.*;

public class PlayVideoFileDemo {

    public static void main(String[] args) throws Exception {

        String srcPath = "path";

        Runtime rt = Runtime.getRuntime();

        String command = "open " + srcPath; // for macOS

        Process p = rt.exec(command);

    }

}
```

Java Program to get input from multiple streams

```
import java.io.*;

public class SequenceInputDemo {

    public static void main(String[] args) throws Exception {

        byte[] b = "It was all yellow".getBytes();

        ByteArrayInputStream bis = new ByteArrayInputStream(b);

        SequenceInputStream sis = new SequenceInputStream(System.in, bis);

        String destPath = "output.txt";

        FileOutputStream fos = new FileOutputStream(destPath);

        int value = sis.read();

        while (value != -1) {

            fos.write(value);

        }

    }

}
```

```
        value = sis.read();
    }

    bis.close();
    sis.close();
    fos.close();

    System.out.println("Keyboard and byte array content are copied.");

    Runtime rt = Runtime.getRuntime();
    String command = "open " + destPath;
    Process p = rt.exec(command);
}
}
```

SequenceInputStream:

SequenceInputStream is a special input stream that joins two or more InputStreams together and makes them behave like one continuous stream.

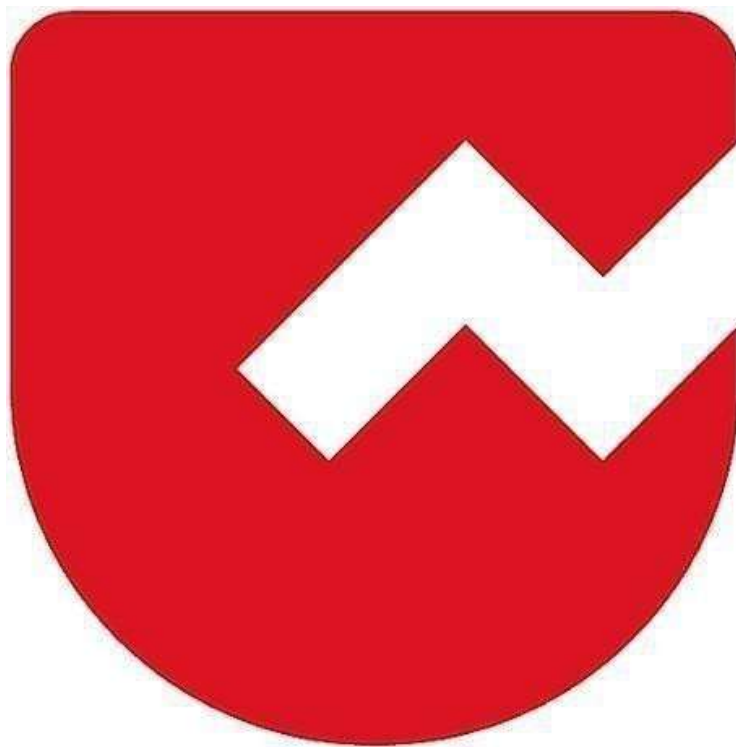
It reads all data from the first stream, and when it finishes, it automatically continues reading from the second stream.

Serialization:

Serialization in Java is the process of converting an object into a byte stream so it can be stored in a file or transferred across a network. To enable this, the class must implement the `Serializable` interface. Java provides `ObjectOutputStream` to write the object and `ObjectInputStream` to read it back during deserialization. This allows you to save and restore the complete state of an object.



Zoho Schools for Graduate Studies



Notes

Day-35

Multithreading

Threads:

- A **thread** is a lightweight sub-process that represents an independent path of execution within a program.
- In Java, every program has **at least one thread**, called the **main thread**, which starts automatically when the program begins execution.
- Threads allow multiple tasks to run **concurrently**, improving performance and responsiveness.

Types of Threads:

Main Thread (Default Thread)

- Created automatically when the Java program starts.
- Responsible for executing the `main ()` method.

Child Threads (User-defined threads)

- Created by the programmer to run tasks parallelly with the main thread.

Defining a Thread in Java:

A thread is defined by overriding the `run ()` method.

`run()` Method

- Contains the logic that the thread will execute.
- When a thread starts, only the `run ()` method code runs.

How to create a Thread in Java:

There are two ways:

1. Extending the Thread class

2. Implementing the Runnable interface

Method 1 – Extending the Thread Class:

Steps

1. Create a class that extends Thread.
2. Override the run () method.
3. Create an object of your class.
4. Call the start () method → this internally calls run () .

Important Notes

- You must use **start()** (NOT run()) to start a new thread.
- A thread **executes only once**. Restarting a thread causes:
→ **IllegalThreadStateException**.



```
1 package part4;
2
3 public class MyThread extends Thread{
4     public void run() {
5         System.out.println(Thread.currentThread().getName());
6     }
7
8 }
9
```



```
1 package part4;
2
3 public class ThreadDemo {
4     public static void main(String[] args) {
5         MyThread t=new MyThread();
6         t.start();
7
8     }
9
10 }
```

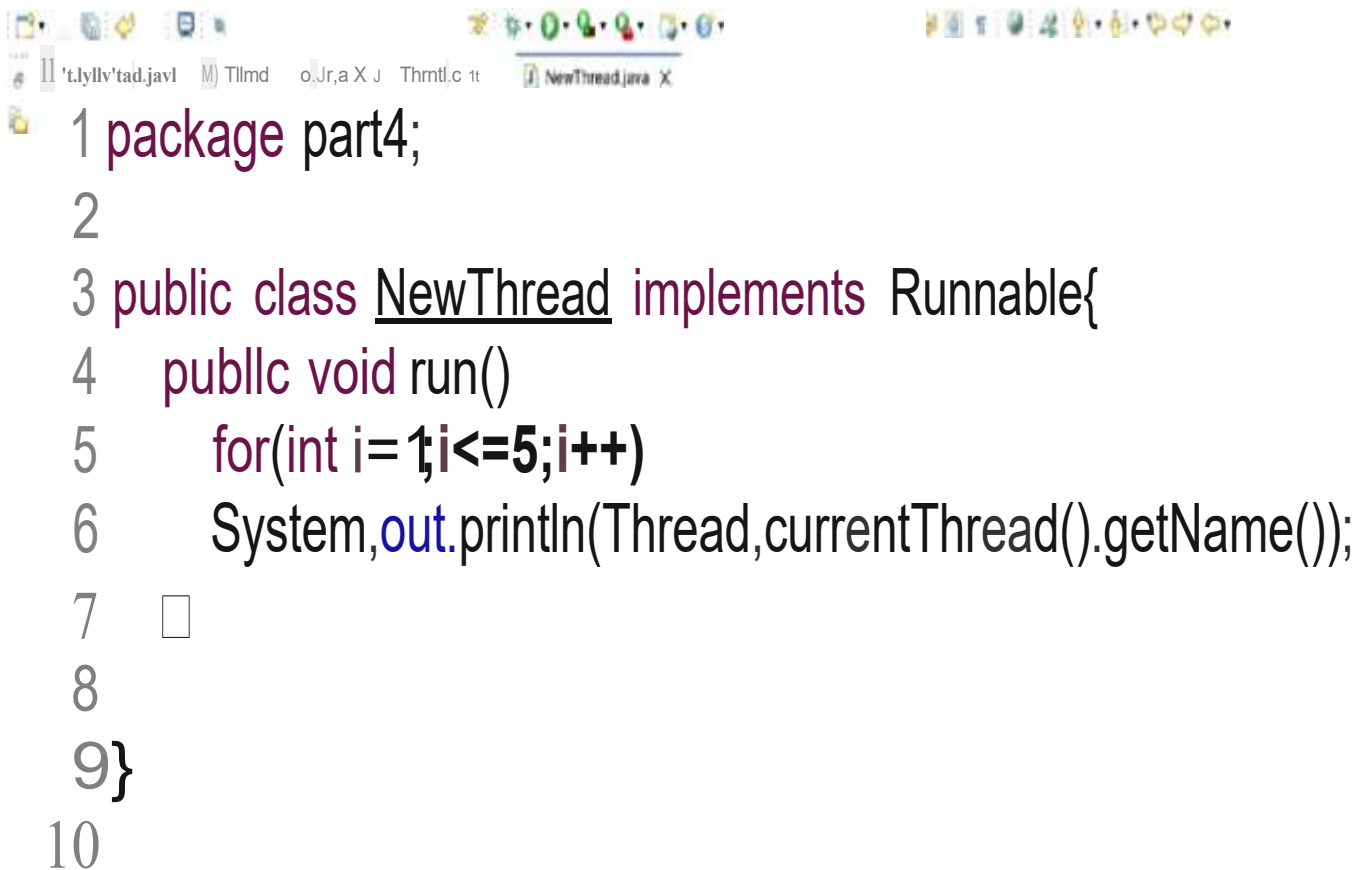
Method 2 – Implementing Runnable Interface:

Why Runnable is better?

- Java supports multiple inheritance only through interfaces.
- To extend another class and still create a thread → Runnable is recommended.
- More flexible for large applications.

Steps

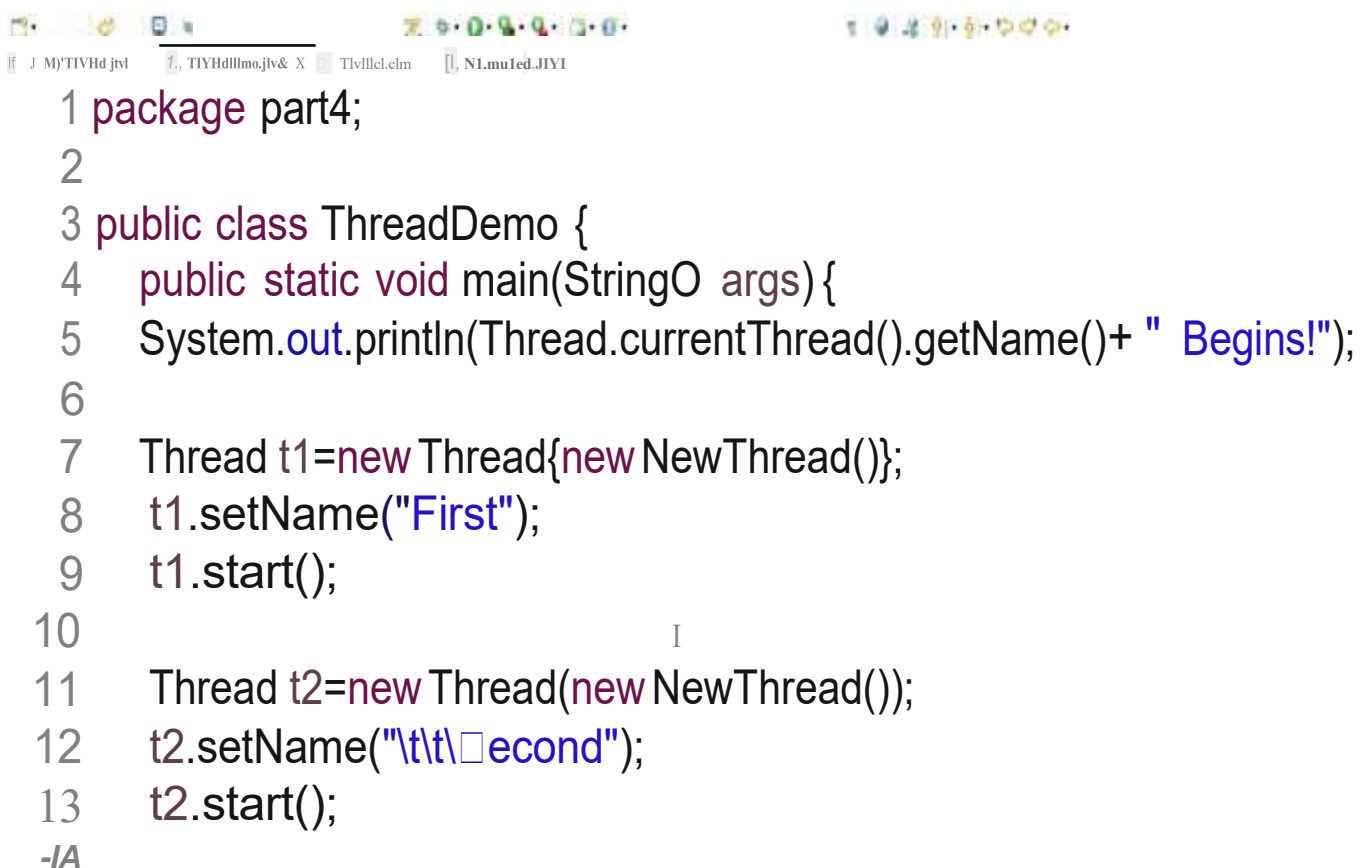
1. Create a class that implements Runnable.
2. Override run () .
3. Create a Thread object and pass your Runnable object to it.
4. Call Start () .



```

1 package part4;
2
3 public class NewThread implements Runnable{
4     public void run()
5         for(int i=1;i<=5;i++)
6         System.out.println(Thread.currentThread().getName());
7
8
9 }
10

```



```

1 package part4;
2
3 public class ThreadDemo {
4     public static void main(StringO args){
5         System.out.println(Thread.currentThread().getName()+ " Begins!");
6
7         Thread t1=new Thread{new NewThread()};
8         t1.setName("First");
9         t1.start();
10
11         Thread t2=new Thread(new NewThread());
12         t2.setName("\t\tSecond");
13         t2.start();
14
15     }
16 }
17
18 -IA

```

Thread Scheduler:

The Thread Scheduler in Java is responsible for deciding the execution order of threads. It uses algorithms like preemptive scheduling and time slicing to determine which thread gets CPU time. Thread execution order is never guaranteed.

Preemptive Scheduling:

Forcible deallocation of a resource from a thread is termed as Preemption

Non-Preemptive:

Thread – A piece of code which has its own path of execution

Sequential Execution:

Tasks run one after another, in a fixed order. Only one task is running at any moment.

Concurrent Execution:

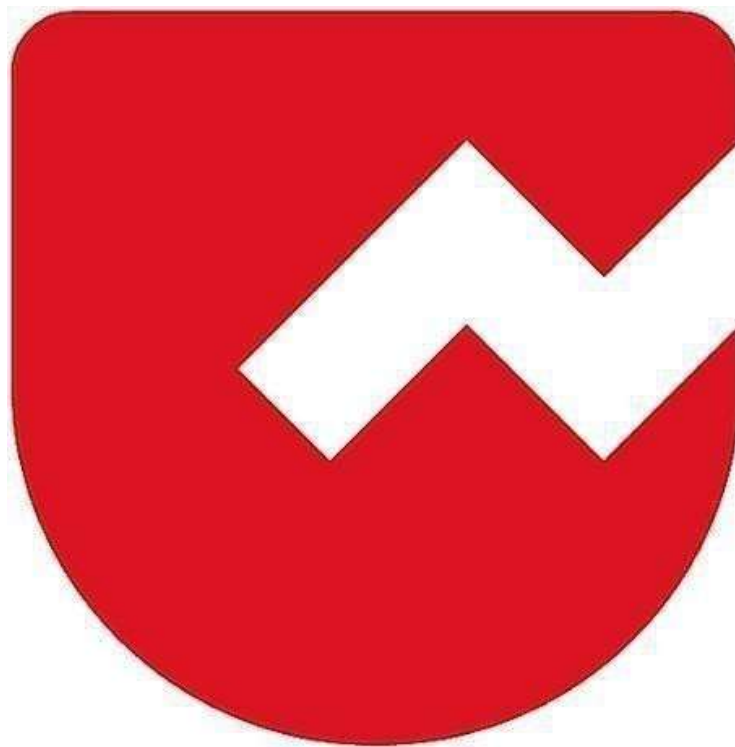
Multiple tasks overlap in time. Not necessarily at the exact same instant but it looks like they run together.

Parallel Execution:

Multiple tasks run at the exact same time, using multiple CPU cores. So actual simultaneous execution happens.



Zoho Schools for Graduate Studies



Notes

Day-36

Multithreading

Multithreading is inspired from multitasking of humans.

Multitasking

- Process based
 - two or more programs running concurrently
 - e.g., Typing in an editor or browsing and hearing songs
- Thread based
 - Within a program, tasks run concurrently
 - e.g., Navigating through a web page in one tab and downloading a file from another tab in a web browser

Two ways of implementation in JAVA

1. By extending Thread class
2. By implementing Runnable interface

By extending Thread class

```
class MyFirstThread extends Thread {  
    public void run() {  
        for(int i = 0; i < 5; i++) {  
            System.out.println("Vanakkam ZOHO");  
        }  
    }  
}
```


By extending Thread class

```
public class MyThreadProgram {  
    public static void main(String args[]) {  
        MyFirstThread threadObject = new MyFirstThread();  
        threadObject.start();  
    }  
}
```

By extending Thread class

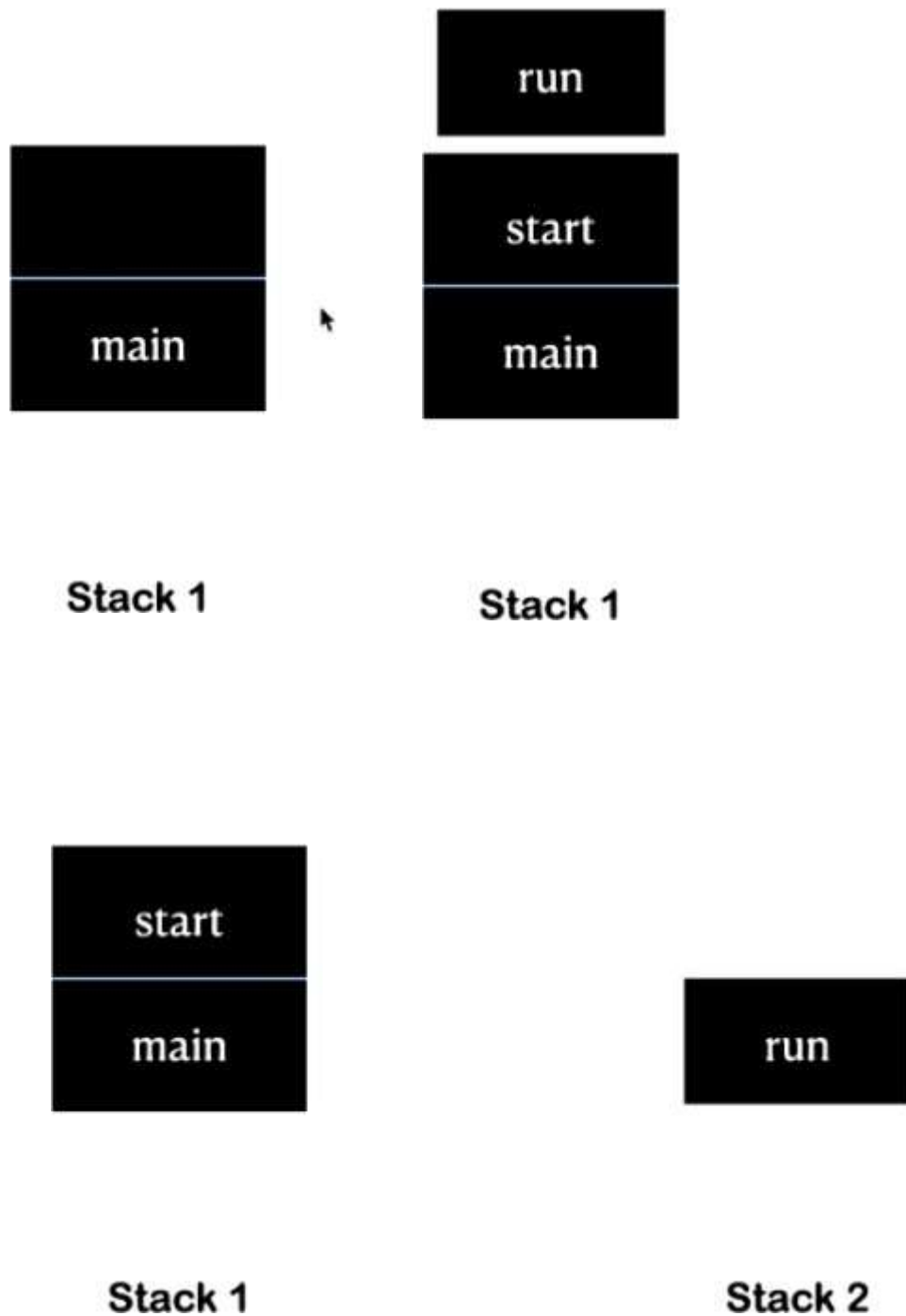
```
public class MyThreadProgram {  
    public static void main(String args[]) {  
        MyFirstThread threadObject = new MyFirstThread();  
        threadObject.start();  
        Thread threadObject = new Thread();  
    }  
}
```

Things to remember

1. Runtime polymorphism

- `run()` is overridden from `Thread` class
- `start()` method is defined in `Thread` class

2. Function call stack



Method - 2

By implementing Runnable interface

```
public class MyRunnableImplementation implements Runnable {  
  
    public void run(){  
        System.out.println("Vanakkam from a runnable thread!");  
    }  
  
}
```

By implementing Runnable interface

```
public class MyRunnableThreadProgram {  
  
    public static void main(String args[]) {  
        Runnable runnableObject = new MyRunnableImplementation();  
    }  
  
}
```

By implementing Runnable interface

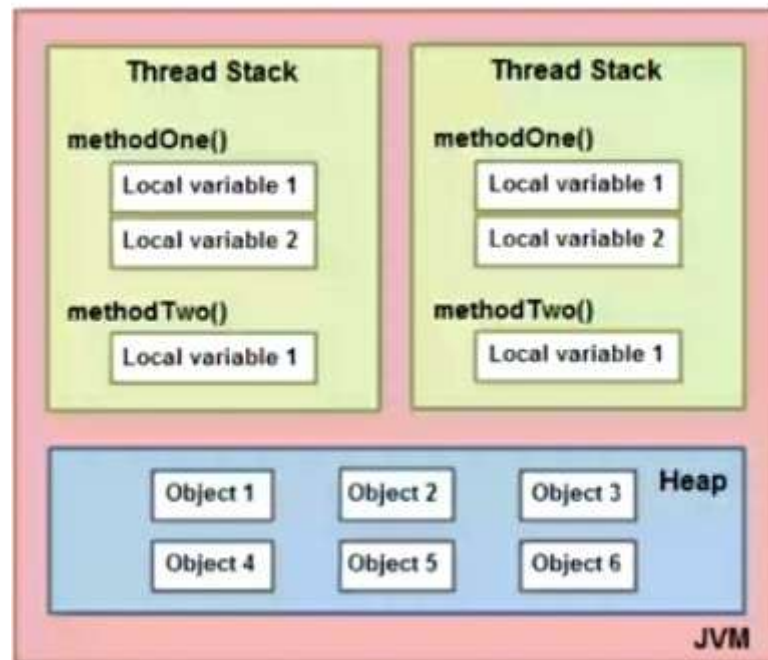
```
public class MyRunnableThreadProgram {  
  
    public static void main(String args[]) {  
        Runnable runnableObject = new MyRunnableImplementation();  
        nmnableObject.st-ert();  
        nmnableObject.run();  
    }  
}
```

By implementing Runnable interface

```
public class MyRunnableThreadProgram {  
  
    public static void main(String args[]) {  
        Runnable runnableObject = new MyRunnableImplementation();  
        Thread threadObject = new Thread(runnableObject);  
        threadObject.start();  
    }  
}
```

Constructor used: Thread(Runnable rObject)
--

Java Memory Model



Heap - dynamically allocated - OutOfMemoryException

Threads:

- A **thread** is a lightweight sub-process that represents an independent path of execution within a program.
- In Java, every program has **at least one thread**, called the **main thread**, which starts automatically when the program begins execution.
- Threads allow multiple tasks to run **concurrently**, improving performance and responsiveness.

Types of Threads:

Main Thread (Default Thread)

- Created automatically when the Java program starts.
- Responsible for executing the main() method.

Child Threads (User-defined threads)

- Created by the programmer to run tasks parallelly with the main thread.

Defining a Thread in Java:

A thread is defined by overriding the run () method.

run() Method

- Contains the logic that the thread will execute.
- When a thread starts, only the run () method code runs.

How to create a Thread in Java:

There are two ways:

1. Extending the Thread class
2. Implementing the Runnable interface

Method 1 – Extending the Thread Class:

Steps

1. Create a class that extends Thread.
2. Override the run() method.
3. Create an object of your class.
4. Call the start() method → this internally calls run().

Important Notes

- You must use **start()** (NOT run()) to start a new thread.
- A thread **executes only once**. Restarting a thread causes:
→ **IllegalThreadStateException**.

Method 2 – Implementing Runnable Interface:

Why Runnable is better?

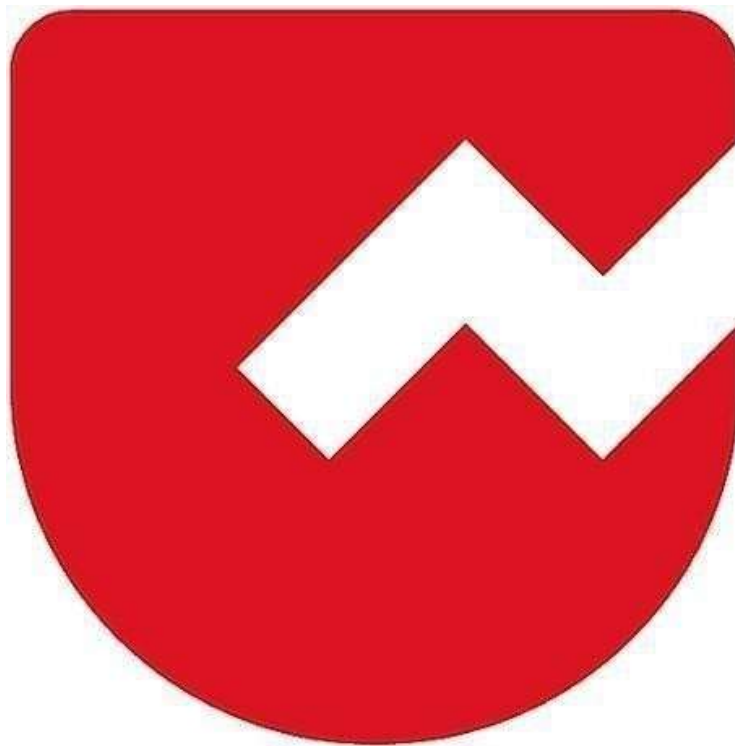
- Java supports multiple inheritance only through interfaces.
- To extend another class and still create a thread → Runnable is recommended.
- More flexible for large applications.

Steps

1. Create a class that implements Runnable.
2. Override run().
3. Create a Thread object and pass your Runnable object to it.
4. Call Start().



Zoho Schools for Graduate Studies



Notes

Day-37

Lambda Expression

- Introduced in **Java version 8**.
- It is used to concisely represent an **anonymous** function.
- Works with **functional interfaces** (interfaces with exactly one abstract method).

Method vs Lambda Conversion:

Normal Method:

```
int add(int a, int b) {  
    return (a + b);  
}
```

Lambda Expression:

```
(int a, int b) -> {  
    return (a + b);  
}
```

Shorter form:

//No need of declare the datatype in the parameter.

```
(a, b) -> {return (a + b)};
```

Lambda Structure Explanation:

Left side -> parameter list (you need not specify datatype)

Right side -> body

Examples:

```
1. (a, b) -> {  
    return (a + b);  
};
```

```
2. (a, b) -> System.out.println(a + b);
```

//If no return statement → no need curly braces (single line).

```

        //If only one statement → braces {} optional.

3. (a) -> System.out.println(a);

4. a -> System.out.println(a);

        // if parameter is single, parentheses not required

5. () -> System.out.println("ZOHO");

        // if no parameter, parentheses mandatory

```

Method Representation:

Normal Method:

```

public static void func(int nums1, int nums2) {
    return nums1 + nums2;
}

```

Lambda:

```

(nums1, nums2) -> { return (nums1 + nums2); };

```

Application of Lambda Expression:

1. To implement a Functional Interface:

Functional Interface (SAM) → (Single Abstract Method):

- An interface that exactly has only one abstract method.
- It has java.util.function.

Examples:

1. Runnable -> run();
2. Comparator -> compare(T o1, T o2);
3. Comparable -> compareTo(T o);

Functional Interface Annotation:

→**@FunctionalInterface**

- Annotation.
- Compiler checks the method extra care.
- (If another method is added into the functional Interface class, at once it shows the error)

Functional Interface & Lambda – Example:

@FunctionalInterface

```
public interface AddInterface {  
    void add(int a, int b);  
}
```

```
public class Demo implements AddInterface {  
    public static void main(String[] args) {  
        Demo d = new Demo();  
        d.add(23, 25);  
    }  
    public void add(int a, int b) {  
        System.out.println(a + b);  
    }  
}
```

Another type without implements:

```
2
3 public class LambdaExpressionDemo //implements AddInterface
4 {
5     public static void main(String[] args) {
6         //LambdaExpressionDemo l=new LambdaExpressionDemo();
7         AddInterface ai=(a, b)->System.out.println(a+b);
8         ai.add(23,45);
9     }
10 }
11
12 /*public void add(int a, int b){
13     System.out.println(a+b);
14 }*/
15 }
16
```

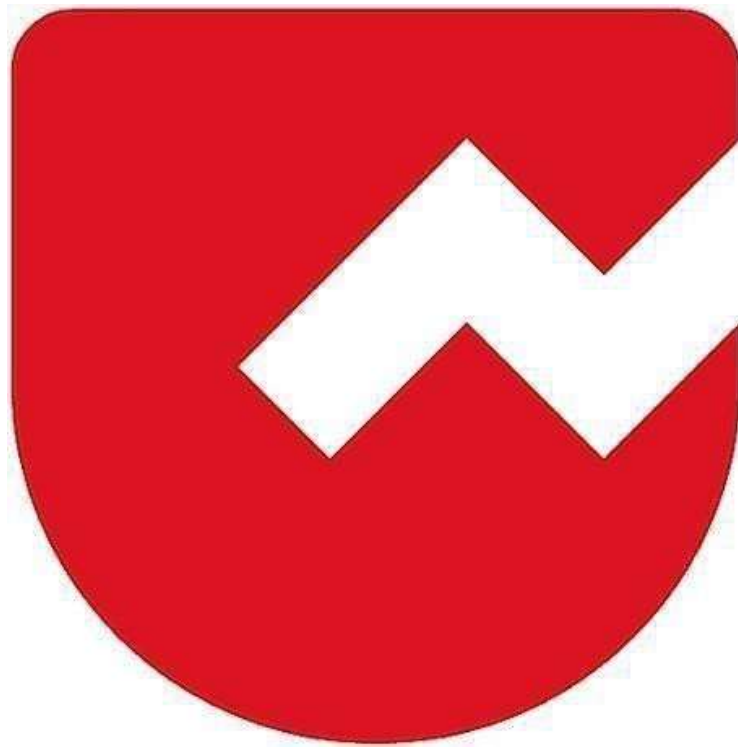
Another example in the Threads:

```
2
3 public class ThreadDemo {
4     public static void main(String[] args) {
5         System.out.println(Thread.currentThread().getName() + " Begins!");
6         Thread.currentThread().setPriority(9);
7
8         Runnable r = ()->{
9             for(int i=1;i<=5;i++)
10                 System.out.println(Thread.currentThread().getName());
11             };
12
13         Thread t1=new Thread(r);
14         t1.setName("First");
15         System.out.println(t1.getPriority());
16         t1.start();
17     }
18 }
```

- No need to write the new Thread() class that implements runnable interface.
- Directly we can write the runnable with the help of Lambda Expression.

Zoho Schools for Graduate Studies

Notes



Day-38

Synchronization in Multithreading:

- ⑩ Synchronization means controlling access to shared resources so that only one thread can use it at a time.
- ⑩ It prevents data inconsistency and race conditions.

Synchronization using join() :

- ⑩ join() is **not locking a resource** like synchronized.
- ⑩ join() synchronizes threads based on execution order.
- ⑩ It makes **one thread wait until another thread finishes**.
- ⑩ synchronized → **controls shared data**.
- ⑩ join() → **controls thread sequence**.

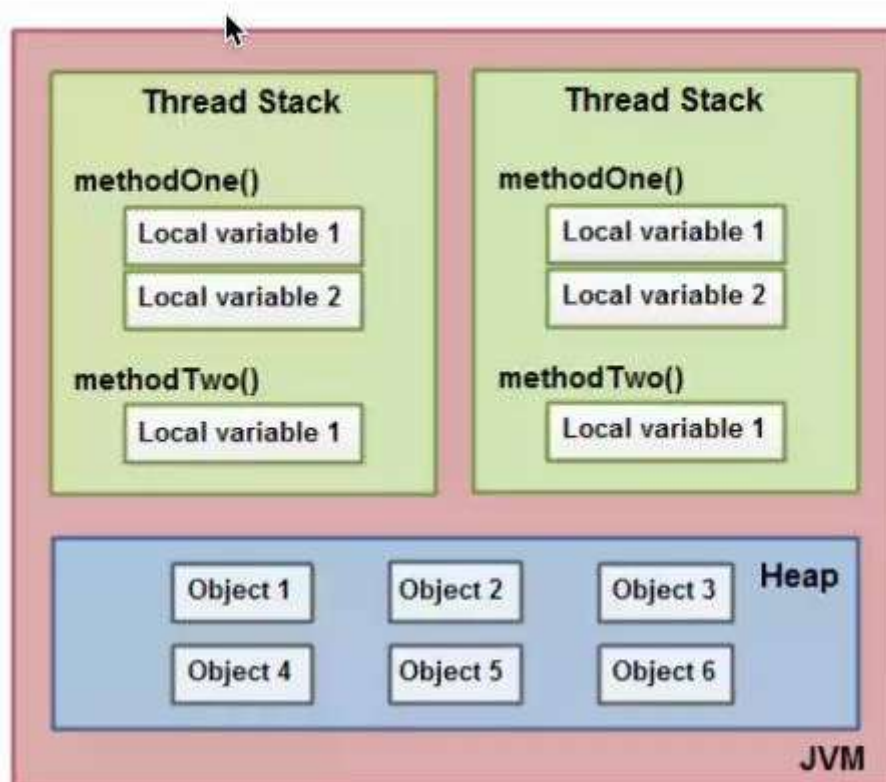
**<threadInstance>.join blocks the calling thread until the
<threadInstance> has finished**

Join supports .interrupts too

Java Memory Model (JMM):

- ⑩ **How memory is structured**
- ⑩ **How threads access memory**
- ⑩ **When changes made by one thread are visible to other threads**

Java Memory Model



- ⑩ Order of execution can be rearranged by compiler for optimisation .This will cause issue when shared variables are used inside threads.

Why did the problem occur?

Story behind shorthand increment/decrement //CPU instructions.

Read a, r1

Add r1, 1

Store r1, a

Synchronization by Mutual Exclusion:

Synchronized methods

Synchronized blocks //multiple lock not allowed

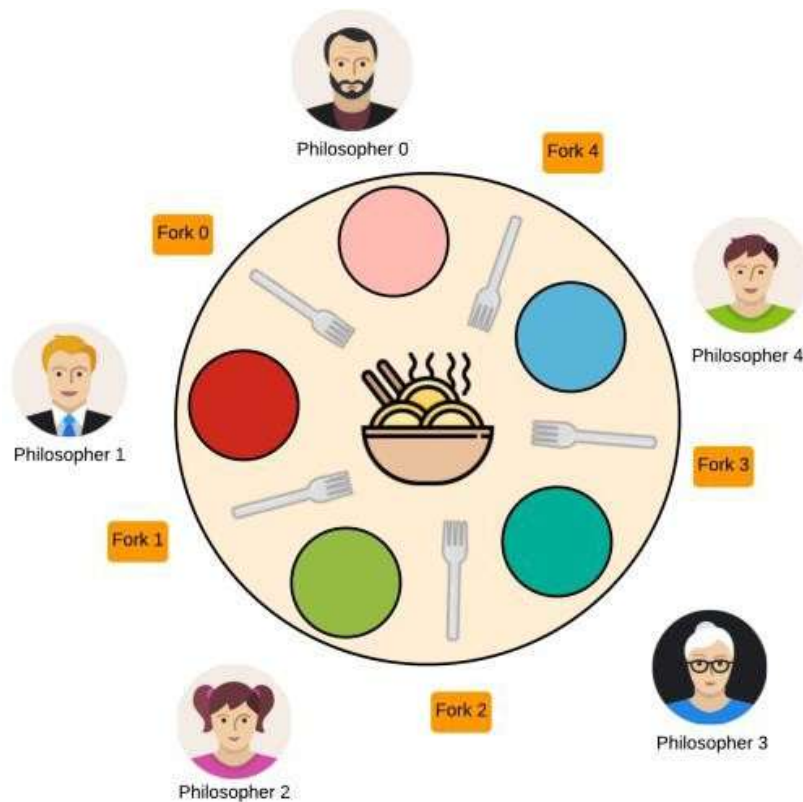
Volatile:

It ensures value of the variable is always read from and written to the main memory,

rather than from the CPU cache of individual threads

Note: Volatile cannot be used for methods and classes

Dining Philosopher's Problem:



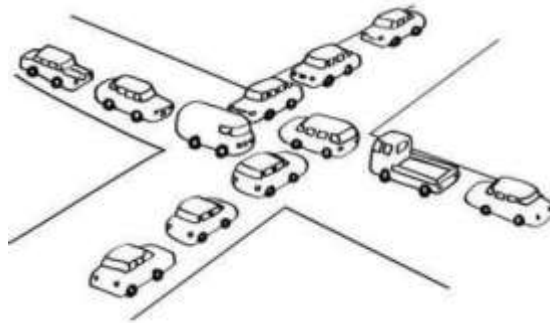
Deadlock:

Note: When synchronization goes wrong

Deadlock is a situation in multithreading where two or more threads are permanently blocked, because each thread is waiting for a resource held by another thread.

Pic 1:

pic 2:



class



```
DeadlockDemo {  
    static Object lock1 = new Object();  
    static Object lock2 = new Object();  
  
    public static void main(String[] args) {  
  
        Thread t1 = new Thread() -> {  
            synchronized (lock1) {  
                System.out.println("Thread 1 locked lock1");  
            }  
        }  
    }  
}
```

```

        try { Thread.sleep(100); } catch (Exception e) {}

        synchronized (lock2) {
            System.out.println("Thread 1 locked lock2");
        }
    }
});

```

```

Thread t2 = new Thread() -> {
    synchronized (lock2) {
        System.out.println("Thread 2 locked lock2");
        try { Thread.sleep(100); } catch (Exception e) {}
    }

    synchronized (lock1) {
        System.out.println("Thread 2 locked lock1");
    }
}
});

```

```

t1.start();
t2.start();
}
}

```

Other things to explore:

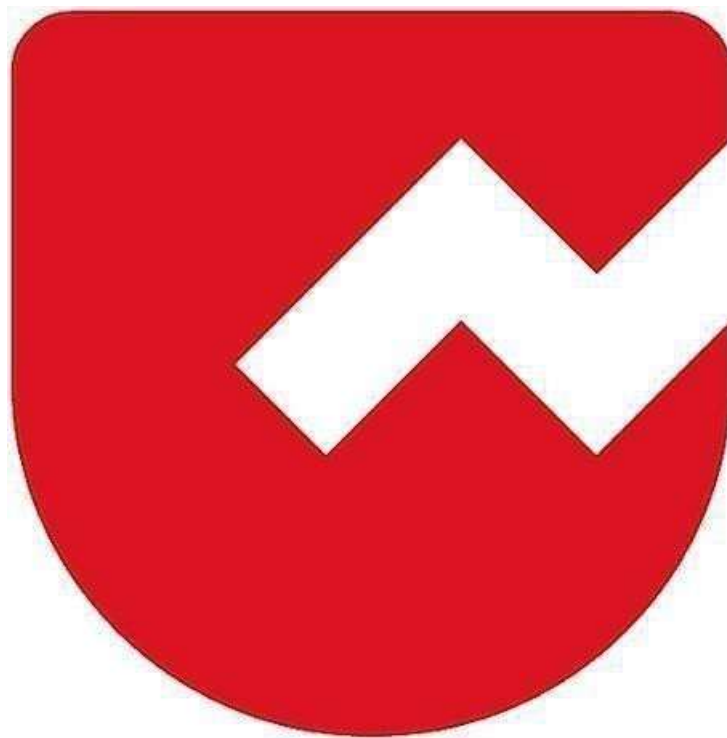
- ⑩ ReentrantLock

⑩ java.util.concurrent package

⑩ Collections.<synchronized methods>

Zoho Schools for Graduate Studies

Notes



Day-39

Inner Class

An Inner Class in Java is a class defined inside another class.

The class inside is called the inner class, and the outer one is called the outer class.

Example

```
class Outer {  
  
    class Inner {  
        // inner class code  
    }  
}
```

Key Points:

- Inner class is tightly connected to the outer class
- It can access:
 - Outer class variables
 - Outer class methods
- Even private members of outer class
- Inner class cannot exist independently
- Object of inner class depends on object of outer class

- Inner class can access private data of outer class, but outside classes cannot access inner class directly.

USE CASES

- Inner classes are used in Java for better design, security, and readability.
- Inner class can access private data of outer class, but outside classes cannot access inner class directly.(ENCAPULATION)
- Improves Security
- Inner class is hidden
- Cannot be accessed without outer class
- Reduces misuse of helper classes

Types of inner class

- Member Inner Class
- Static Inner Class
- Local Inner Class
- Anonymous Inner Class

Member Inner Class

- A class defined inside another class
- It is non-static
- Needs outer class object to create inner class object
- Can access all members of outer class (including private)
- Used when inner class is closely related to outer class

Example

```
class Outer {  
    class Inner {  
        void show() {  
            System.out.println("Hello Inner");  
        }  
    }  
}
```

```
public class Test {  
    public static void main(String[] args) {  
        Outer o = new Outer();           // outer object  
        Outer.Inner i = o.new Inner();    // inner object  
        i.show();  
    }  
}
```



```
public class Test {  
    public static void main(String[] args) {  
        Outer.Inner i = new Outer().new Inner();  
        i.show();  
    }  
}
```

Example 2

```
class Outer {  
    void display() {  
        class Inner {  
            void msg() {  
                System.out.println("Local Inner Class");  
            }  
        }  
        Inner i = new Inner();  
        i.msg();  
    }  
}
```

```
public class Test {  
    public static void main(String[] args) {  
        Outer o = new Outer();  
        o.display();  
    }  
}
```

Static Inner Class

- A class defined inside another class using static
- Does not need outer class object
- Can access only static members of outer class
- Behaves like a normal class but grouped inside another class
- Used when inner logic is related but independent

```
class Outer {
    static class Inner {
        void show() {
            System.out.println("Static Inner");
        }
    }
}
```

```
public class Test {
    public static void main(String[] args) {
        Outer.Inner i = new Outer.Inner(); // no outer object
        i.show();
    }
}
```

Anonymous Inner Class

- Inner class without a name

- Created for one-time use
- Mostly used with interfaces or abstract classes
- Reduces code length
- Common in event handling and threads

```
interface A {  
    void show();  
}
```

```
public class Test {  
    public static void main(String[] args) {  
        A obj = new A() {  
            public void show() {  
                System.out.println("Anonymous Inner Class");  
            }  
        };  
        obj.show();  
    }  
}
```

Lazy Loading in Inner Classes

- Outer class loads first
- Inner class loads only when it is used
- If inner class is never used → it is never loaded

- Saves memory
- Faster program startup
- Loads only required classes
- Important for large applications

Example

```
class Outer {  
  
    static {  
        System.out.println("Outer class loaded");  
    }  
  
    class Inner {  
        static {  
            System.out.println("Inner class loaded");  
        }  
  
        void show() {  
            System.out.println("Inner method called");  
        }  
    }  
}
```

Main Class (Case 1: Inner NOT used)

```
public class Test {  
    public static void main(String[] args) {  
        Outer o = new Outer();  
        System.out.println("Main ends");  
    }  
}
```

```
}  
}
```

Output

Outer class loaded
Main ends

Inner class NOT loaded
Because it was never used

Main Class (Case 2: Inner USED)

```
public class Test {  
    public static void main(String[] args) {  
        Outer o = new Outer();  
        Outer.Inner i = o.new Inner();  
        i.show();  
    }  
}
```

Output

Outer class loaded
Inner class loaded
Inner method called

Inner class loads only when object is created
This is Lazy Loading

Compilation and runtime

At Compilation Time

Java compiler creates:

- Outer.class
- Outer\$Inner.class

Both .class files are created

At Runtime (JVM Execution)

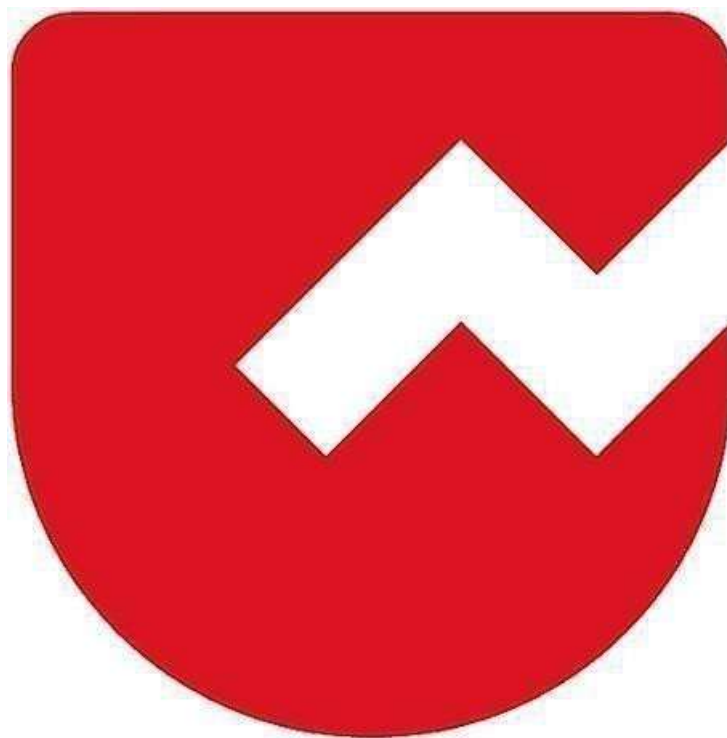
- JVM does NOT load all .class files

JVM loads:

- Only classes that are used
- Inner class .class file exists but JVM loads it only when required

Zoho Schools for Graduate Studies

Notes



Day-40

Computation of Running Time

Time complexity (Big-O) — quick recap

Algorithm	Time Complexity
For loop	$O(n)$
Nested loops	$O(n^2)$
Binary search	$O(\log n)$
Merge sort / quicksort avg	$O(n \log n)$
Traversing array once	$O(n)$
HashMap get/put (avg)	$O(1)$

Time Complexity

- **For loop – $O(n)$**

If you go through items one by one, the time grows directly with the number of items.

- **Nested loops – $O(n^2)$**

A loop inside another loop. For every item, you again go through all items, so work grows much faster (square of n).

- **Binary search – $O(\log n)$**

Each step cuts the list in half (like guessing a number game by halving the range). Very fast even for big lists.

- **Merge sort / Quicksort average – $O(n \log n)$**

You both divide the data ($\log n$ part) and process all elements (n part). Faster than n^2 , slower than n .

- **Traversing array once – $O(n)$**

You look at each element only one time from start to end.

- **HashMap get/put (average) – $O(1)$**

You can directly jump to the needed item without searching through others. Time is constant, doesn't depend on size (on average).

Counting number of operations

1. only constant amount of work $O(1)$

(i)

```

1 package part4;
2
3 public class RunningTimeDemo {
4     public static void main(String[] args) {
5         System.out.println(sumToN(20));
6     }
7 }
8 static int sumToN(int n)
9 {
10     return (n*(n+1))/2;
11 }
12
13 }
14

```

Expression:

$$(n * (n + 1)) / 2$$

Count basic operations:

1. $n + 1 \rightarrow$ **1 operation**
2. $n * (\text{result}) \rightarrow$ **1 operation**
3. division by 2 $/ 2 \rightarrow$ **1 operation**
4. final return (sometimes counted as constant) $\rightarrow$ **1 operation**

Total operations

= 3 arithmetic operations
 + 1 return
 = 4 operations (constant number)

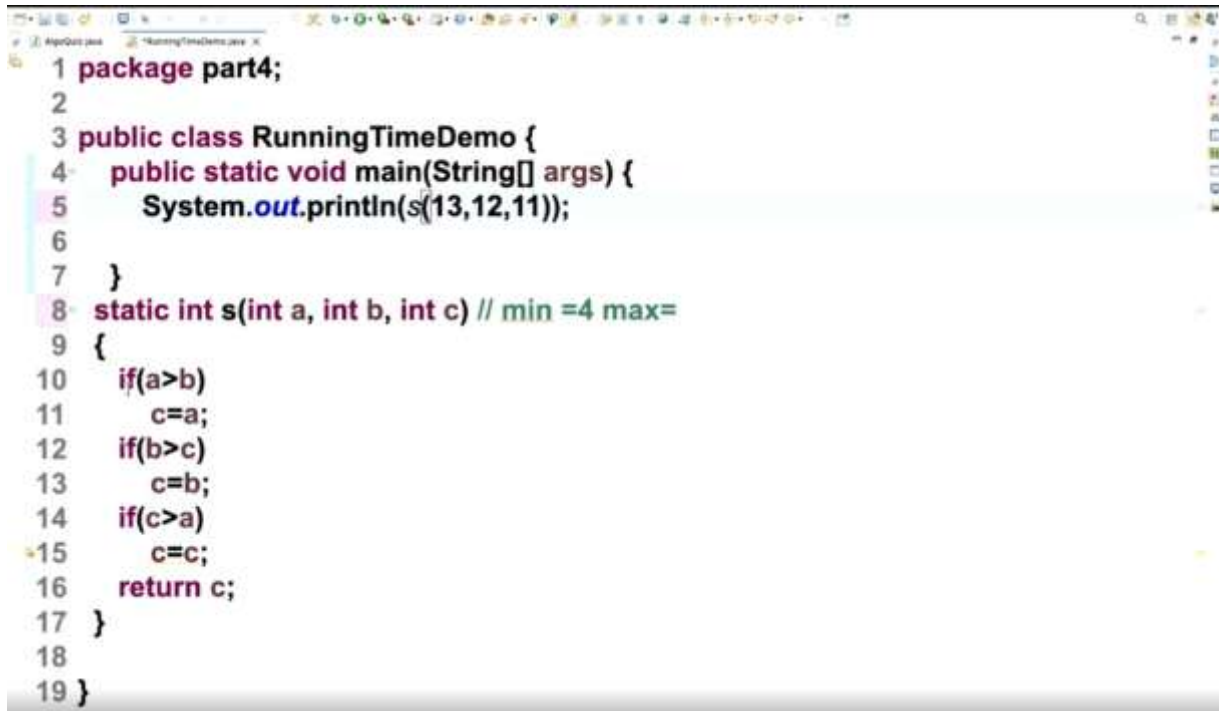
Even if you call it for $n = 10, 1000, 10^6, 10^9$,
 operations are **still 4** — it doesn't change.

Time Complexity

Since the number of operations **does not depend on n**:

Time Complexity = $O(1)$ *//(constant time)*

(ii)



```
1 package part4;
2
3 public class RunningTimeDemo {
4     public static void main(String[] args) {
5         System.out.println(s(13,12,11));
6     }
7 }
8 static int s(int a, int b, int c) // min =4 max=
9 {
10     if(a>b)
11         c=a;
12     if(b>c)
13         c=b;
14     if(c>a)
15         c=c;
16     return c;
17 }
18
19 }
```

We count:

- each **comparison** as 1 operation
- each **assignment** as 1 operation
- return as 1 operation

Expression:

a = 13, b = 12, c = 11

Step-by-step:

1. $a > b \rightarrow 13 > 12 \rightarrow \mathbf{true} \rightarrow$ assignment executes
comparison = 1
assignment $c = a = 1$
Now $c = 13$

2. $b > c \rightarrow 12 > 13 \rightarrow \mathbf{false}$
comparison = 1

3. $c > a \rightarrow 13 > 13 \rightarrow \mathbf{false}$
comparison = 1

4. return c
return = 1

Total operations executed

comparisons: 3
assignments: 1
return: 1
TOTAL = 5 operations

This is **neither** minimum (4) nor maximum (7);

(iii)

```

1 package part4;
2
3 public class RunningTimeDemo {
4     public static void main(String[] args) {
5         System.out.println(s(10,12,1));
6     }
7 }
8 static int s(int a, int b, int c) // min =4 max=
9 {
10     if(a>b)
11         c=a;//c=13
12     if(b>c)
13         c=b;
14     if(c>a)
15         c=c;
16     return c;
17 }
18
19 }

```

Expression:

a = 10, b = 12, c = 1

Step-by-step:

1. $a > b \rightarrow 10 > 12 \rightarrow \text{false}$
comparison = 1

2. $b > c \rightarrow 12 > 1 \rightarrow \text{true} \rightarrow$ assignment executes
comparison = 1
assignment $c = b = 1$
Now c = 12

3. $c > a \rightarrow 12 > 10 \rightarrow \text{true} \rightarrow$ assignment executes
comparison = 1
assignment $c = c = 1$

4. return c
return = 1

Total operations executed

comparisons: 3
assignments: 2
return: 1
TOTAL = 6 operations

This is **close to max** but not full 7 (because first if failed).

Summary Table

Input	Comparisons	Assignments	Return	Total
s(13,12,11)	3	1	1	5
s(10,12,1)	3	2	1	6

MINIMUM OPERATIONS

```
1 package part4;
2
3 public class RunningTimeDemo {
4     public static void main(String[] args) {
5         System.out.println(s(10,10,10));
6     }
7 }
8 static int s(int a, int b, int c)
9 {
10     if(a>b)
11         c=a;
12     if(b>c)
13         c=b;
14     if(c>a)
15         c=c;
16     return c;
17 }
18
19 }
```

Expression:

a = 10, b = 10, c = 10

Step-by-step:

1 First if (a > b)

- Check: $10 > 10$
- This is false
- So the statement $c = a$ does not execute

2 Second if (b > c)

- Check: $10 > 10$
- This is false
- So the statement $c = b$ does not execute

3 Third if (c > a)

- Check: $10 > 10$

- This is false
 - So the statement $c = c$ does not execute
-

4 return c

- c is still 10
 - So the function returns 10
-

Operation counting

- 3 comparisons $\rightarrow a > b, b > c, c > a$
- 0 assignments (because all conditions are false)
- 1 return statement

Total operations executed

$3 + 0 + 1 = 4$ operations

This is the **minimum possible operations** for this program.

Time Complexity = $O(1)$ *//(constant time)*

2. linear $O(n)$


```
1 package part4;
2
3 public class RunningTimeDemo {
4     public static void main(String[] args) {
5         System.out.println(sum(100));
6     }
7 }
8
9 static int sum(int n)
10 {
11     int s=0;
12     for(int i=0;i<n;i++)
13         s=s+i;
14     return s;
15 }
16 }
17
```

We will find:

- ✓ number of operations
- ✓ minimum & maximum (here both same)
- ✓ time complexity
- ✓ short explanation

Step-by-step operation counting

1) Initialization

int s = 0; → 1 operation

int i = 0; → 1 operation

2) Loop condition check

i < n happens:

- **$n + 1$ times**
(because one extra false check when loop ends)

→ **$n + 1$ operations**

3) Increment

$i++$ → n operations

4) Inside loop

$s = s + i$
= addition → n operations
+ assignment → n operations
So inside loop = **$2n$ operations**

5) Return

return s ; → 1 operation

Operation counting

- 3 comparisons → $a > b$, $b > c$, $c > a$
- 0 assignments (because all conditions are false)
- 1 return statement

Total operations executed

Add everything:

1 ($s = 0$)
+ 1 ($i = 0$)

+ (n + 1) comparisons

+ n increments

+ 2n inside loop

+ 1 return

= 1 + 1 + n + 1 + n + 2n + 1

= 4n + 4 operations

So exact operation count = $4n + 4$.

For n = 100

$$4(100) + 4 = 404 \text{ operations}$$

Time Complexity = $O(n)$

Since operations grow **proportionally to n**:

3. $O(n^2)$

```

1 package part4;
2
3 public class RunningTimeDemo {
4     public static void main(String[] args) {
5         System.out.println(sum(10));
6     }
7 }
8
9 static int sum(int n) //
10 {
11     int s=0;
12     for(int i=1;i<=n;i++)
13     {
14         for(int j=1;j<=n;j++)
15         {
16             s=i+j;
17         }
18     }
19     return s;

```

Operation counting like your style

◆ Step 1 — constant

statements `int s = 0;` // 1

operation `return s;` // 1

operation

So far:

= 2 operations

◆ Step 2 — outer loop

`for(int i = 1; i <= n; i++)`

Operations inside loop header:

- initialization → `i = 1` → 1
- comparison `i <= n` → happens **`n + 1`** times
- increment `i++` → happens **`n`** times

Total outer loop control:

$$1 + (n + 1) + n = 2n + 2$$

◆ **Step 3 — inner loop**

for(int j = 1; j <= n; j++)

For **each i**, inner loop runs **n times**

Inner loop control operations:

- initialization $j = 1 \rightarrow 1$
- comparison $j \leq n \rightarrow (n + 1)$ times
- increment $j++ \rightarrow n$ times

So for one outer loop run:

$$1 + (n + 1) + n = 2n + 2$$

But outer loop itself runs **n times**, so multiply:

$$n(2n + 2) = 2n^2 + 2n$$

◆ **Step 4 — statement inside inner loop**

$s = i + j;$

Includes:

- addition $\rightarrow 1$
- assignment $\rightarrow 1$

So:

2 operations per iteration

Number of executions:

$$n \times n = n^2$$

So total:

$2n^2$ operations

Now add everything

outer loop control: $2n + 2$

inner loop control: $2n^2 + 2n$

inner statement: $2n^2$

constants: 2

TOTAL = $4n^2 + 4n + 4$ operations

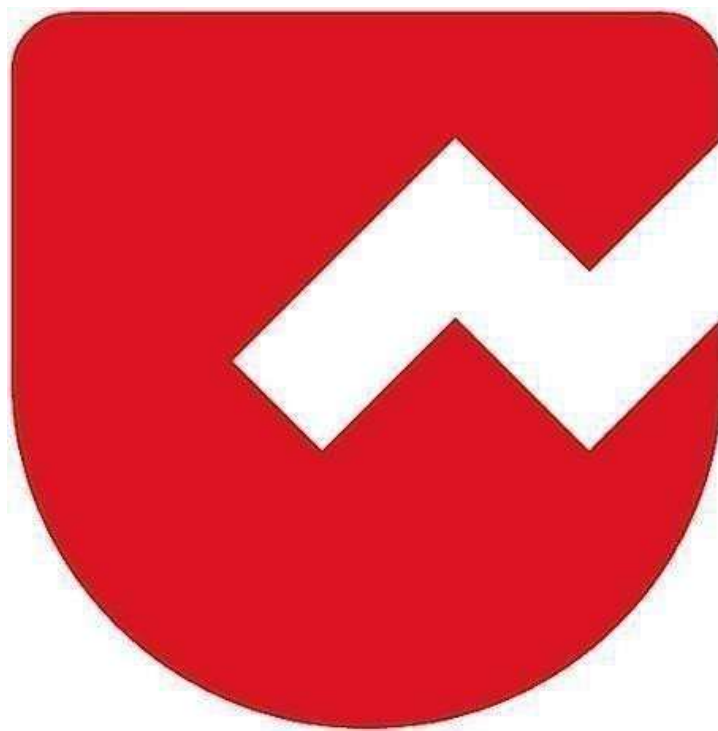
Time Complexity

Since the number of operations grows proportional to n^2 (because of the two nested loops), the time complexity is $O(n^2)$.

Time Complexity = $O(n^2)$

Zoho Schools for Graduate Studies

Notes



Day-41

Computation of Running Time - Quizzes

Quiz 1

Find the Time complexity:

```
for(int i=1; i<n; i=i*2)  
    System.out.println("Hello");
```

- a) $O(n)$
- b) $O(\sqrt{n})$
- c) $O(\log n)$
- d) $O(n^2)$

Quiz 1 - Answer

Find the Time complexity:

```
for(int i=1; i<n; i=i*2)  
    System.out.println("Hello");
```

- a) $O(n)$
- b) $O(\sqrt{n})$
- c) $O(\log n)$
- d) $O(n^2)$

Quiz 1 - Explanation

Find the Time complexity:

```
for(int i=1; i<n; i=i*2)
    System.out.println("Hello");
```

Explanation:

- 'i' gets incremented as $1(2^0), 2, 2^2, 2^3, \dots, 2^k$
- Suppose if the loop gets terminated after k iterations, then

$$2^k > n$$

$$k > \log(n)$$

- Hence, time complexity = $O(\log n)$

Rules for Algorithmic Analysis

1. Count the number of **basic operations** in each statement (Instruction)
2. Conditional statements : take the longer path
3. Functions : count the statements
4. Loops : count the number of iterations
5. Nested loops : similar to loops
6. Each statement should take constant time
 1. Add, sub, shift, incre/decre : 1 clock cycle
 2. Mul, Div : 1 clock cycle
 3. Memory access : 1 clock cycle
 4. Branch, Return : 3 clock cycle

Quiz 2

Find the total basic operations:

```
def fact(n):  
    if n==1:  
        return 1  
    else:  
        return fact(n-1)*n
```

+

- a) $6n+4$
- b) $6n-2$
- c) $9n+4$
- d) $9n-5$

Quiz 2 - Answer

Find the total basic operations:

```
def fact(n):  
    if n==1:  
        return 1  
    else:  
        return fact(n-1)*n
```

+

- a) $6n+4$
- b) $6n-2$
- c) $9n+4$
- d) $9n-5$

Quiz 2 - Explanation

Find the total basic operations:

```
def fact(n):          -> T(n)
    if n==1:          -> 1
        return 1      -> 3
    else:              -> 0
        return fact(n-1)*n -> 3 + 31 + T(n-1) + 1 + 1
```

$T(n) = T(n-1) + 9$, $n > 1$ and $T(1)=4$

Quiz 2- Solving Recurrence Relation

Find the total basic operations:

$$T(n) = T(n-1) + 9, n > 1$$

$$T(n-1) = T(n-2) + 9$$

$$T(n-2) = T(n-3) + 9$$

$$T(2) = T(1) + 9$$

$$T(1) = 4$$

Adding both LHS and RHS, we get

$$\begin{aligned} T(n) &= 9(n-1) + 4 \\ &= 9n - 5 \end{aligned}$$

Quiz 3

Find the recurrence relation:

```
def fibRec(n):  
    if n==0:  
        return 0  
    elif n==1:  
        return 1  
    else:  
        return fibRec(n-1)+fibRec(n-2)
```

Quiz 3 - Options

- a) $T(n) = 15 + T(n-1) + T(n-2), n > 1$
- b) $T(n) = 10 + T(n-1) + T(n-2), n > 1$
- c) $T(n) = 12 + T(n-1) + T(n-2), n > 1$
- d) $T(n) = 14 + T(n-1) + T(n-2), n > 1$

Quiz 3-Answer

- a) $T(n) = 15 + T(n-1) + T(n-2), n > 1$
- b) $T(n) = 10 + T(n-1) + T(n-2), n > 1$
- c) $T(n) = 12 + T(n-1) + T(n-2), n > 1$
- d) $T(n) = 14 + T(n-1) + T(n-2), n > 1$

Quiz 3 - Explanation

```
def fibRec(n):          -> T(n)
    if n==0:            -> 1
        return 0        -> 3
    elif n==1 :        -> 1
        return 1        -> 3
    else:
        return fibRec(n-1)+fibRec(n-2)
    -> 3+ (3+1) + T(n-1) + (3+1) + T (n-2) + 1
    T(n) = 14 + T(n-1) + T(n-2) , n>1
    T(0) = 1 + 3 =4
    T(1) = 1 + 1+ 3 =5
```

Quiz 4

Solve the recurrence relation:

$$T(n) = 14 + T(n-1); T(0) = 4; T(1) = 5$$

Quiz 4 - Answer

Solve the recurrence relation:

$$T(0) = 4; T(1) = 5$$

$$T(n) = 14 + T(n-1)$$

$$T(n-1) = 14 + T(n-2)$$

$$T(n-2) = 14 + T(n-3)$$

.....

$$T(2) = 14 + T(1) +$$

$$T(n) = 14 T(n-1) + 5 - O(n)$$

Quiz 5

Find the recurrence relation:

```
def towersOfHanoi(src,tmp,dst,n):  
    if (n>0):  
        towersOfHanoi(src,dst,tmp,n-1)  
        print(src,"to",dst)  
        towersOfHanoi(tmp,src,dst,n-1)
```

- a) $T(n) :::: 8 + 2T(n-1), n>0$
- b) $T(n) :::: 10 + 2 T(n-1), n>0$
- c) $T(n) = 12 + 2T(n-1), n>0$
- d) $T(n) = 14 + 2T(n-1), n>0$

Quiz 5 -Answer

Find the recurrence relation:

```
def towersOfHanoi(src,tmp,dst,n):  
    if (n>0):  
        towersOfHanoi(src,dst,tmp,n-1)  
        print(src,"to",dst)  
        towersOfHanoi(tmp,src,dst,n-1)
```

- a) $T(n) = 8 + 2T(n-1), n>0$
- b) $T(n) = 10 + 2 T(n-1), n>0$
- c) $T(n) = 12 + 2T(n-1), n>0$
- d) $T(n) = 14 + 2T(n-1), n>0$

QUIZ 6

Find the time complexity:

```
for(int i=1; i*i<=n; i++)  
    System.out.println(i);
```

- a) $n\sqrt{n}$
- b) n
- c) $n/2$
- d) $\sqrt{n}$

QUIZ 6 - Answer

Find the time complexity:

```
for(int i=1; i*i<=n; i++)  
    System.out.println(i);
```

- a) $n\sqrt{n}$
- b) n
- c) $n/2$
- d) $\sqrt{n}$

QUIZ 6 - Explanation

Find the time complexity:

```
for(int i=1; i*i<=n; i++)  
    System.out.println(i);
```

Answer: $O(\lfloor \sqrt{n} \rfloor)$ { Floor value of $\sqrt{n}$ }

Explanation:

$$i^2 \leq n$$

$$i \leq \sqrt{n}$$

QUIZ 7

Find the time complexity:

```
for( int i=0 ; i<=n ; i++) {  
    for(int j=1;j<=i;j++) {  
        for(int k=1;k<=100;k++)  
            System.out.println("Hi");  
    }  
}
```

a) $n^2\sqrt{n}$

b) n^3

c) $n^3 \sqrt{n}$

d) n^2

Answer : n^3

QUIZ 8

Find time complexity:

```
While(n>1)  
    n=n/2;
```

- a) $n/2$
- b) $\log_2 n$
- c) n
- d) $\log_2 n - 1$

1

QUIZ 8 - Answer

Find time complexity:

```
While(n>1)  
    n=n/2;
```

- a) $n/2$
- b) $\log_2 n$
- c) n
- d) $\log_2 n - 1$

QUIZ 8 - Explanation

Explanation:

if $n=2^k$, then, $k=\log_2 n$
Therefore the loop runs $\log_2 n$ times.

QUIZ 9

Find number of basic operations:

```
for(int i=1;i<=m;i++){  
  for(int j=1;j<=n;j++){  
    for(int k=1;k<=p;k++){  
      count++;  
    }  
  }  
}
```

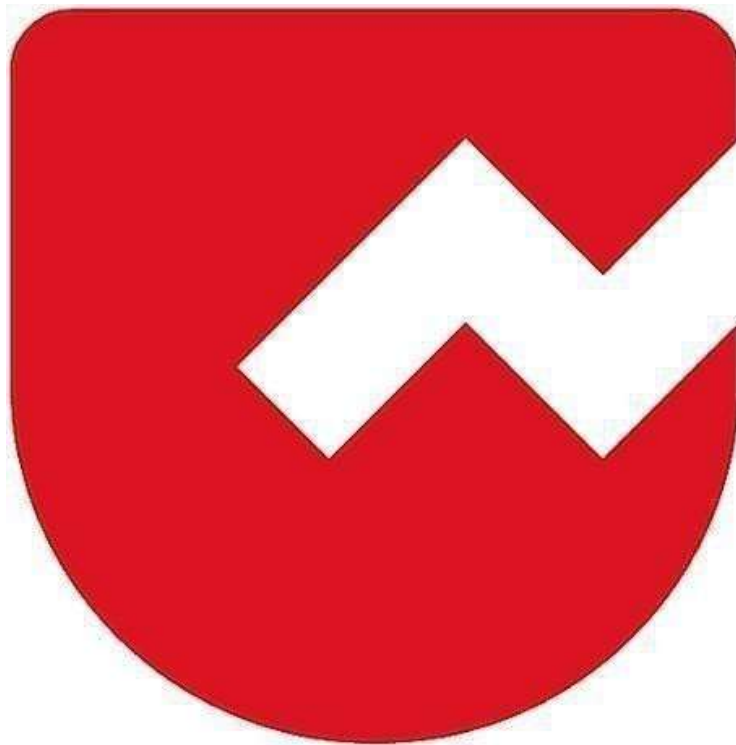
Answer: $3mnp + 4mn + 4m + 2$

QUIZ 9 - Explanation

I	$n=2^k$	$i \leq n$	$n=n/2$	$i++$
0	$2^5=32$	T	16	1
1	$2^4=16$	T	8	2
2	$2^3=8$	T	4	3
3	$2^2=4$	T	2	4
4	$2^1=2$	F		



Zoho Schools for Graduate Studies



Notes

Day-42

Static

- It belongs to class and it have class level access.
- Values / data doesn't change from object to object , it should be declare with static keyword.
- It can be accessed anywhere on the block or class
- If we want to access access outside the class,

ClassName.StaticMember

When to you use static ?

- Common properties for all objects.
- To store single copy in a class.

Real Time Example for static usage :

- DB Connections
- Network Sockets

Static Block :

- Static blocks will execute whenever the class gets loaded for the very first time in the memory .
- Static block was executed before main method and object creation during class loading .
- If there are multiple static blocks present, they will execute in which the order they have present
- Static blocks doesn't need object creation

Can a constructor be static ?

No , It is object specific , we can't make it as static.

Non – static Block :

- It is also called as instance initializer.
- It runs prior to the constructor ie) it runs before the constructor and after the object creation.

When to use Instance Initializer ?

Instance Initializer initializes instance variables before the constructor runs

Feature	Static Block	Non-Static Block (Instance Initializer)
When it runs	When the class is loaded	When an object is created
Number of times	Once only	Every time an object is created
Purpose	Initialize static variables	Initialize instance variables
Syntax	<code>static { }</code>	<code>{ }</code>
Executes before	Before any object creation	Before constructor
Access	Can access only static members	Can access static and non-static members
Belongs to	Class	Object
Use case	Load configuration, static data	Common initialization for all constructors

Code Snippets :

1) package part1;

```
public class StaticDemo {
```

```
public static void main(String[] args) {  
    System.out.println("Inside main method !");  
}  
  
static {  
    System.out.println("Inside static block!");  
}  
}
```

Output :

Inside static block!

Inside main method !

Explanation :

Static block executes first because it runs during class loading, before the main method is executed.

2) package part1;

```
public class StaticDemo {  
  
    public static void main(String[] args) {  
        System.out.println("Inside main method !");  
    }  
  
    static {
```

```
        System.out.println("Inside static block 1!");
    }

    static void m() {
        System.out.println("Inside static method");
    }

    static {
        System.out.println("Inside static block 2!");
        m();
    }
}
```

Output:

Inside static block 1!

Inside static block 2!

Inside static method

Inside main method !

Explanation :

Static blocks execute first during class loading in top-to-bottom order, and only after that the main method runs.

3) package part1;

```
public class StaticDemo {
```

```
    StaticDemo() {
```

```
        System.out.println("Inside main method !");
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        System.out.println("Inside main method !");
```

```
    }
```

```
    static {
```

```
        System.out.println("Inside static block 1!");
```

```
    }
```

```
    static void m() {
```

```
        System.out.println("Inside static method");
```

```
    }
```

```
    static {
```

```
        System.out.println("Inside static block 2!");
```

```
        // m();
```

```
    }
```


Output :

Inside static block 1!

Inside static block 2!

Inside main method !

Explanation :

Static blocks execute during class loading, constructor executes only when an object is created, and main method runs after static blocks.

4) package part1;

```
public class StaticDemo {
```

```
    StaticDemo() {
```

```
        System.out.println("Inside constructor !");
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        new StaticDemo();
```

```
        System.out.println("Inside main method !");
```

```
    }
```

```
    static {
```

```
        System.out.println("Inside static block 1!");
```

```
    }
```

```
static void m() {  
    System.out.println("Inside static method");  
}  
  
static {  
    System.out.println("Inside static block 2!");  
}  
}
```

Output :

Inside static block 1!
Inside static block 2!
Inside constructor !
Inside main method !

Explanation :

Static blocks run first when the class loads, constructor runs when an object is created inside main, and then the remaining main code executes.

5) package part1;

```
public class StaticDemo {
```

```
// Non-static block 1 (Instance Initializer)
{
    System.out.println("Inside Non-static block 1!");
}
```

```
// Non-static block 2 (Instance Initializer)
{
    System.out.println("Inside Non-static block 2!");
}
```

```
// Constructor
StaticDemo() {
    System.out.println("Inside constructor");
}
```

```
public static void main(String[] args) {
    System.out.println("Inside main method !");
    new StaticDemo();
}
```

```
// Static block 1
static {
    System.out.println("Inside static block 1!");
}
```

```
// Static method (not called)
static void m() {
    System.out.println("Inside static method");
}

// Static block 2
static {
    System.out.println("Inside static block 2!");
}
}
```

OutPut :

Inside static block 1!

Inside static block 2!

Inside main method !

Inside Non-static block 1!

Inside Non-static block 2!

Inside constructor

Explanation :

Static blocks execute during class loading, non-static blocks execute during object creation before the constructor, and main runs after static blocks.

6) package part1;

```
public class StaticDemo {
```

```
    // Non-static block 1
```

```
    {
```

```
        System.out.println("Inside Non-static block 1!");
```

```
    }
```

```
    // Non-static block 2
```

```
    {
```

```
        System.out.println("Inside Non-static block 2!");
```

```
    }
```

```
    // Constructor
```

```
    StaticDemo() {
```

```
        System.out.println("Inside constructor");
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        System.out.println("Inside main method !");
```

```
        new StaticDemo(); // first object
```

```
        new StaticDemo(); // second object
```

```
    }
```

```
// Static block 1
static {
    System.out.println("Inside static block 1!");
}

// Static method (not called)
static void m() {
    System.out.println("Inside static method");
}

// Static block 2
static {
    System.out.println("Inside static block 2!");
}
}
```

Output :

Inside static block 1!

Inside static block 2!

Inside main method !

Inside Non-static block 1!

Inside Non-static block 2!

Inside constructor

Inside Non-static block 1!

Inside Non-static block 2!

Inside constructor

Explanation :

Static blocks execute once during class loading, non-static blocks and constructors execute every time an object is created.